September 2, 2024
Milan, Italy

Association for Computing Machinery

Advancing Computing as a Science & Profession

FARM '24

Proceedings of the 12th ACM SIGPLAN International Workshop on

Functional Art, Music, Modelling, and Design

Edited by:
Mae Milano and Stephen Taylor

Sponsored by:
ACM SIGPLAN

Co-located with:
ICFP '24

Association for Computing Machinery, Inc.
1601 Broadway, 10th Floor
New York, NY 10019-7434
USA

Copyright © 2024 by the Association for Computing Machinery, Inc (ACM). Permission to make digital or hard copies of portions of this work for personal or classroom use is granted without fee provided that the copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted.

To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permission to republish from: Publications Dept. ACM, Inc.
Fax +1-212-869-0481 or E-mail permissions@acm.org.

For other copying of articles that carry a code at the bottom of the first or last page, copying is permitted provided that the per-copy fee indicated in the code is paid through the Copyright Clearance Center, 222 Rosewood Drive, Danvers, MA 01923, USA.

ACM ISBN: 979-8-4007-1099-5

Cover photo:
Title: "The Milan cathedral in the evening"
Photographer: Marco Nürnberger, 2016
https://www.flickr.com/photos/mnuernberger
License: Creative Commons Attribution 2.0 Generic
https://creativecommons.org/licenses/by/2.0/deed.en
Cropped from original:
https://commons.wikimedia.org/wiki/File:Milan_Cathedral_%28Duomo_di_Milano%29_evening.jpg

Production: Conference Publishing Consulting
D-94034 Passau, Germany, info@conference-publishing.com

Welcome from the Chairs

It is our pleasure to welcome you to FARM 2024, the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modeling, and Design! FARM gathers together people from across disciplines who are harnessing functional techniques in the pursuit of creativity and expression.

By gathering researchers, practitioners, artists, designers and anyone interested, FARM aims to develop this interdisciplinary field. Work in this challenging area has already led to new language designs, abstraction techniques, and execution models, as well as the use of old ideas in new, novel, and surprising ways.

The first FARM was held in Boston in 2013, and since then, FARMers have gathered in Gothenburg, Vancouver, Nara, Oxford, St Louis, Berlin, Seattle, and Ljubljana, in addition to two online-only meetings in 2020 and 2021. The 2024 edition of FARM will take place in Milan, Italy on September 2nd.

FARM 2024 experienced a surge in the number of submissions relative to recent years. As in past years, we solicited both full papers and demonstrations with associated extended abstracts. This cycle, we received 19 total submissions, including 6 demos and 13 full papers. After thorough peer review with all submissions receiving 2 independent reviews, we accepted 12 submissions, including 8 full papers and 4 demonstrations. The final versions of these papers and demonstration abstracts can all be found in this volume. This year, our proceedings has features a majority of papers on musical topics; we also include papers on geometry and artwork.

As in previous editions of FARM, FARM 2024 has scheduled an evening of performances, headlined by a keynote speaker. This year, we have secured the Auditorium San Fedele, a venue in the center of Milan, to host our performance evening. We will kick things off with a Keynote Presentation by Dmitri Tymoczko (Princeton), and continue with a series of diverse performances combining music, visuals, interaction, and technology. We encourage you to attend what should be a very memorable evening.

Further information about FARM is available on our website, https://functional-art.org.

We would like to thank ACM and the organizers of ICFP 2024 for their support. Thanks also to the members of the FARM Program Committee, and to everyone who submitted papers, demos, and performance proposals. And, of course, we are grateful to all of the participants. We hope that you will enjoy the workshop as much as we know we will.

Welcome to the FARM!

Mae Milano, *General Chair*

Stephen Taylor, *Program Chair*

FARM 2024 Workshop Organization

Workshop Chair: Mae Milano *(Princeton University)*

Program Chair: Stephen Taylor *(University of Illinois Urbana-Champaign)*

Program Committee: Mae Milano *(Princeton University)*
Stephen Taylor *(University of Illinois School of Music)*
Kenichi Asai *(Ochanomizu University)*
Jose Calderon *(Galois, inc.)*
Kerry Hagan *(University of Illinois Urbana-Champaign)*
Jay McCarthy *(Reach)*
Dmitri Tymoczko *(Princeton University)*

Contents

Frontmatter

Keynote

SESSION 1

SESSION 2

SESSION 3

Refactoring Musical Thought (Keynote)

Dmitri Tymoczko
Princeton University
Santa Fe Institute
USA
dmitri@princeton.edu

Abstract

My talk will introduce some twenty-first century music-theoretical ideas that I have found helpful musically, beginning with scales and macroharmony, proceeding through voice-leading geometry, and concluding with the idea of the "quadruple hierarchy", a recursive nesting of collections supporting the same basic operations at each level. I will introduce Arca, a new musical programming language, and will make the case that musicians have made intelligent but tacit use of nonobvious mathematics. The core ideas will be illustrated with live improvisations that use the internet to transmit notation in real time, allowing for both spontaneity and coordination.

ACM Reference Format:
Dmitri Tymoczko. 2024. Refactoring Musical Thought (Keynote). In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy.* ACM, New York, NY, USA, 1 page. https://doi.org/10.1145/3677996.3689597

Biography

Dmitri Tymoczko is a composer, music theorist, programmer, and improviser. He is Professor of Music at Princeton University and external professor at the Santa Fe Institute. He is the author of two books, A Geometry of Music (2011) and Tonality: an owner's manual (2023), numerous articles, including the first two music-theory articles ever published by Science, and four CDs.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
FARM '24, September 2, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3689597

Using Functional Reactive Programming for Robotic Art: An Experience Report

Eliane Irène Schmidli
OST University of Applied Sciences of Eastern Switzerland
Rapperswil, Switzerland
eliane.schmidli@ost.ch

Farhad Mehta
OST University of Applied Sciences of Eastern Switzerland
Rapperswil, Switzerland
farhad.mehta@ost.ch

Abstract

The control software for robotics applications is usually written in a low-level imperative style, intertwining the program sequence and commands for motors and sensors. To describe the program's behavior, it is typically divided into different states, each representing a specific system condition. This way of programming complicates the comprehension of the code, making changes to the program flow a tedious task. Functional Reactive Programming (FRP) offers a composable, modular approach for developing reactive applications. To examine the strengths and limitations of FRP compared to the conventional imperative style, the control software for a robotic artwork was implemented using a form of FRP in the Haskell programming language. The resulting design separates the control of the hardware from the implementation of its behavior. In addition, state transitions are presented more clearly, resulting in code that is more understandable, especially for people with little programming experience.

CCS Concepts: • **Software and its engineering** → **Domain specific languages**; • **Applied computing** → *Media arts*; *Performing arts.*

Keywords: Functional Reactive Programming, Robotics

ACM Reference Format:
Eliane Irène Schmidli and Farhad Mehta. 2024. Using Functional Reactive Programming for Robotic Art: An Experience Report. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3677996.3678288

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.

FARM '24, September 2, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3678288

1 Introduction

The conventional approach to implementing control software for robotic art is often based on encoding a state machine in a low-level imperative language, whereby commands are sent to motors and sensors based on the system's current state. This way of programming limits the complexity of what can be done and leads to challenges in understanding where state transitions are triggered, making changes to the code error-prone. Additionally, it seems that programming in this style is not something that artists are confident about or enjoy.

The main motivation of this work was to explore how well functional programming would be received by artists: Given that artists have a strong appreciation of beauty and elegance, could it be true that the area of art is well suited to the use of functional programming, which similarly emphasizes simplicity, beauty, and elegance? The application of functional programming for art has a rich history [3, 5, 12].

FRP is a composable, modular alternative for programming reactive applications. In FRP, instead of sending commands directly to motors, a signal is created that represents the current behavior of the artwork. This signal is dynamically adapted based on inputs such as sensor data and can be interpreted by various outputs. For example, the signal can indicate the current positions to which the motors of the real artwork should move. Alternatively, the positions of the same signal can be used in a simulation whereby the Graphical User Interface (GUI) places the elements of the artwork accordingly.

To illustrate the application of FRP in the programming of robotic art, a design for the control software of the artwork Edge Beings was developed. The concept for this artwork was created by Pors & Rao [15] and is currently being realized. The idea features a composition of different-sized panels mounted on walls (see Figure 1). Over time, round, black silhouettes of creatures move slowly over the edges of the panels.

Close observation reveals that there are social constellations among the beings. They belong to different, overlapping groups, of which only one can be in view at a time. Each group has a suppressor, displacing weaker groups to make room for its own group. If the suppressor is weaker than the currently visible group, it must retreat.

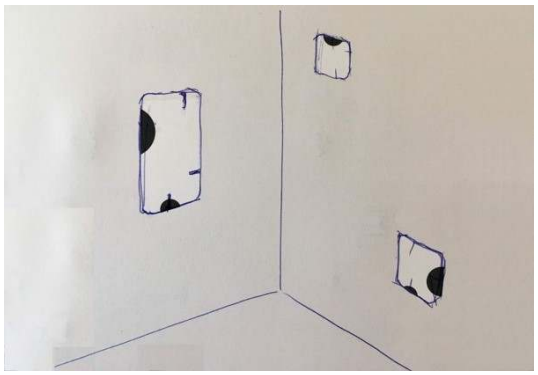


Figure 1. A sketch of Edge Beings by Søren Pors [15]. © Søren Pors, Pors & Rao.

When viewers closely approach the artwork, they observe the beings becoming aware of their presence. In response, the beings behave more nervously, refraining from emerging fully. Instead, they peek briefly over the edge before hiding again. As soon as no one is too close anymore, the creatures resume their normal behavior.

A special feature of Pors & Rao's artworks[1] is the lifelike appearance of the creatures' movements. To achieve this, they develop complex behavioral algorithms, which are neglected in this paper to maintain focus on the concepts of FRP and its significance and implications. Instead, a simplified design was created and implemented once in imperative style and once in FRP style using Yampa [6] (see Sections 3.1 and 3.2). Section 2 provides an initial introduction to the concept of FRP and the use of Yampa. The resulting code design in FRP was discussed with the artists (see Section 3.3).

2 FRP and Yampa

This section provides an overview of the key concepts in FRP and Yampa [6] that are relevant to understand the content of this paper.

2.1 FRP

FRP is a style of programming for implementing reactive systems using the principles of functional programming. Reactive systems are systems whose behavior depends on the occurrence of external events. Functional programming is defined as programming using pure (mathematical, side-effect free) functions and values elegance, algebraic compositionality and mathematical precision over quick wins in speed, space and size of user base. Therefore, FRP proposes a declarative approach to implement the behaviors of reactive systems.

A simplified example of how to think of FRP is a spreadsheet. A spreadsheet consists of cells that can contain values or functions. For example, a function can check whether the value in a particular cell is larger than the value in another particular cell. When programming the function, the user describes the dependency of the result on the two cells (`result = cell1 > cell2`). If the value in a cell is changed, the result is automatically updated. The refreshing is done by the spreadsheet program. In imperative programming, the refreshing of the values has to be specified by the programmer [1, Chapter 1].

In 1997, the concept of FRP was proposed by Conal Elliott and Paul Hudak [2], and implemented in the Fran system. Fran is a domain-specific language (DSL) for animations in Haskell. Fran makes it possible to separate the presentation of animations from their descriptions.

Two years later, these concepts were applied to Frob [13], a DSL for use in robotic systems. Frob hides the details of low-level robot programming and makes programming more hardware independent. Unlike Fran, Frob must also manage hardware control, which adds an additional layer of complexity.

2.2 Yampa

Yampa is an FRP implementation inspired by Fran and Frob. Yampa has been used in various areas such as robotics, GUI applications, and games [6]. For instance, Pembeci, Nilsson, and Hager [11] have built vision-guided, semi-autonomous robots with FRP using Yampa.

2.2.1 Signals and Events.

The most important concepts of FRP in Yampa are `Signals` and `Events`. A `Signal` is a continuous, time-varying value. It can be understood as a mapping `Time -> a` of a value of type `Time` to a value of type `a`.

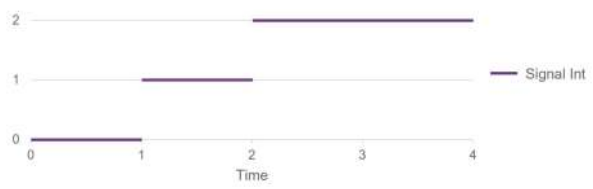


Figure 2. An example of an integer `Signal` that changes its value over time.

Figure 2 visualizes an example of an integer `Signal`. A `Signal` always contains a value at each point in time. An FRP program describes how the value of a `Signal` changes in response to events.

A stream of events is modeled by `Signal (Event b)` in Yampa, which is a `Signal` that yields either nothing or an `Event` with a value of type `b`. The value of type `b` is generated each time the event occurs. For example, the current position of the mouse can be attached to the event. A visualization

[1] An insight into the artworks and working methods of Pors & Rao can be found here [4, 16].

of an event stream `Signal (Event Int)` can be found in Figure 3.

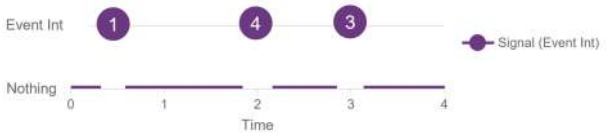


Figure 3. An example of a `Signal` that yields nothing or an `Event` containing a value of type `Int`.

The concept of `Signal` enables the writing of programs that have time and space leaks. The reason for this is explained in [6, 9, 14]. Yampa solves this problem by not allowing Signals as first-class values. The value contained in a `Signal` can only be modified using signal functions. These cannot be constructed directly, but only using a set of given combinators. The given combinators ensure that time and space leaks are avoided. In Yampa, this concept was implemented with the help of arrows, a generalization of monads proposed by John Hughes [7].

A signal function `SF a b` is an instance of the `Arrow` class and can be thought of as a mapping `Signal a -> Signal b` of Signals to Signals.

2.2.2 Example. To better understand how signal functions work, this section provides a small example. It creates an animation of a straightforward motion of an object. The animation can be started and controlled using mouse clicks. Depending on whether the left or the right button is clicked, the current speed is slow or fast. The result is a signal that represents the position. Its value should be zero at the beginning and after a mouse click, increase with the corresponding speed. The resulting signal can be used to create an animation of an arbitrary object as shown in Figure 4.

Figure 4. Animation of a star depending on the position `pos`. The direction is determined by the definition of the coordinates in the GUI. E.g. the coordinates on the left correspond to a movement along the x-axis, whereas those on the right result in a movement along the y-axis.

Creation of a Position Signal. The example starts with the creation of a signal function with `constant` to represent the velocity. It is of type `constant :: b -> SF a b` and creates a signal function providing an output signal with a constant value.

```
1  type Velocity = Double
2
3  velocity :: Velocity -> SF () Velocity
4  velocity v = constant v
```

To calculate the position based on a constant velocity, the value of the input signal must be integrated with the signal function `integral` (see Figure 5).

Figure 5. Visualization of `integral` accumulating a constant input value over time.

To pass the output signal of `velocity` to the `integral` signal function, as in Figure 6, a combinator is needed. One possibility to compose two signal functions is the operator `>>>`.

```
(>>>) :: SF a b -> SF b c -> SF a c
```

Now the function `positionSF` can be defined that will increase the value of the resulting signal steadily. This can be interpreted as a movement with a constant speed `v`.

```
1  type Position = Double
2
3  positionSF :: Velocity -> SF () Position
4  positionSF v = velocity v >>> integral
```

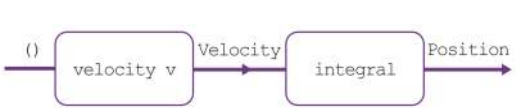


Figure 6. Visualization of `positionSF`. The arrow represents the signal that is modified by the signal functions `velocity` and `integral`.

Start on Mouse Click. We want the movement to start when a mouse click occurs. So, the velocity should first be zero. After the mouse click the velocity should be set to the slow speed (here ten). For this, the function `hold` is needed. This function receives a start value as a parameter and creates a signal function from it. This signal function takes an event stream as input and produces an output signal. In the beginning, the output signal holds the start value. When an event occurs, the value is replaced by the value of the event.

```
hold :: a -> SF (Event a) a
```

The output signal of `hold` should indicate the new speed after the click. For this, the velocity needs to be attached to the event using the function `tag`.

```
tag :: (Event a) -> b -> (Event b)
```

Figure 7 visualizes the transformation of the event stream to the position signal. First, the input signal yields nothing and `hold` produces a signal containing the value zero. Therefore, the output signal of `integral` also indicates zero. When a mouse click event occurs, ten is attached to it, and `hold` changes the value accordingly to ten. After the change, `integral` starts increasing the value of the position.

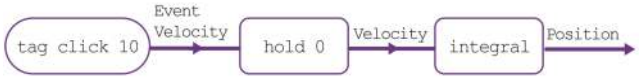


Figure 7. Visualization of `positionSF` transforming the event stream to the position signal. Note that `click` is of type `Event()`.

The new version of `positionSF` receives the mouse event of type `Event()` as an input signal. To simplify the readability of the implementation, the arrow notation is used here. An introduction to the arrow notation can be found in [8].

```
1  positionSF' :: SF (Event ()) Position
2  positionSF' = proc click -> do
3      v <- hold 0 -< tag click 10
4      pos <- integral -< v
5      returnA -< pos
```

The extracted value of the input signal is `click` of type `Event()`. The event is tagged with ten and is passed to the signal function (`hold 0`). This produces an output signal containing the current velocity. The velocity `v` can be passed to the function `integral` which produces the output position `pos`. With `returnA` the value `pos` is wrapped in the outgoing signal.

Switch between Speeds. To switch between two different speeds depending on the left and right click of the mouse, two event streams must be used. This can be realized by changing the input signal of `programSF'` to the type `(Event(), Event())`. Then a different speed is tagged to each of the two click events. To use `hold`, both event streams have to be merged into one. For this, a decision must be made as to which of the two events has priority when both occur simultaneously. With `rMerge`, the right event is always preferred as in Figure 8.

```
rMerge :: (Event a) -> (Event a) -> (Event a)
```

Here is the new implementation of `positionSF'`, where `lbp` corresponds to a left mouse click, and `rbp` to a right mouse click. Each time the left mouse button is pressed, `v` is changed to the value ten, and when the right button is pressed to the value twenty.

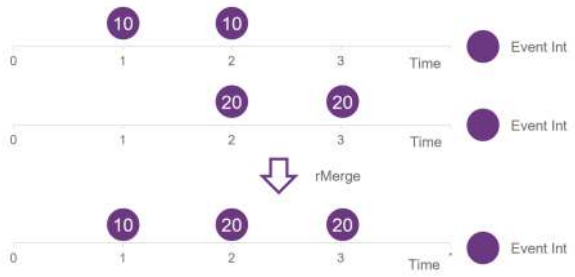


Figure 8. Visualization of `rMerge`. It merges the two event streams preferring the events from the second stream.

```
1  positionSF'' :: SF (Event (), Event()) Position
2  positionSF'' = proc (lbp, rbp) -> do
3      let slowEv = tag lbp 10
4      let fastEv = tag rbp 20
5      v <- hold 0 -< rMerge slowEv fastEv
6      pos <- integral -< v
7    returnA -< pos
```

3 Design of Edge Beings

To illustrate the differences in applying imperative programming versus FRP using Yampa, the following chapters present the behavior of a being in both approaches. The implementation of the Edge Beings artwork shown here is simplified to illustrate the concepts without going into fine details. In particular, the beings are currently assigned to a single group instead of several groups, and their movement patterns are not as complex.

3.1 Imperative Style

In imperative programming, the translation of a problem description into code often involves the creation of a state machine. In the context of the Edge Beings artwork, a state is managed for each being. The main loop adjusts these states at each iteration based on events that have occurred. The code execution for each being is determined by its current state, with the following code specifying actions for each state per iteration. Figure 9 illustrates a state machine representing the code.

```
1  def step():
2    current_time = time.time()
3
4    if state == HIDE:
5      motor.go_to_position(0)
6      set_time_next_move(wait_before_peek)
7      state = PEEK
8
9    elif state == PEEK:
10     if (current_time >= time_next_move):
11       motor.go_to_position(peek_pos)
12       set_time_next_move(wait_before_stand)
13       state = WAIT_PEEK
14
15   elif state == WAIT_PEEK:
16     if (current_time >= time_next_move):
17       if (not motion and is_stronger_suppressor):
```

```
18          suppressor_events.insert(suppressor_rank)
19          state = STAND
20        elif (is_allowed_to_go_out):
21          state = STAND
22        else:
23          state = HIDE
24
25    elif state == STAND:
26      set_time_next_move(out_time)
27      motor.go_to_position(stand_pos)
28      state = WAIT_STAND
29
30    elif state == WAIT_STAND:
31      if (current_time >= time_next_move):
32        if (is_suppressor):
33          current_group = 0
34        state = HIDE
```

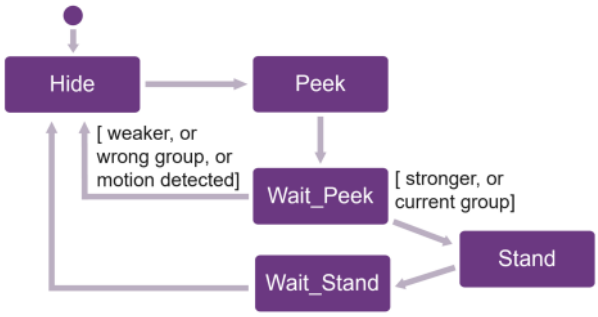


Figure 9. State Machine representing the state transitions for a being of the simplified Edge Beings implementation.

In the `'Hide'` state, a being's motor moves to the hiding position. After the transition to the `'Peek'` state, the being waits and moves to the peeking position after a specified time, followed by waiting again in the `'Wait_Peek'` state. Subsequently, the program checks if the being is permitted to emerge completely or if it must hide again based on the following conditions: If the motion sensor detects a visitor too close to the artwork, no being is allowed to move to the standing position. Otherwise, the code checks if the being is a suppressor. If not, the being can only creep out if it belongs to the currently permitted group. If the being is a suppressor, it can emerge only if it outranks the current group. In this case, the being registers for the suppressor event prompting the current group to hide.

In the `'Stand'` state, the motor moves the being to the standing position. The being then waits in the `'Wait_Stand'` state until eviction or the expiration of its allocated time. In the second case, suppressors reset the current group to zero before retreating, enabling other suppressors to emerge.

The code of the state machine only shows certain state transitions. For example, it is not apparent that all beings are set to the `'Hide'` state when a motion event occurs. For this insight, examining the event handling in the main loop is necessary. The events are gathered in a list which is processed with each main loop iteration. The function `handle_events` invokes the handlers for the current events.

```
1  def handle_events(event_list):
2    handle_suppressor_events(event_list)
3
4    motion_event = last_motion_event(event_list)
5    if is_motion_event(motion_event):
6      handle_motion_events()
7
8    if is_no_motion_event(motion_event):
9      handle_no_motion_events()
```

The function `handle_motion_events` sets all beings to the `'Hide'` state, while the function `handle_suppressor_events` manages the suppressors that were previously registered in the `'Wait_Peek'` state. The function determines the current group by the suppressor with the highest rank. Then, beings beyond the peeking position belonging to the previous group are set to the `'Hide'` state.

The sequence of event handler calls in the function `handle_events` is critical and depends on state changes within the event handlers. To illustrate this, the bad handling of motion events in the following code can be considered.

```
1  def handle_events_wrong(list):
2    # ...
3    motion_event = has_motion_event(list)
4    if motion_event:
5      handle_motion_events()
6
7    no_motion_event = has_no_motion_event(list)
8    if no_motion_event:
9      handle_no_motion_events()
```

Instead of handling only the last movement event, both event handlers are called here. If the sensor emits the three events `Motion`, `No_Motion`, and `Motion`, the bad implementation would inaccurately set the variable `motion` to `False`. The beings are allowed to emerge even if the last event signifies that a visitor is indeed present in the area.

Therefore, understanding the complete dependencies between states and events is crucial to making changes to this imperative implementation. Failure to do so may result in unintended side effects. The next chapter proposes a solution to this using FRP in Yampa.

3.2 Design in Yampa

In FRP it is possible to separate the behavior of the beings and the instructions given to the motors. The behavior is defined within a signal function, generating a signal that indicates the current positions of the beings. This signal can be consumed by motors that move to the corresponding positions. For instance, if a being is instructed to wait, the same position is repeatedly sent during this interval, causing the motors to remain stationary.

The signal function dynamically changes the behavior and therefore the positions of the beings based on the input signals. For instance, when the motion sensor near the artwork detects someone in its proximity, it sends a message to the FRP service. This message is then integrated into the

input signal of the ongoing signal function and used to generate an event. This event triggers a change in the behavior leading to a change in position and motor movement. Then, the viewer of the artwork can observe how the figures are hiding. The beings resume normal movement as soon as the viewer moves out of the range of the sensor.

3.2.1 External Interface. All the panels' motors and the sensors are orchestrated by a Python framework running on a Raspberry Pi. Simultaneously, the FRP implementation is launched on the same Raspberry Pi. Sockets are used to establish a connection between the Python service and the Haskell service, enabling the exchange of input and output information.

This design makes it possible to separate the description of the beings' behavior from the representation of the output. Modifications to the behavior therefore become immediately visible without the need to edit the motor controls. Furthermore, the Python service can be replaced with an alternative solution. Specifically, a simulation was developed for the Edge Beings artwork. This enables users to visually observe the effects of modifications in the program without access to the physical artwork hardware. Figure 10 shows a screenshot of the GUI from the resulting application.

The simulation interfaces with the Haskell service via the same socket connection. Keyboard inputs emulate the motion sensor, while the GUI adjusts the coordinates of the beings instead of controlling physical motors. The shift in positions is perceived by the viewer as an animation.

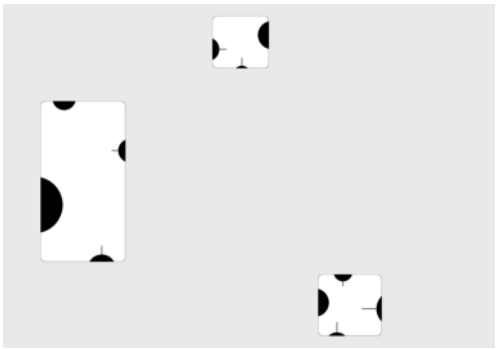


Figure 10. GUI of the Edge Being simulation

The following sections provide a deeper insight into the implementation of the beings' behavior.

3.2.2 Execution. The behavior of the beings within the Haskell service is described in a signal function. In Yampa, this function can be executed indefinitely using the `reactimate` function, enabling the reception of new inputs and the processing of outputs.

```
reactimate :: Monad m
  -- Initialization action
  => m a
  -- Input sensing action
  -> (Bool -> m (DTime, Maybe a))
  -- Output processing action
  -> (Bool -> b -> m Bool)
  -- Signal function
  -> SF a b
  -> m ()
```

Further information about the usage of the `reactimate` function can be found in [10]. For Edge Beings, the function is used as follows:

```
type Pos = Double
type Vel = Double
edgeBeingsBehavior :: SF (Maybe Int) [Pos]

reactimate
  (initializationAction)
  (\_ -> inputSensingAction)
  (\_ positions -> outputProcessingAction positions)
  (edgeBeingsBehavior)
```

The `edgeBeingsBehavior` function first interprets the motion sensor data and transforms it into an event stream of type `SF (Maybe Int) (Event(), Event())`. Then the `edgeBeingsBehavior` function starts the `behavior` signal function for each being and passes the motion event stream as input. In the absence of numeric values from the motion sensor, no event is generated. If a person is detected in the proximity of the artwork, the left event is fired; conversely, the right event is fired if there is no person close.

Each being has its own motion parameters, causing the beings to move differently from each other. These parameters are stored in instances of the `Being` data type:

```
type Time = Double
type Rank = Int
type Group = Int
type Mov = (Pos, Vel)

data Being = Being
  { waitToPeek :: Time,
    waitToStand :: Time,
    waitToGoBack :: Time,
    hiding :: Mov,
    peeking :: Mov,
    standing :: Mov,
    socialGroup :: Group,
    rank :: Rank -- 0 if not suppressor
  }
```

Each being is assigned to a group, and if it belongs to the suppressors, a corresponding rank is determined. The rank of the suppressor corresponds to the priority of its group, enabling it to displace lower suppressors and their groups.

To allow for dynamic changes in the beings' movement parameters and waiting times, the list of `Being` instances is integrated into the input signal. Within the function `edgeBeingsBehavior`, each being receives its own instance, coupled with sensor events.

3.2.3 Beings' Behavior. The beings exhibit two distinct patterns of behaviors: normal social behavior and a disturbed state. During social behavior, the beings appear in groups, with weaker groups being driven away by the suppressors of more powerful groups. If the viewer approaches the artwork

too closely, a shift to the disturbed behavior is triggered. As soon as the viewer reverts to a more distant position, the beings resume their normal social behavior.

This procedure can be represented in Yampa as follows.

```
type StartPos = Pos
type Events = (Event(), Event())

socialBehavior, disturbedBehavior
 :: [StartPos] -> SF [Being] [Pos]

motionEv, noMotionEv :: SF Events (Event ())

behavior
  :: [StartPos]
  -> SF ([Being], Events) [Pos]
behavior startPos =
  socialBehavior startPos `doUntil'` motionEv
  `switch` \pos ->
    disturbedBehavior pos `doUntil'` noMotionEv
  `switch` \pos -> behavior pos
```

First, the `socialBehavior` signal function is executed, until an event in the `motionEv` event stream triggers a switch to the signal function `disturbedBehavior`. An event in the `noMotionEv` event stream leads to a switch back to the initial behavior. The `switch` function enables the transition between these behaviors and is defined by the following type signature:

```
switch
  :: SF a (b, Event c)
  -> (c -> SF a b)
  -> SF a b
```

In this signature, the first parameter represents the signal function initially executed, combined with the event stream that triggers the switch. As soon as an event of type `Event c` occurs, the signal function in the second argument is invoked. The value of the event serves as input for creating the new signal function, allowing for adaptive behavior transitions.

In Yampa, the signal function following the switch begins afresh from time zero [6]. In the context of Edge Beings, the output positions are therefore reset to zero after each switch. To avoid this, the `doUntil'` function retrieves the position values from the running signal function and attaches them to the event from the event stream. These positions are then passed to the new signal function, allowing it to commence from the specified positions.

```
doUntil'
  :: SF a b
  -> SF a' (Event ())
  -> SF (a, a') (b, Event b)
```

3.2.4 Disturbed Behavior.

When the disturbed behavior is invoked, individual signals are generated for each being using the `disturbedBeing` function.

```
integrate :: StartPos -> SF Vel Pos

disturbedBeing :: StartPos -> SF Being Pos
disturbedBeing startPos = proc state -> do
  rec
    let input = (state, noGroup)
    v <- check -< (input, pos)
    pos <- integrate startPos -< v
  returnA -< pos
  where
    noGroup = 0
```

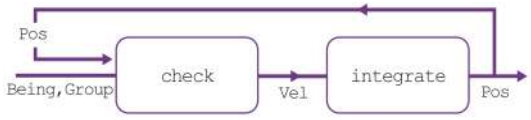


Figure 11. Visualization of `disturbedBeing` transforming the input signal into a position signal.

This function calculates the position of the being by integrating the current velocity, determined by the `check` function. If the being is in motion with a specific velocity, the position is increased accordingly. When the being stops, the velocity becomes zero, and the position remains constant. To stop at a designated target position, the calculated position is fed back to the `check` function (see Figure 11). To enable this, the keyword `rec` is used in the arrow notation.

The `disturbedBeing` signal function shares some functions with the signal function defining the normal behavior. In the latter, an input signal denotes the current group allowed to be present. As group membership does not influence the disturbed behavior, the current group is set to zero using the variable `noGroup`.

The `check` function describes the movement behavior of the disturbed being:

```
type CurrPos = Pos
type Input = ((Being, Int), CurrPos)

check :: SF Input Vel
check = goHiding
    `switch` (\par -> waitBeforePeek par
    `switch` (\_ -> goPeeking
    `switch` (\par -> waitBeforeStand par
    `switch` const check)))
```

At the beginning, the being hides, then it waits for its individual waiting time before moving to the peeking position and waiting there again. The function is called recursively to repeat the behavior from the beginning.

During each movement, a signal indicating the current velocity is generated. As an example, the implementation of `goPeeking` is shown here:

```
moveOut :: (Being -> Mov) -> SF Input Vel
arrivalPeeking :: SF Input (Event Time)

goPeeking, goHiding
  :: SF Input (Vel, Event Time)
goPeeking =
  moveOut peeking `doUntil` arrivalPeeking
```

Using `doUntil`, the signal is combined with the event stream, producing an event as soon as the target position is reached. The `doUntil` function uses the Yampa operator (`&&&`) for this purpose.

```
(&&&) :: SF a b -> SF a b' -> SF a (b, b')

doUntil
  :: SF a b
  -> SF a (Event c)
  -> SF a (b, Event c)
doUntil behavior event = behavior &&& event
```

When an arrival event occurs, the current waiting time is retrieved from the `Being` input signal and is tagged to the generated event. This time value is then used to create a wait signal function. These functions operate similarly to the movement functions, producing a signal indicating zero speed. This signal is then combined with the event stream, which generates an event after the specified time has elapsed.

```
waitBeforePeek, waitBeforeStand
  :: Time -> SF Input (Vel, Event ())
```

3.2.5 Normal Behavior. If no visitor stands in close proximity to the artwork, the beings exhibit their normal social behavior, with distinct actions initialized for suppressors and other beings. A signal of type `Group` signifies the current group and is used as input by normal beings. If a suppressor with a higher rank than the current group appears, it replaces the value of the group signal with its rank.

Normal Being. The `normalBeing` signal function is started for non-suppressors. The function operates similarly to the `disturbedBeing` signal function, with the difference that its input signal contains the current group allowed to be outside.

```
normalBeing :: StartPos -> SF (Being, Group) Pos
normalBeing startPos = proc input -> do
  rec
    v <- normalBehavior -< (input, pos)
    pos <- integrate startPos -< v
  returnA -< pos
```

If the current group aligns with that of the being, it may move out to the standing position. Otherwise, it behaves similarly to the disturbed behavior. This procedure can be seen in the `normalBehavior` function.

```
currentGroupEvents :: SF Input (Event())
notCurrentGroupEvents :: SF Input (Event())

normalBehavior :: SF Input Vel
normalBehavior =
  check `doUntil` currentGroupEvents
    `switch` \_ ->
      (creepOut `doUntil` notCurrentGroupEvents)
    `switch` const normalBehavior
```

When the event stream `currentGroupEvents` generates an event, the behavior switches from `check` to `creepOut`. If a suppressor displaces the group, the input stream's group changes, and `notCurrentGroupEvents` generates an event, restarting the `normalBehavior` recursively.

The `creepOut` function has a similar structure to the `check` function. However, after reaching the peeking position, the being moves to the standing position.

```
waitBeforeGoBack
  :: Time
  -> SF Input (Vel, Event ())
goStanding :: SF Input (Vel, Event Time)

creepOut :: SF Input Vel
creepOut = getWaitingTime
    `switch` (\par -> waitBeforePeek par
    `switch` (\_ -> goPeeking
    `switch` (\par -> waitBeforeStand par
    `switch` (\_ -> goStanding
    `switch` (\par -> waitBeforeGoBack par
    `switch` (\_ -> goHiding
    `switch` const creepOut))))))
```

To retrieve the wait parameter for `waitBeforePeek` at the start of `creepOut`, the `getWaitingTime` step is introduced.

```
getWaitingTime :: SF Input (Velocity, Event Time)
```

It generates an event immediately and reads the required waiting time from the `Being` state. This time value is then tagged to the event and passed to the `waitBeforePeek` function.

Suppressor. The `suppressor` signal function is initiated for each suppressor.

```
suppressor
  :: StartPos
  -> SF (Being, Group) (Pos, Rank)
suppressor startPos = proc input@(state, _) -> do
  rec
    (v, rank') <- supBehavior -< (input, pos)
    pos <- integrate startPos -< v

    isGoingOut <- goingOut -< (v, pos, state)
    let rank = if isGoingOut then rank' else 0

  returnA -< (pos, rank)
```

The distinct feature of the `suppressor` function is that its output signal contains beside the position also the rank, indicating zero unless the suppressor is visible. The ranks of the apparent suppressors are compared, with the highest determining the current group.

Similar to the non-suppressors, the `supBehavior` function defines the movement of the suppressor.

```
supBehavior :: SF Input (Vel, Rank)
```

The `supBehavior` function follows a similar pattern to the other beings' behavior but sets the `rank'` accordingly. During `check`, the output `rank'` is set to zero and during `creepOut`, it corresponds to the rank of the suppressor.

At the end of `creepOut`, the suppressor withdraws. The signal function `goingOut` ensures that the other beings can already emerge when the suppressor is retreating.

```
goingOut :: SF (Vel, Pos, Being) Bool
```

To achieve this, `goingOut` indicates `False` when the suppressor is moving backward. Therefore, the output signal of the `suppressor` function indicates a rank of zero.

3.2.6 Comparison to State Machine. In contrast to the state machine implementation, functions such as `behavior`, `normalBehavior`, or `creepOut` offer a more intuitive representation of the program flow and a better understanding of what reactions are triggered by certain events.

To highlight the advantages of this implementation over a state machine, the introduction of a restart event to the program is considered. In the state machine, all beings must be set to the correct state when the event occurs. This requires careful placement of the event handling to prevent states from being overwritten by other events afterward. These modifications require familiarity with the code of the state machine and the event handler. To enable an immediate restart, additional complexity is introduced by interrupting motors in motion.

In natural language, the problem can be described as executing the behavior until a restart event occurs and then restarting from the beginning. In Yampa, this description can be represented elegantly in the code, as the `restartBehavior` function demonstrates.

```
1  type Events' = (Events, Event())
2
3  restartEvent
4    :: SF ([BeingState], Events') (Event ())
5
6  startPos :: [Pos]
7
8  restartBehavior
9    :: SF ([BeingState], Events') [Pos]
10 restartBehavior =
11   (behavior startPos `doUntil` restartEvent)
12     `switch` const restartBehavior
```

It is only necessary to include the event in the `Events` type of the input signal and to ensure that `behavior` starts with the `goHiding` function. Other events and the deeper implementation of `behavior` do not need to be considered. Implementing changes in this manner reduces the likelihood of errors.

3.3 Interaction with Artists

The design for the control software was developed in close cooperation with the artists. Their developers normally program the artworks in the state machine style as described in Section 3.1. It is therefore not surprising that the artists' written description of the behavioral logic was not a problem description in natural language. Instead, it exhibited a structure that is suitable for programming a state machine.

When the implementation in FRP reached the state described in this paper, both versions of solutions were presented to one of the artists. The explanation of the code was accompanied by the illustration of the state machine shown in Figure 9 and similar graphics to Figure 11. The implementation in Yampa was convincing with its comprehensible state transitions. The artist recognized the potential to make parts of the programmed algorithm easier to understand for people without programming knowledge. This, in turn, offers a certain degree of clarity concerning the developers' understanding of the problem.

It is important to mention while Yampa facilitates comprehensible code, the possibility of producing poorly written and unreadable code still exists. Therefore, in the current state, knowledge of Haskell and Yampa is required to create or make larger modifications to the code. But the improved comprehensibility allows artists to be more involved in this process. It is also conceivable that artists could make minor changes to existing code themselves. However, this would require implementing an interface or code convention that hides certain details.

Further advantages of using FRP in art can be seen in the Edge Beings simulation. The simple addition of the simulation allowed the artists to easily experiment with various movement parameters for the creatures. To facilitate this process, sliders were integrated into the GUI for adjusting the values in the input signal `Being`. The application of FRP enables the dynamic adaptation of values in a signal and thus allows immediate modifications to the beings' movements.

4 Conclusion

The implementation using FRP offers several advantages. The modular architecture makes it easier to supplement the implementation with a simulation, enabling independent testing of code changes without the need for hardware interaction. The use of Yampa provides clear and comprehensible state changes that can also be understood by people with little programming experience. In addition, the real-time response to alterations in movement parameters is a valuable addition to the simulation's functionalities.

The implementation in Yampa requires prior knowledge of Haskell, including some advanced concepts like Arrows. Furthermore, getting started with FRP can be challenging due to varying definitions and frameworks based on different approaches. In contrast, employing a state machine approach is less complex and more standardized, aligning with the general education of programmers.

Although the initial design and major code changes still require knowledge of Haskell and Yampa, the potential for better collaboration and minor customization by the artists themselves was recognized. This suggests that functional programming, with the further development of user-friendly interfaces, could improve the state of the art in the programming of robotic art by allowing artists to better collaborate with programmers, and in the future, possibly bring programming closer to an artist's inherent need to express beauty and elegance in their work.

Acknowledgments

The authors would like to thank Søren Pors and Aparna Rao from Pors & Rao for their support and openness towards

this project, as well as the anonymous reviewers for their valuable comments and suggestions.

References

[1] Stephen Blackheath and Anthony Jones. 2016. *Functional Reactive Programming*. Manning. https://books.google.ch/books?id=aO0zrgEACAAJ

[2] Conal Elliott and Paul Hudak. 1997. Functional Reactive Animation. *SIGPLAN Not.* 32, 8 (aug 1997), 263–273. https://doi.org/10.1145/258949.258973

[3] FARM. 2024. Workshop on Functional Art, Music, Modeling and Design. https://functional-art.org/. Accessed: 2024-05-03.

[4] Google Arts & Culture. 2021. PATHOS: ROBOTIC ANIMATION with Pors & Rao | Google Arts & Culture. https://youtu.be/_SRj-jpwbZ8?si=Y94cS6WexU8d-UoD. Accessed: 2024-05-09.

[5] Paul Hudak. 2000. *The Haskell School of Expression: Learning Functional Programming through Multimedia*. Cambridge University Press.

[6] Paul Hudak, Antony Courtney, Henrik Nilsson, and John Peterson. 2003. Arrows, Robots, and Functional Reactive Programming. In *Advanced Functional Programming: 4th International School, AFP 2002, Oxford, UK, August 19-24, 2002. Revised Lectures*. Springer Berlin Heidelberg, Berlin, Heidelberg, 159–187. https://doi.org/10.1007/978-3-540-44833-4_6

[7] John Hughes. 2000. Generalising monads to arrows. *Science of Computer Programming* 37, 1 (2000), 67–111. https://doi.org/10.1016/S0167-6423(99)00023-4

[8] John Hughes. 2005. Programming with Arrows. In *Advanced Functional Programming*, Varmo Vene and Tarmo Uustalu (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 73–129.

[9] Neelakantan R. Krishnaswami. 2013. Higher-Order Functional Reactive Programming without Spacetime Leaks. *SIGPLAN Not.* 48, 9 (sep 2013), 221–232. https://doi.org/10.1145/2544174.2500588

[10] Henrik Nilsson, Antony Courtney, and Ivan Perez. 2024. Yampa: Elegant Functional Reactive Programming Language for Hybrid Systems . https://hackage.haskell.org/package/Yampa-0.14.8/docs/FRP-Yampa.html#g:28. Accessed: 2024-05-08.

[11] Izzet Pembeci, Henrik Nilsson, and Gregory Hager. 2002. Functional Reactive Robotics: An Exercise in Principled Integration of Domain-Specific Languages. In *Proceedings of the 4th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming* (Pittsburgh, PA, USA) *(PPDP '02)*. Association for Computing Machinery, New York, NY, USA, 168–179. https://doi.org/10.1145/571157.571174

[12] Ivan Perez. 2023. The Beauty and Elegance of Functional Reactive Animation. In *Proceedings of the 11th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design* (<conf-loc>, <city>Seattle</city>, <state>WA</state>, <country>USA</country>, </conf-loc>) *(FARM 2023)*. Association for Computing Machinery, New York, NY, USA, 8–20. https://doi.org/10.1145/3609023.3609806

[13] John Peterson, Paul Hudak, and Conal Elliott. 1999. Lambda in Motion: Controlling Robots with Haskell. In *Proceedings of the First International Workshop on Practical Aspects of Declarative Languages (PADL '99)*. Springer-Verlag, Berlin, Heidelberg, 91–105.

[14] Atze van der Ploeg and Koen Claessen. 2015. Practical principled FRP: forget the past, change the future, FRPNow! *SIGPLAN Not.* 50, 9 (aug 2015), 302–314. https://doi.org/10.1145/2858949.2784752

[15] Søren Pors and Aparna Rao. 2020. Pors & Rao. http://www.porsandrao.com/. Accessed: 2024-01-18.

[16] TED-Ed. 2012. High-tech art (with a sense of humor) - Aparna Rao. https://youtu.be/kJLDl2uDNaA?si=QVFeVN7Tn0ng-UJ7. Accessed: 2024-05-22.

Received 24-MAY-2024; accepted 2024-07-02

Demo: The Fun of Robotic Artwork

Eliane Irène Schmidli
OST University of Applied Sciences of Eastern Switzerland
Rapperswil, Switzerland
eliane.schmidli@ost.ch

Farhad Mehta
OST University of Applied Sciences of Eastern Switzerland
Rapperswil, Switzerland
farhad.mehta@ost.ch

Abstract

Developing software for robot control often is tedious, difficult and error prone. This is more so for people without a computer science background, such as artists. Functional Reactive Programming (FRP) aims to make reactive programs, such as those used for robotics control, more modular, composable, and aesthetically pleasing. We present the results of a collaboration with the art studio Pors & Rao, specializing in robotic artwork. The aim of this collaboration was to explore the use of FRP in this application area to allow artists to express their intentions for robot control in a more effective manner. We demonstrate the artworks that we jointly worked on, as well as our impressions of the acceptance and potential of FRP for artists without a computer science background.

CCS Concepts: • **Software and its engineering** → **Domain specific languages**; • **Applied computing** → *Media arts*; *Performing arts*.

Keywords: Functional Reactive Programming, Robotics

ACM Reference Format:
Eliane Irène Schmidli and Farhad Mehta. 2024. Demo: The Fun of Robotic Artwork. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy*. ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3677996.3678286

1 Introduction

Developing software for robot control is tedious, difficult and error prone. One typically needs to program at the very low level of currents, voltages, or hardware protocols in order to communicate with sensors and actuators. It is therefore common practice to use a low-level programming language such as C for such programs. At the same time, the behavior that is required of the robotic system is complex and nuanced. Programming such behavior at the same low level of abstraction often results in code that is more complex than it needs to be, and is therefore harder to write, understand and maintain. This is especially the case for someone without a computer science or programming background.

Functional Reactive Programming (FRP) is a technique that aims to make reactive programs more modular and composable. It is claimed that the resulting code is not only aesthetically pleasing, but allows for more nuanced, effective and powerful control. The use of functional programming for art has a rich history [1, 3, 5].

In order to practically test these claims, a collaboration with the artist duo Pors & Rao was started.

2 Pors & Rao

The Indo-Danish artist duo Pors & Rao [7] create robotic artworks[1] incorporating physical animation and responsive nuanced behavior. Many of their works are conceived of as 'beings' with basic behaviors such as shyness, fatigue and dependency, highlighting involuntary aspects of human behavior and relationships. Their work is iconic in the field and has been shown in Spain, India, Switzerland, Israel, Austria, Italy, Norway, Japan, South Korea, and the United States.

3 Artworks

First the control of an existing artwork "Untitled" [6] was reimplemented using FRP. This was followed by using the FRP technique to envision and specify the behavior of the artist's next artwork "Edge Beings".

In the Edge Beings artwork, the 'beings' are depicted as black silhouettes that crawl over the edge of different sized panels mounted on walls in a room (see Figure 1). Enhanced by motion sensors, these panels dynamically respond to their environment. When a viewer approaches too closely, the 'beings' react and hide. Instead of crawling out completely, they only look briefly over the edge and give the impression that they want to check whether anyone is still standing there.

On closer inspection, social dynamics between the 'beings' can be recognized. They belong to different groups, of which only one can emerge at a time. Weaker groups are driven away by the suppressors of stronger groups.

[1] An insight into the working methods and artworks of Pors & Rao can be found here [2, 9].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
FARM '24, September 2, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3678286

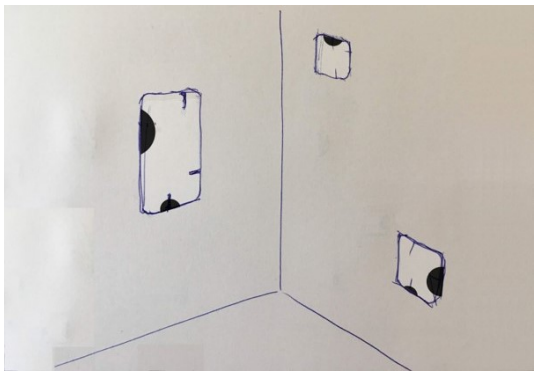


Figure 1. A sketch of Edge Beings by Søren Pors [7]. © Søren Pors, Pors & Rao.

4 Collaboration with the Artists

The implementation of the artwork in FRP was created in close collaboration with the artist duo. Their developers normally use an imperative style. To describe the behavior of the program, it is divided into different states, each representing a specific system condition. It can be challenging to recognize the points where state transitions are triggered. This type of programming therefore complicates the comprehension of the code, making changes to the program flow a tedious task.

The following code shows a simplified implementation of a behavior for a 'being' using the imperative style.

```
def step():
  if state == State.HIDE:
    motor.go_to_position(Pos.HIDE)
    state = State.PEEK
  elif state == State.PEEK:
     motor.go_to(Pos.PEEK)
     if (not motion and is_stronger_suppressor):
       suppressor_events.insert(rank)
       state = State.STAND
     elif (group_is_allowed_to_go_out):
       state = State.STAND
     else:
       state = State.HIDE
  elif state == State.STAND:
    motor.go_to(Pos.STAND)
    state = State.HIDE

def event_handling():
    # Sets states depending on events
```

The new implementation in FRP uses Yampa [4], an FRP implementation in Haskell. Instead of communicating directly with the motors, the FRP approach produces a signal that indicates the current position of the 'beings' during runtime. This signal can be interpreted by the motors, which move to the corresponding position. In this way, the behavior of the creatures is decoupled from the motor control. Although this form of encapsulation would have also been possible in the original imperative code, we assume it was not done since the low level of abstraction used was not amenable to thinking about the problem in a modular way. Additionally, there was no convenient way to use the behavior independently of the motor control.

The design allows changes to be made to the behavior without changing the hardware control. In addition, the signal produced can also be used to simulate the artwork. This allows artists to visually observe modifications in the program without access to the physical hardware.

To simplify the testing of various movement parameters, sliders have been added to the graphical user interface (GUI). By using FRP, the current parameters can be transferred directly to the input signal of type `Input`. As a result, the movement of the 'beings' adapts in real time when the sliders are changed.

Furthermore, the Yampa implementation shows the state transitions more clearly.

```
type Velocity = Double
type Position = Double
data Being = Being
  { vel :: Velocity,
    socialGroup :: Int,
    rank :: Int
  }

type Input = ((Being, Int), Position)

doUntil
  :: SF a b
  -> SF a (Event c)
  -> SF a (b, Event c)

motionEv, noMotionEv :: SF Input (Event ())

social :: SF Input [Position]
-- starts 'being' and 'suppressor' depending on the
     beings' role, calculates position from velocity

disturbed :: SF Input [Position]
-- runs 'check' for all, calculates position from
     velocity

behavior :: SF Input [Position]
behavior = social `doUntil'` motionEv
  `switch` \_ -> disturbed `doUntil` noMotionEv
  `switch` const behavior
```

For example, in the `behavior` function, the social behavior (`social`) changes to the disturbed behavior (`disturbed`) as soon as a motion event (`motionEv`) occurs. The behavior then changes back again as soon as the motion sensor no longer registers anyone (`noMotionEv`).

The `social` function defines the behavior corresponding to each being's role. For a suppressor the `suppressor` function initiates `creepOut` if it outranks the current group, allowing the suppressor to fully emerge. This triggers the `currentGroupEv` event for the suppressor's group, enabling it to emerge as well.

```
currentGroupEv, notCurrentGroupEv
  :: SF Input (Event ())
higherRankEv, notHigherRankEv
  :: SF Input (Event ())

check    :: SF Input Velocity
```

```
-- hide -> peek -> hide

creepOut :: SF Input Velocity
-- hide -> peek -> stand -> hide

suppressor :: SF Input Velocity
suppressor = check `doUntil` higherRankEv
  `switch` \_ -> (creepOut `doUntil` notHigherRankEv
    )
  `switch` const suppressor

being :: SF Input Velocity
being = check `doUntil` currentGroupEv
  `switch` \_ -> (creepOut `doUntil`
    notCurrentGroupEv)
  `switch` const being
```

This code design is very close to the human description of the problem, which simplifies communication between artists and developers. This design also improved the understanding of the code for the artist duo. They recognized the potential to understand the developers' work and ensure that their requirements were understood correctly.

Further information on the use of Yampa and insights into the resulting Edge Beings code, as well as a comparison with its imperative version can be found in [8].

5 Impressions & Conclusion

The current results indicate that it is indeed possible to design software for robotics control using FRP, but further work is needed in order to make this style of programming accessible enough to increase the power and effectiveness with which artists can express their intentions.

The use of FRP enables a more modular architecture allowing to simply replace the physical hardware by a simulation. This simplifies the development and adaption of the code defining the behavior of an artwork.

The code design shown in this paper can make the implementation of an algorithm easier to understand for people with little programming experience even if they are not able to come up with the code themselves. This could enable closer collaboration between artists and developers.

Acknowledgments

The authors would like to thank Søren Pors and Aparna Rao from Pors & Rao for their support and openness towards this project, as well as the anonymous reviewers for their valuable comments and suggestions.

References

[1] FARM. 2024. Workshop on Functional Art, Music, Modeling and Design. https://functional-art.org/. Accessed: 2024-05-03.

[2] Google Arts & Culture. 2021. PATHOS: ROBOTIC ANIMATION with Pors & Rao | Google Arts & Culture. https://youtu.be/_SRj-jpwbZ8?si=Y94cS6WexU8d-UoD. Accessed: 2024-05-09.

[3] Paul Hudak. 2000. *The Haskell School of Expression: Learning Functional Programming through Multimedia.* Cambridge University Press.

[4] Paul Hudak, Antony Courtney, Henrik Nilsson, and John Peterson. 2003. Arrows, Robots, and Functional Reactive Programming. In *Advanced Functional Programming: 4th International School, AFP 2002, Oxford, UK, August 19-24, 2002. Revised Lectures.* Springer Berlin Heidelberg, Berlin, Heidelberg, 159–187. https://doi.org/10.1007/978-3-540-44833-4_6

[5] Ivan Perez. 2023. The Beauty and Elegance of Functional Reactive Animation. In *Proceedings of the 11th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design* (<conf-loc>, <city>Seattle</city>, <state>WA</state>, <country>USA</country>, </conf-loc>) *(FARM 2023).* Association for Computing Machinery, New York, NY, USA, 8–20. https://doi.org/10.1145/3609023.3609806

[6] Søren Pors and Aparna Rao. 2006. Untitled. http://www.porsandrao.com/work/?workid=21. Accessed: 2024-01-18.

[7] Søren Pors and Aparna Rao. 2020. Pors & Rao. http://www.porsandrao.com/. Accessed: 2024-01-18.

[8] Eliane I. Schmidli and Farhad Mehta. 2024. Using Functional Reactive Programming for Robotic Art – An Experience Report. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM 2024).* Association for Computing Machinery, New York, NY, USA.

[9] TED-Ed. 2012. High-tech art (with a sense of humor) - Aparna Rao. https://youtu.be/kJLDl2uDNaA?si=QVFeVN7Tn0ng-UJ7. Accessed: 2024-05-22.

Received 24-MAY-2024; accepted 2024-07-02

Bridging Art and Mathematics with Tessella: A Scala Functional Library for Regular Polygon Finite Tessellations of a Plane

Mario Càllisto
Independent
Milan, Italy
mario.callisto@gmail.com

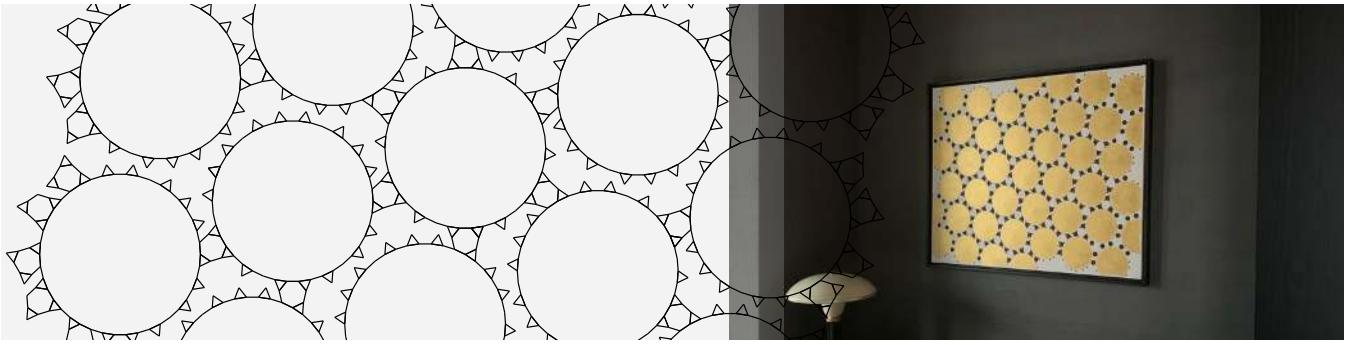


Figure 1. Tessellation (left) and acrylic painting on canvas (right), ©Mario Càllisto 2023.

Abstract

Tessellations of a plane surface by means of unit-sided regular polygons is a classical subject both in art, with examples dating back to the earliest human civilisations, and mathematics, where the complex patterns generated by a very simple set of rules have fascinated generations of inquisitive souls. While several taxonomies of the repeating patterns of a tessellation exist, the mathematical treatment of a particular finite tessellation is a quite recent field, greatly aided by computer algorithms. Introducing Tessella, a free software library of functional algorithms, written in Scala, where finite tessellations are described as immutable undirected planar graphs; thus providing the capabilities to experiment with manual and programmatic creation of finite tessellations, able in turn of inspiring new artistic insights. Abstract paintings are shown as an example, together with ideas on how this exploration journey might be taken further.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
FARM '24, September 2, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3678287

CCS Concepts: • **Applied computing** → **Fine arts**; • **Mathematics of computing** → **Graphs and surfaces**.

Keywords: tessellation, planar graph, art, painting

ACM Reference Format:
Mario Càllisto. 2024. Bridging Art and Mathematics with Tessella: A Scala Functional Library for Regular Polygon Finite Tessellations of a Plane. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3677996.3678287

1 Introduction

This work considers the potential of functional programming to facilitate an investigation into the artistic aspects of a specific class of two-dimensional geometric patterns, obtained from the tiling of a plane surface with regular polygons.

These tilings, known as tessellations, have been used by humans for their aesthetic value since well before they were formally described mathematically: one of their defining characteristics is the ability to generate a vast array of beautiful regular configurations from a very simple rule set.

As visually suggested in Figure 1, the attempt is to establish another thin but solid bridge between art and mathematics in this space. On the functional side, a computational model is presented for the purpose of handling any finite tessellation as a precise collection of polygonal sides and vertices, that could be eventually translated into a real object. At the same time, the main attention is directed towards the artistic side, identifying methods that enable the generation

of novel visual insights. In particular, random searches for tessellations with increasingly strong symmetric qualities are implemented and refined.

The presentation includes examples of geometric abstract acrylic painting on canvas, just one possible medium of expression among many. Additionally, the discussion addresses general observations on the transfer of a perfect theoretical concept into a tangible piece of art.

2 Tessellation Overview

A tessellation is a collection of geometric shapes that cover a surface without leaving any gap or overlap. Focusing on the purest and most classical form of tessellations, each shape is a regular polygon, connected edge-to-edge to other regular polygons, and the sides of each polygon have the same length.

2.1 Mathematical Definition

A tessellation $\mathbf{T}$ of the Euclidean plane $\mathbf{E}^2$ is a countable set

$$\mathbf{T} = \{T_1, T_2, \ldots, T_n\}$$

where $T_1, T_2, \ldots, T_n$ are called *tiles* of $\mathbf{T}$ and each *tile* is a closed set. *Tiles* cover the plane

$$\{T_1 \cup T_2 \cup \ldots \cup T_n\} = \mathbf{E}^2$$

without gaps

$$\{intT_i \cap intT_j\} = \varnothing \; \forall (i, j) \text{ where } i \neq j$$

and without overlapping between the interior of any two of them. All *tiles* have the shape of a convex regular polygon and their boundaries

$$\{\delta T_i \cap \delta T_j\} \in \{\varnothing, \text{vertex}, \text{side}\} \; \forall (i, j) \text{ where } i \neq j$$

are connected edge-to-edge, intersecting along a common side, at a vertex, or not at all. Therefore the side of each polygon has identical (unit) length.

A finite tessellation F

$$F \subset \mathbf{T}$$

is a proper subset of $\mathbf{T}$.

2.2 In Nature

Regular polygon tessellations are not common in nature, but they still can be found at different scales. Examples, all approximations of the regular hexagonal tessellation, include:

- atomic scale: graphene lattice (side length ≈ 0.14 nm)
- cellular scale: corneal endothelium (≈ 50 µm)
- human scale: honeycomb (≈ 3 mm), Giant's Causeway (≈ 50 cm)

For the physics behind the emergence of some of these structures [16], it should be noted that Joseph Louis Lagrange proved in 1773 that the highest-density lattice packing of circles is the hexagonal arrangement [6], see Figure 3.

2.3 In Art

Humans seems to be attracted to the beauty of regular polygon tessellations.

An hypothesis that could be put forward is that large repetitions of a very simple unit element provide a powerful direct visual approach to the concept of infinity [18] in space and time: each finite tessellation is a glimpse of a complete entity that could cover all space and take all time to do so.

Early examples of geometric tessellations come from Uruk, Iraq, ancient Mesopotamia [1]. Chinese lattice design dates back to the first millenium BCE [8].

Figure 2. Roman mosaic with a $[(3_3.4_2); (3.4.6.4)]$ pattern at Conimbriga, Coimbra, Beira Litoral, Portugal.

Roman buildings and floors were decorated with mosaic tiles arranged in beautiful geometric shapes, see Figure 2. The Latin word *tessella*, meaning small tile, is the root of the English word tessellation.

Persians demonstrated their mastery in tile decorations [20]. Similarly, artisans in Islamic Spain used congruent, multicoloured tiles on the walls and floors of buildings [5]. Islamic geometric patterns with bold colors still thrive and evolve [9].

2.4 In Mathematics

In 1619, Johannes Kepler made an early documented study of regular and semi-regular tessellations in his Harmonices Mundi[17]; he was possibly the first to explore and to explain the hexagonal structure of honeycomb.

Almost three hundred years later in 1891, the Russian crystallographer Evgraf Fedorov proved that every periodic tiling of the plane features one of 17 different groups of isometries [11], thus marking the unofficial beginning of the mathematical study of tessellations. Other prominent contributors are Alexei Vasilievich Shubnikov and Nikolai

Belov [22] (1950) and Heinrich Heesch and Otto Kienzle [15] (1963).

3 Model of a Finite Tessellation

A mathematical computational model of a finite tessellation is needed to define basic transformation functions, such as adding or removing one regular polygon at time. These algorithmic building blocks allow more complex methods, especially when directed towards artistic goals.

3.1 Tessellations with Repeating Patterns

There are only 5 regular polygons, listed below in Table 1, that can be combined edge-to-edge to cover the whole plane.

Table 1. Regular polygons combinable in infinite tessellations

Number of sides	Common name
3	Triangle
4	Square
6	Hexagon
8	Octagon
12	Dodecagon

Nevertheless their combinations can form an infinite number of distinct tessellations [19].

3.2 Preferred Notation for a Pattern

Different symbols have been used to identify tessellations with repeating patterns [4]. In this paper, focusing on edge-to-edge regular polygon tessellations, the preference is to expand the notation adopted in [14] and based on *vertex symbol*, which represents in cyclic order the size s of each of the $\{T_1, T_2, \ \ldots\ , T_m\}$ polygons meeting at a vertex. Therefore

$$VS = (s_1.s_2.\ \ldots\ .s_m)$$

and each pattern P will be written as

$$P = [VS_1; VS_2; \ \ldots\ ; VS_n]$$

An example is $[(3.3.3.3.3.3); (3.3.3.3.3.3); (3.3.3.3.6)]$, see Figure 15. The notation can be shortened to $[2 * (3_6); (3_4.6)]$. In both variants this notation preserves info about the pattern's:

- hedrality: number of distinct regular polygon sizes (2 in the example);
- gonality: number of distinct vertex configurations (2 in the example);
- uniformity: number of vertex configurations (3 in the example), each representing a different transitivity class in respect to the group of symmetries of the tessellation.

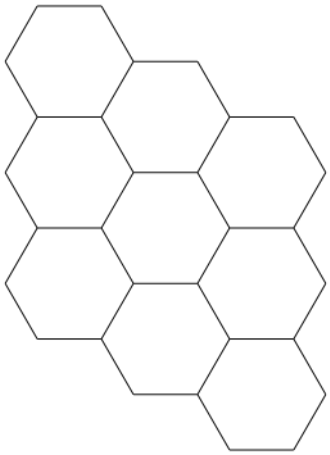


Figure 3. Finite set of the $[(6_3)]$ regular hexagonal pattern, with 3 hexagons meeting at each vertex.

It has to be noted that this simple notation cannot distinguish between similar, sufficiently complex, patterns. For example the pattern $[(3_3.4_2); (3_2.4.3.4)]$ is the same for two completely different tessellations, see Figures 16 and 17.

3.3 Finite Tessellations as Undirected Graphs

Even assuming that symbols could uniquely represent any regular polygon tessellation, with or without repeating patterns, they would not be able to distinguish between different finite sets of the same tessellation.

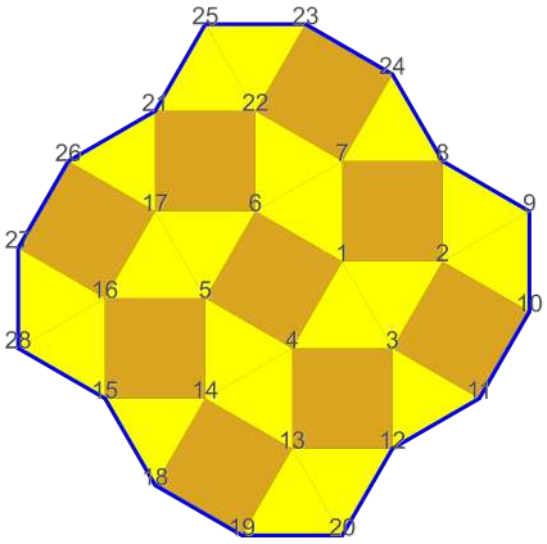


Figure 4. Finite set of the $[(3_2.4.3.4)]$ pattern.

The desired result of mapping every possible finite edge-to-edge regular polygon tessellation can be achieved by describing each as an undirected planar graph

$$F \longrightarrow G = (E, V) \text{ is injective}$$

where

$$F \text{ polygon vertices } \longrightarrow V \text{ is bijective}$$
$$F \text{ polygon sides } \longrightarrow E \text{ is bijective}$$

each node of the graph is mapped to by exactly one vertex of a polygon and *vice versa*, and each edge of the graph by exactly one side of a polygon and *vice versa*.

The special relations between tessellations and graphs are the subject of a large literature; on finite polygonal tessellations see Grünbaum and Shephard [13] and others [2] [21] [3].

3.4 Additional Constraints

For each finite tessellation F, two new constraints must be added to those in section 2.1.

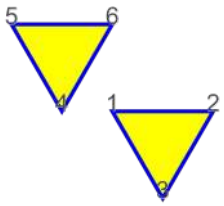


Figure 5. Smallest disconnected graph describing multiple finite tessellations.

The first is easily defined in terms of properties of the undirected graph G that describes F.

$$G \text{ is connected}$$

If not connected, G describes a set of finite tessellations with more than one element (see Figure 5).

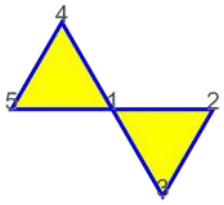


Figure 6. Smallest finite tessellation with non adjacent polygons at vertex, a so called *butterfly graph*.

The second, which also implies the first, states that

$$\delta\{T_1 \cup T_2 \cup \ldots \cup T_n\} \text{ is a simple closed curve}$$

all polygons must be adjacent, the boundary of their union being a simple closed curve. This can be false at the boundary of an invalid finite tessellation (see Figure 6).

4 A Scala Functional Library for JVM and JavaScript

Tessella [7] is a free software library for handling edge-to-edge regular polygon finite tessellations, written in Scala [10] in functional style, where each tessellation is an immutable collection of graph edges.

It runs on the Java Virtual Machine (JVM), interacts seamlessly with Java code and is optimised as highly efficient JavaScript for browsers.

4.1 Validation of Constraints

We know from section 3.3 that every tessellation can be described as an undirected planar graph, but

$$G = (E, V) \longrightarrow F \text{ is non-injective and non-surjective}$$

not every undirected graph describes a tessellation (see Figures 5 and 6).

Tessella permits the input of undirected graphs as tessellation descriptors; however, the builder functions employed always perform a validation check against the previously defined mathematical constraints.

```
def maybe(
  edges: Edge*
): Either[String, Tiling]

Tiling.maybe(1--2, 1--3, 2--3)
// Right(Tiling(1--2, 1--3, 2--3))
Tiling.maybe(1--2, 1--3, 2--3, 1--4)
// Left("Node 4 is not compliant")
```

4.2 Equality

Every tessellation can be described by a finite (combinatorial) number of differently labelled graphs, even assuming to always label the nodes from 1 to $|V|$.

In the smallest example the same finite tessellation F, a square polygon, can be described by three different graphs of type $G = (E, V = \{1, 2, 3, 4\})$, with the following edges:

$G_A = (\{1\text{–}2, 2\text{–}3, 3\text{–}4, 4\text{–}1\}, V)$
$G_B = (\{1\text{–}2, 2\text{–}4, 4\text{–}3, 3\text{–}1\}, V)$
$G_C = (\{1\text{–}3, 3\text{–}2, 2\text{–}4, 4\text{–}1\}, V)$

Tessella is able to correctly identify equality among finite tessellations, such that $G_A = G_B = G_C$.

```
val a = Tiling.maybe(1--2, 2--3, 3--4, 4--1)
val b = Tiling.maybe(1--2, 2--4, 4--3, 3--1)
a == b // true
```

4.3 Creation Methods

Tessellations can be created programmatically in Tessella following some simple regularities. Triangle, square and hexagon tilings - of pattern $[(3_6)]$, $[(4_4)]$ and $[(6_3)]$ respectively - can be decorated and/or combined in layers.

But the most fertile ground has been found by implementing the basic function

```
def maybeGrowEdge(
  edge: Edge,
  polygon: Polygon
): Either[String, Tiling]
```

that, pending validation, attempts to add a single regular polygon to an edge of the graph G (that is, a side of the tessellation F). From this fundamental building block, a wide range of algorithms can be conceived, with the use of equality to prune search trees that converge to identical results.

4.4 Vertex Configurations and Patterns

As stated in section 3.3, a pattern cannot precisely identify a specific finite tessellation; for that purpose Tessella relies on an undirected graph instead, represented as a collection of graph edges.

Nevertheless patterns are defined and used in the library, for instance in constrained growth algorithms, where a tessellation must adhere to given rules; examples will be provided in section 5.2.3.

A pattern can be created, starting if needed from a collection of full vertex configurations; see below how $[2 * (3_6); (3_4.6)]$ is built:

```
val vs1 = FullVertex.s("(3.3.3.3.3.3)")
// a vertex configuration
val vs2 = FullVertex.s("(3.3.3.3.6)")
// another vertex configuration

Pattern(vs1, vs1, vs2)
// [2x(3.3.3.3.3.3);(3.3.3.3.6)]
```

4.5 SVG Drawing

Tessella includes functions to convert a finite tessellation into Scalable Vector Graphics XML, or SVG [23], with different visualization options. Without a curated spatial rendering, it is almost impossible to tell what a grown $G = (E, V)$ looks like and whether it is pleasing to the eye.

Figures 3, 4, 7, 8, 9, 14, 15, 16 and 17 are all direct outputs from the library. For example Figure 4 with the function:

```
Tiling
  .pattern_33434(side = 1)
  .toOption.get // gets the valid tiling
  .toSVG(
    fillPolygons = true,
    labelledNodes = LabelledNodes.ALL
  )
```

5 Geometric Abstract Art

The utilisation of growable finite tessellations facilitates the emergence of novel artistic insights, as it permits a greater degree of randomness and heterogeneity in experimentation.

In fact, a human preference for complexity and variety in regular polygon tessellations has been observed in terms of both vertex qualities and symmetric transformations [12].

5.1 Adding More Polygons

Although it is known from section 3.1 that some finite tessellations cannot be expanded to an infinite version according to the edge-to-edge regular polygon rule set, it is now possible and advisable to play with them.

Table 2. Regular polygons combinable in a full angle, in addition to Table 1

Number of sides	Common name
5	Pentagon
7	Eptagon
9	Ennagon
10	Decagon
15	-
18	-
20	Icosagon
24	-
42	-

This is only one, perhaps the most obvious, of the possible aesthetic avenues that can be followed: to use a wider set of regular polygons, adding the 9 listed in Table 2 to the 5 previously listed in Table 1.

5.2 In Search of Odd Regularities

5.2.1 Randomic. An already available pseudo-randomic generation function can be used.

```
def randomSteps(
  steps: Int,
  maybePolygons: Option[List[Polygon]]
):Option[Tiling]

Tiling
```

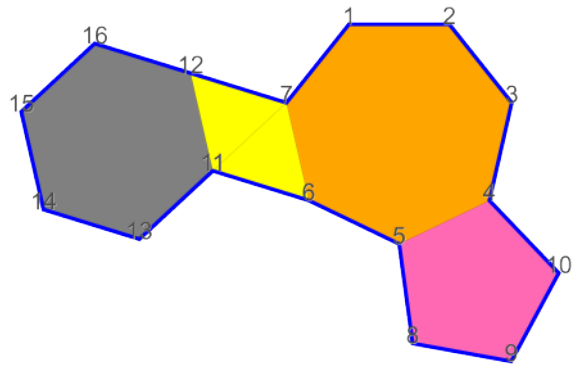


Figure 7. Random growth of regular polygons.

```
.empty // starts from an empty tiling
.randomSteps(5, None)
```

A possible output is shown in Figure 7.

5.2.2 Randomic with Selected Polygons. More refined results are obtained by using selected regular polygons only.

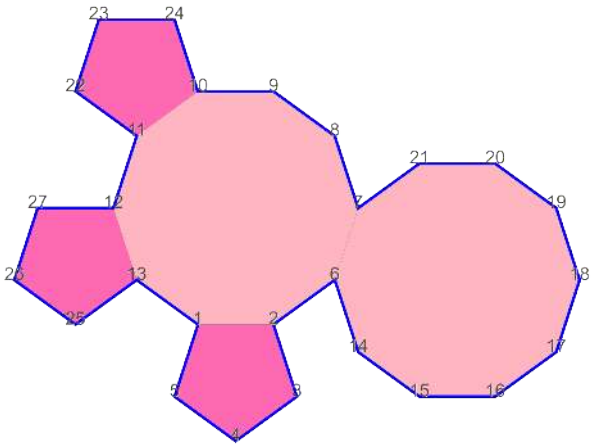


Figure 8. Random growth of pentagons and decagons.

```
Tiling
  .empty
  .randomSteps(
    5,
    Option(List(Polygon(5), Polygon(10)))
  )
```

A possible output is shown in Figure 8.

5.2.3 Randomic with Selected Pattern at Vertices. A further regularity is observed when a similar function is employed to verify that each vertex in V belongs to a given pattern P.

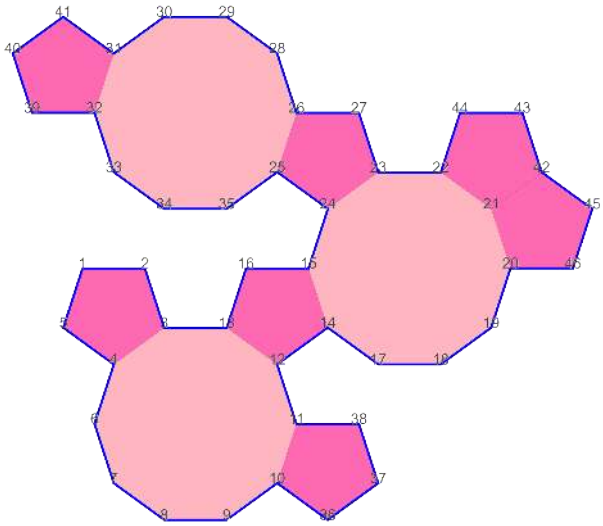


Figure 9. Pattern growth of pentagons and decagons.

```
def randomStepsWithinPattern(
  steps: Int,
  pattern: Pattern
):Option[Tiling]

Tiling
  .empty
  .randomStepsWithinPattern(
    10,
    Pattern.s("[(5.5.10)]")
  )
```

A possible output is shown in Figure 9.

When the function is used with a pattern able to produce a 1-uniform tessellation, the entire plane tends to be filled.

```
Tiling
  .empty
  .randomStepsWithinPattern(
    30,
    Pattern.s("[(3.4.6.4)]")
  )
```

A possible output is shown in Figure 10

5.2.4 Further Sophistication. From here, more sophisticated algorithms could be implemented, for example using functions that verify the compactness, uniformity, rotational and reflection symmetry of the grown tessellations.

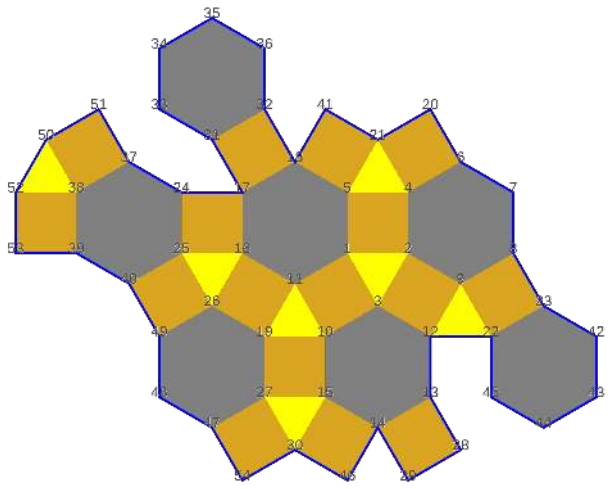


Figure 10. Pattern growth of triangles, squares, hexagons.

5.3 Painting

Artistic sensibility becomes, from now onwards, the key part of the process. Identifying a theme, reinterpreting it according to one's own goals, bending the rules, finding colour combinations, defining the most appropriate medium: all these are very personal choices made by the creator.

Here below a painting that, although not a proper tessellation of regular polygons only, can be traced back to Figure 9.

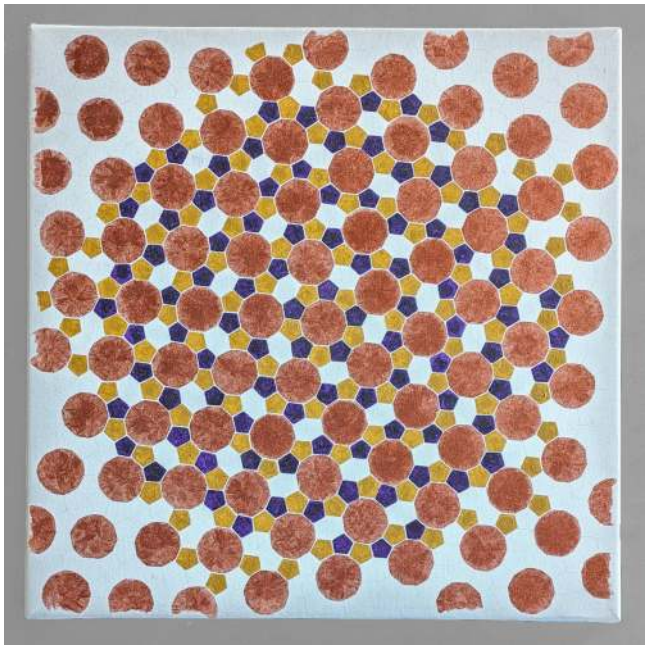


Figure 11. Untitled, acrylic on canvas, ©Mario Càllisto 2023.

However, some general observations can be made.

There is an aesthetic pleasure in trying to approach perfection, but never really achieving it. Whatever the medium chosen - and the problem is certainly evident in painting - the end result can be very precise, but never as precise as the abstract regular polygon tessellation in our minds.

On the other hand, there is a kind of mirrored pleasure, that can be shared with anyone who looks at the art, in observing how the human brain is able to reassemble perfection, automatically trying to filter out and capture the essence of the composition.

6 Limitations and Future Work

Tessella as a core library is open and welcomes contributions. Its latest published version can be already used as a starting point to create more specific functions, according to one's mathematical and/or artistic inclination.

One current limitation is the lack of functions dealing with differences between finite tessellations. The ability to check whether a finite tessellation is contained in a larger one would be probably useful in growth algorithms.

Another area of possible development is to allow some selected shapes of unit-sided *irregular* polygons, by modifying the current constraints and their validation. Readers with a keen eye may have noticed a repeated *irregular hexagon* in Figure 11.

7 Conclusion

A library to handle edge-to-edge regular polygon finite tessellations, Tessella, is suggested. It relies on representing finite tessellations as undirected planar graphs, taking care of their validation against the identified constraints. Through a gradual example from code to finished acrylic painting on canvas, it is confirmed how some basic functions built on top of the computional model can disclose artistically relevant research paths.

This paper represents a preliminary investigation into the problem space, hopefully opening up avenues for more interesting and complete insights, implementations and artistic ideas to come.

A Paintings

For further reference, other two paintings are presented, always derived, as in Figure 1 and 11, from edge-to-edge regular polygon finite tessellations.

B Patterns with Uniformity

The non-trivial patterns of sections 2.3 and 3.2 are shown here for better clarity.

Each vertex is highlighted with a different mark to show the uniformity groups. It has to be noted in Figure 15 how identical vertex configurations can belong to different symmetry classes. Figure 15 can be generated with:

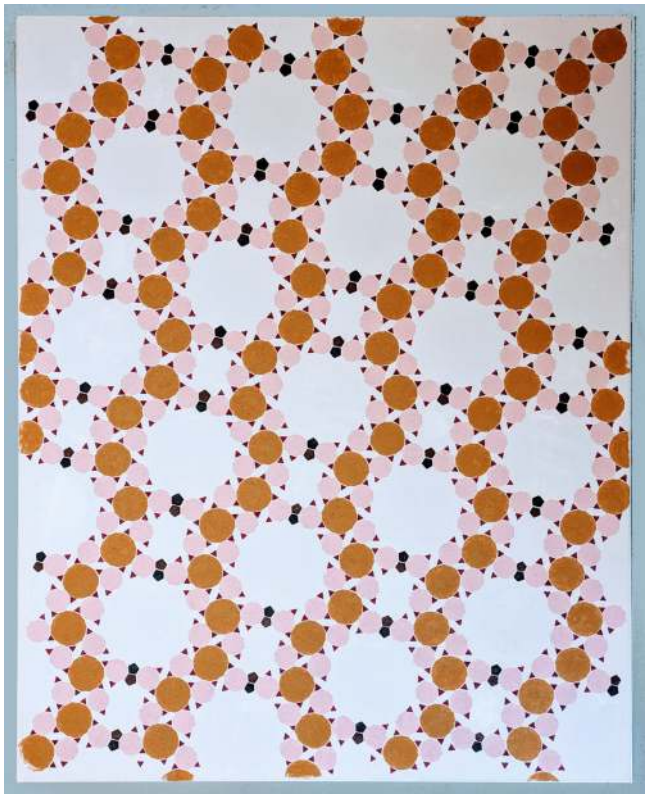


Figure 12. *Ciliegio*, acrylic on canvas, ©Mario Càllisto 2023.

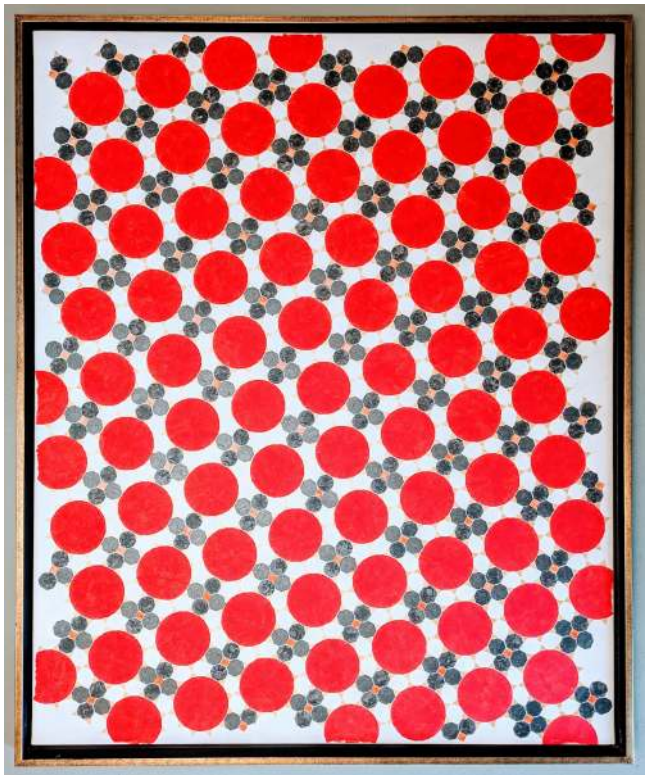


Figure 13. *Sandalo*, acrylic on canvas, ©Mario Càllisto 2021.

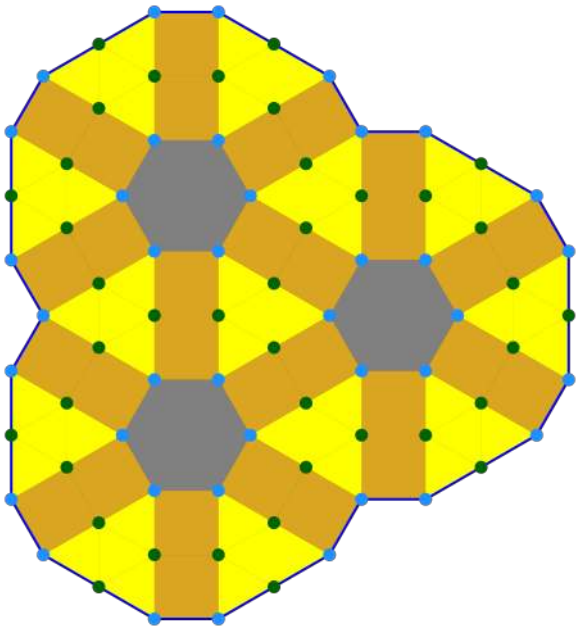


Figure 14. Finite set of the $[(3_3.4_2); (3.4.6.4)]$ pattern.

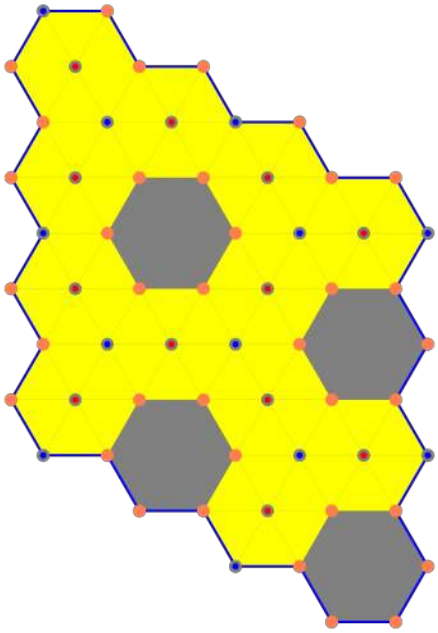


Figure 15. Finite set of a $[2 * (3_6); (3_4.6)]$ pattern.

```
Tiling
  .pattern_2x333333_33336(
    width = 4,
    height = 4
  )
```

```
.toOption.get
.toSVG(
  fillPolygons = true,
  labelledNodes = LabelledNodes.NONE,
  markStyle = MarkStyle.UNIFORMITY
)
```

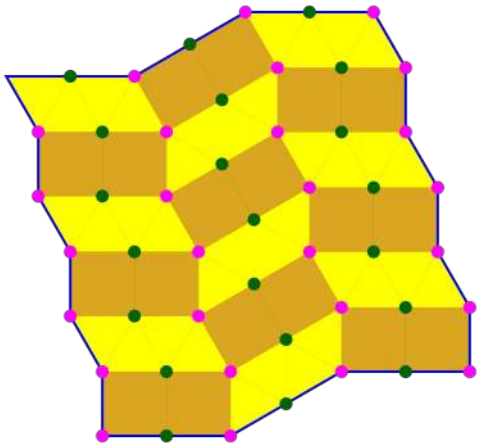


Figure 16. Finite set of a $[(3_3.4_2); (3_2.4.3.4)]$ pattern.

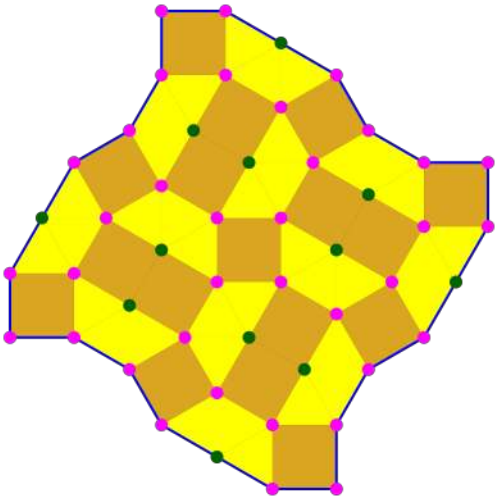


Figure 17. Finite set of a different $[(3_3.4_2); (3_2.4.3.4)]$ pattern.

References

[1] Fanny Alloteau, Sarah Dermech, Adriana Iuliano, Stefan Röhrs, Stefan Simon, Sonja Radujkovic, Helen Gries, and Barbara Helwing. 2022. Cone mosaic façades in Mesopotamia (4th millenium BCE): archaeometric approach to an architectural innovation. In *Mosaïk in situ – transloziert – museal. Beiträge des 15. Konservierungswissenschaftlichen Kolloquiums in Berlin/Brandenburg am 4. November 2022*. Number 64. Michael Imhof Verlag, 12–21. https://hal.science/hal-04196421

[2] László Babai. 1991. Vertex-transitive graphs and vertex-transitive maps. *Journal of graph theory* 15, 6 (1991), 587–627. https://doi.org/10.1002/jgt.3190150605

[3] Oliver Baues and Norbert Peyerimhoff. 2001. Curvature and geometry of tessellating plane graphs. *Discrete & Computational Geometry* 25 (2001), 141–159. https://doi.org/10.1007/s004540010076

[4] Vladislav A. Blatov, Michael O'Keeffe, and Davide M. Proserpio. 2010. Vertex-, face-, point-, Schläfli-, and Delaney-symbols in nets, polyhedra and tilings: recommended terminology. *CrystEngComm* 12, 1 (2010), 44–48. https://doi.org/10.1039/B910671E

[5] B Lynn Bodner. 2003. Construction and classifying designs of al-andalus. In *Meeting Alhambra, ISAMA-BRIDGES Conference Proceedings*. 61–68. http://archive.bridgesmathart.org/2003/bridges2003-61.html

[6] Hai-Chau Chang and Lih-Chung Wang. 2010. A Simple Proof of Thue's Theorem on Circle Packing. https://doi.org/10.48550/arXiv.1009.4322

[7] Mario Càllisto. 2024. *Tessella: Tilings by regular polygons.* https://doi.org/10.5281/zenodo.12747300

[8] Daniel Sheets Dye. 1974. *Chinese lattice designs*. Vol. 5. Courier Corporation.

[9] Mohamed Rashid Embi and Yahya Abdullahi. 2012. Evolution of Islamic geometrical patterns. *Global Journal Al-Thaqafah* 2, 2 (2012), 27–39.

[10] Lightbend Inc. EPFL. 2002. *Scala: a modern multi-paradigm programming language.* ScalaCenter, Lausanne, Switzerland. https://www.scala-lang.org/

[11] Evgraf Stepanovich Fedorov. 1891. The symmetry of regular systems of figures. *Proceedings of the Imperial St. Petersburg Mineralogical Society), series 2, 28 : 1–146* (1891).

[12] Jay Friedenberg. 2019. The Perceived Beauty of Regular Polygon Tessellations. *Symmetry* 11, 8 (2019). https://doi.org/10.3390/sym11080984

[13] Branko Grünbaum and Geoffrey Colin Shephard. 1981. The geometry of planar graphs. *Combinatorics* 8 (1981), 124.

[14] Branko Grünbaum and Geoffrey Colin Shephard. 1987. *Tilings and patterns.* Courier Dover Publications.

[15] Heinrich Heesch and Otto Kienzle. 1963. *Flächenschluß: System der formen lückenlos aneinanderschließender Flachteile.* Vol. 6. Springer-Verlag.

[16] Bhushan Lal Karihaloo, K. Zhang, and J. Wang. 2013. Honeybee combs: how the circular cells transform into rounded hexagons. *J. R. Soc. Interface* (2013). https://doi.org/10.1098/rsif.2013.0299

[17] Johannes Kepler. 1997. *The harmony of the world.* Vol. 209. American Philosophical Society.

[18] Ivars Peterson. 2008. *Fragments of Infinity: A kaleidoscope of math and art.* Turner Publishing Company.

[19] Paul Robin. 1887. Carrelage illimité en polygones réguliers. *La Nature, N°737 - 16 juillet* (1887).

[20] Reza Sarhangi, Slavik Jablan, and Radmila Sazdanovic. 2004. Modularity in medieval Persian mosaics: textual, empirical, analytical, and theoretical considerations. In *Bridges: Mathematical Connections in Art, Music, and Science*. 281–292.

[21] Brigitte Servatius and Herman Servatius. 1998. Symmetry, automorphisms, and self-duality of infinite planar graphs and tilings. In *International Scientific Conference on Mathematics. Proceedings*. 83–116.

[22] Alekseĭ Vasilevich Shubnikov, Nikolaĭ Vasilevich Belov, William T Holser, et al. 1964. Colored symmetry. (1964).

[23] W3C. 2010. *SVG is a markup language for describing two-dimensional graphics applications and images, and a set of related graphics script interfaces.* W3C, Cambridge, Massachusetts, United States. https://www.w3.org/Graphics/SVG/

Received 19 May 2024; accepted 02 July 2024

Functional Curves and Surfaces: Algebraic Geometry Inspired Visuals in Hydra

Yoni Maltsman
Harvey Mudd College
Claremont, USA
jmaltsman@g.hmc.edu

Abstract

We present FCS (Functional Curves and Surfaces), a framework for synthesizing visual patterns by composing functions for curves and surfaces, implemented as an extension to Hydra, a popular tool for live coding visuals. The extension consists of functions for dozens of implicit and parametric curves and surfaces, which can be chained together in numerous ways to create striking visuals. FCS can also serve as a tool for students and practitioners of math, art, and computer science to learn algebraic geometry in a fun, creative way.

CCS Concepts: • **Applied computing** → **Media arts**.

Keywords: Algebraic Geometry, Live Coding

ACM Reference Format:
Yoni Maltsman. 2024. Functional Curves and Surfaces: Algebraic Geometry Inspired Visuals in Hydra. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3677996.3678292

1 Introduction

Hydra-FCS investigates the question of how we can use mathematical representations of curves and surfaces in two, three, and potentially even higher dimensions as mappings between screen positions, greyscale colors, and RGB colors to create interesting visual effects.

Typically, the visualization of curves and surfaces is limited to a fairly literal representation, where the locations of all points in the object are displayed in two or three dimensional space. [4] provides a history of educational software tools for visualizing geometric objects. More recently with the adoption of GPU rendering techniques, ray tracing and ray marching have served as powerful algorithms for visualizing 3D surfaces in photorealistic scenes [1]. These tools and techniques often emphasize the visualization of mathematical functions as geometric objects, whereas FCS goes in the other direction, visualizing geometric objects as *maps*, which can be chained together in complex ways to create dynamic visual effects.

The two primary mathematical representations of curves and surfaces are the parametric and the implicit representations. Consider the analogy of an instruction manual for building a house. The manual can either provide explicit, step by step instructions on where to place every beam and brick and the order in which to place them. Alternatively, the manual can tell you the properties that the house must satisfy: the beams must be at right angles, the bricks must be exactly on top of each other, and each brick's distance from the center of the house depends on its distance from the corners of the house. In the latter case one would have to work backwards to figure out the locations of bricks and beams that satisfy these properties, whereas in the former case one simply has to follow the instructions.

This former case is analogous to the parametric representation of a geometric object, which is an algorithm dictating where to plot every point. The latter case is similar to the implicit representation, a set of mathematical relations satisfied by every point in the object. These two representations are fundamental to FCS functions, which we discuss in section 3. For a thorough, mathematically detailed introduction to parametric and implicit curves and surfaces, see [2].

2 Background: Hydra, Functional Programming, and Live Coding

Hydra, developed by Olivia Jack, is a popular web based tool for live coding visuals. Although it is implemented in Javascript, which is not a purely functional language, Hydra largely follows a functional paradigm in that visuals are created solely by composing functions. Each Hydra function encodes a GLSL (OpenGL Shading Language) shader, which is a program specialized for rendering graphics that conducts mathematical operations on pixels in parallel.

The development of Hydra was inspired by analog video synthesizers such as the Sandin Image Processor [3]. An analog synthesizer consists of various modules which apply different transformations to an electrical signal, and these modules can be physically connected and tuned in all sorts

This work is licensed under a Creative Commons Attribution 4.0 International License.

FARM '24, September 2, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3678292

of different ways. Jack describes how understanding analog synthesis through a functional paradigm informs the development of Hydra:

> Functional paradigms are really helpful in describing analog synthesis – each module is a function that always does the same thing when it receives the same input (parameters are like knobs). I'm very inspired by the modular idea of defining the pieces to maximize the amount that they can be rearranged with each other. The code describes the composition of those functions with each other [3].

This contrasts to the imperative event loop paradigm of many other popular creative coding frameworks, such as Processing and OpenFrameworks: first, objects such as textures, shapes, and shaders are initialized when the program launches, and then during program execution, objects are continuously updated and drawn to the screen (Figure 1). In Hydra, users can solely focus on composing and tuning functions to quickly iterate over a massive space of possible visual patterns, abstracting away technical control over the objects on which these functions operate.

[6] describe the influence of functional programming in a number of popular domain specific languages for live coding music. They explain the inspiration of principles from functional reactive programming in the design of Tidal, where musical patterns are represented by functions and can be flexibly composed with each other, providing users with access to a vast exploratory space for music synthesis.

Figure 1. Generating a simple visual oscillator in Hydra (top) vs OpenFrameworks (bottom). Hydra abstracts away the configuration of objects such as frame buffers and shaders so that the user can solely focus on chaining together effects.

3 FCS Functions

FCS consists of six types of functions: implicit curves, parametric curves, implict surfaces, parametric surfaces, inverse parametric surfaces, and parametric hypersurfaces. An implicit curve is a curve defined by a function f of the form:

$$f(x, y) = 0$$

For example, every point (x, y) on a circle of radius R satisfies the equation

$$x^2 + y^2 = R^2$$

where rearranging yields

$$x^2 + y^2 - R^2 = 0.$$

Notice that the function $f(x, y) = x^2 + y^2 - R^2$ is a map from a two dimensional space to a one dimensional space. We can thus use implicit equations to generate visual patterns: each position on a screen is mapped to a greyscale value based on the implicit equation (Figure 2). Many other curves can be represented implicitly, including lines, cassini ovals, and astroids, all of which are implemented in FCS. In the FCS library, we also animate implicit equations with a *time* variable. This tracks the time elapsed over the course of the program execution, and is used to animate the visuals (Appendix). The author makes aesthetic choices in deciding how time is incorporated in each of the functions for implicit curves. In the FCS extension, the first argument to every implicit curve function is the frequency of the time varying pattern, while the other arguments are the other parameters of the equation. Users can thus connect these parameters with external signals such as music or the position of the user's mouse.

Some curves also admit a *parametric* representation. A parametric curve is defined by two equations $x(t), y(t)$ which take a parameter t as input over some range $[t_i, t_f]$. For example, if we consider the system of equations:

$$x(t) = R\cos(t)$$

$$y(t) = R\sin(t)$$

and we plot the points $(x(t), y(t))$ for all t between 0 and 2π, we get a circle of radius R.

In contrast to implicit curves, parametric curves are defined by maps from one to two dimensions. We can thus feed the output of an implicit equation as the input of a parametric equation. The parametric equation can then output a new position, which we can feed back into an implicit equation to generate a greyscale color. In FCS, parametric equations have the effect of modulating the positions of pixels, as depicted in Figure 2 and the appendix.

In addition to implicit curves, parametric surfaces provide another way to map position to color. A parametric surface

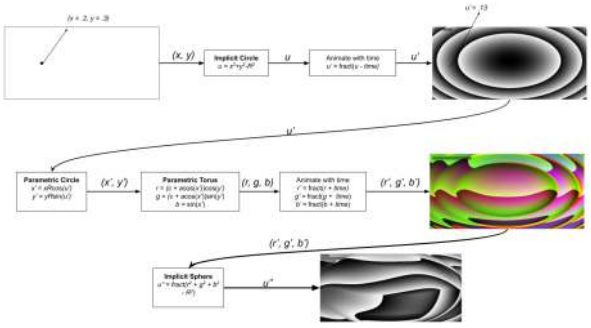


Figure 2. Chain of functions to create a visual pattern. First, each position on the screen is mapped to a greyscale color based on the implicit equation for a circle. This color then yields new positions via the parametric equation for a circle, which are fed into the equation for a torus to yield RGB colors. These colors are then mapped to greyscale using the implicit equation for a sphere.

is a surface in three dimensional space defined by three equations with two input parameters. An example is a sphere of radius R, which we obtain by varying input parameter θ from 0 to 2π and ϕ from 0 to π and plotting the equations

$$x(\theta, \phi) = R\sin(\phi)\cos(\theta)$$
$$y(\theta, \phi) = R\sin(\phi)\sin(\theta)$$
$$z(\theta, \phi) = R\cos(\phi).$$

We use parametric surfaces to map positions on the screen to red-green-blue (RGB) colors, where the input position (x, y) correspond to the control parameters θ and ϕ and the outputs x, y, and z correspond to the red, green, and blue of the rendered color, respectively. Other examples of parametric surfaces include tori, cylinders, and cones.

Implicit representations of surfaces endow us with a map from RGB color space back to greyscale color space, allowing us to generate feedback between the various types of maps we have described thus far (figure 2). A surface can be described implicitly if every point on that surface satisfies an equation relating its x, y, and z coordinates. For example, points on a sphere of radius R satisfy the implicit equation

$$x^2 + y^2 + z^2 = R^2.$$

The function $f(x, y, z) = x^2 + y^2 + z^2 - R$ is thus a map from a three dimensional space to a one dimensional space, so the RGB output of a parametric surface function can serve as the input of an implicit surface function.

While parametric curves enable us to use greyscale textures to modulate visuals, we use inverse parametric surfaces and parametric hypersurfaces to construct maps from three to two dimensions, letting us use RGB textures to transform screen space.

We can obtain an inverse parametric surface by solving for the two control parameters in the system of equations for a parametric surface. For example, consider the system of equations to parameterize a sphere described earlier. Rearranging those equations to solve for θ and ϕ yields

$$\theta = \arctan(\frac{y}{x})$$
$$\phi = \arccos(\frac{z}{R^2}).$$

Note, however, that arccos and arctan are not injective functions, for example $\arccos(0) = n\pi/2$ for any odd n, and GLSL restricts the values returned to the range $[0, \pi]$. Thus inverse parametric surface functions don't map very strictly to their respective surfaces, but are included in FCS because of the striking visual effects they create (e.g. figure 3b).

In addition to scaling, we can transform screen coordinates with 2×2 matrices, which can both scale and rotate positions. We can obtain 2×2 matrices from RGB textures using parametric equations for surfaces in four dimensional space. For example, a four dimensional sphere of radius R consists of the set of points (x, y, z, w) satisfying the equation

$$x^2 + y^2 + z^2 + w^2 = R^2.$$

We can construct this sphere, also known as a hypersphere, parametrically with three control parameters θ_1, θ_2, an θ_3 and the equations

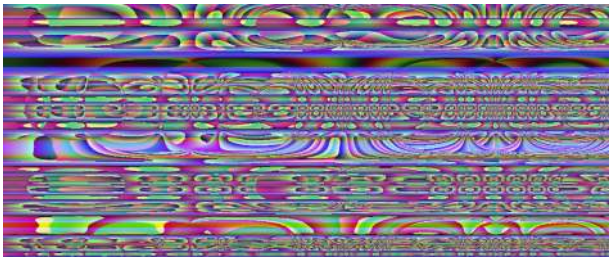
$$x(\theta_1, \theta_2, \theta_3) = R\cos(\theta_1)$$
$$y(\theta_1, \theta_2, \theta_3) = R\sin(\theta_1)\cos(\theta_2)$$
$$z(\theta_1, \theta_2, \theta_3) = R\sin(\theta_1)\sin(\theta_2)\cos(\theta_3)$$
$$w(\theta_1, \theta_2, \theta_3) = R\sin(\theta_1)\sin(\theta_2)\sin(\theta_3).$$

4 Discussion

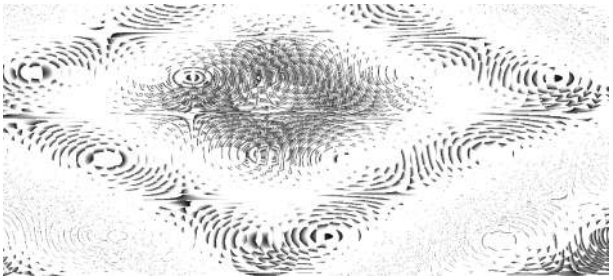
FCS offers its practitioners a more dynamic perspective of curves and surfaces than traditional visualizations. A sphere, for example, in addition to being a three dimensional ball, is representative of functions that can act on other functions. Different curves and surfaces not only look different geometrically, but are also functionally unique. FCS also offers the opportunity for shapes to interact in ways that simply plotting them does not. A sphere can interact with a circle, which can act upon a torus, and so forth. FCS can thus prime students for engaging with the field of mathematics, which often employs multiple representations to illuminate a single object. For example, one can study a square as a graph with four edges and four vertices, the cyclic group C_4, the dihedral group D_8, or a parametric curve.

[5] develop a strata based approach to characterize both artistic and scientific works of sonification, where data is mapped to music. This framework can also shed light on FCS, where we map geometric objects to visuals. The authors describe three approaches to sonification. The *generative* approach heavily emphazises the sound that is produced without much care for the underlying source of the data.

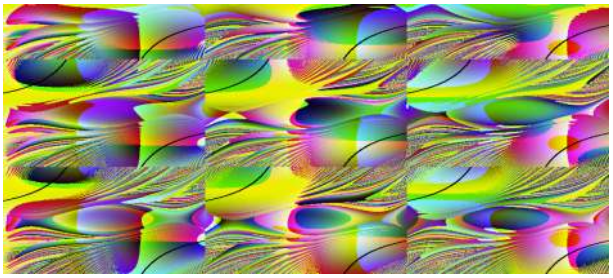
(a) iSpiral(). hpSphere(pSteiner()).out()



(b) pTorus(). ipTorus(pTorus()).out()



(c) iCissoid().kaleid().modulate(pSphere(1, 6)).out()



(d) pCrossCap().pSpiral(iDevil().repeat()).out()

Figure 3. Screenshots of example sketches created with a combination of FCS and core Hydra functions, with the code used to generate them in the captions.

On the opposite end, the *curatorial* approach uses sound to express salient features of the data. The *allusive* approach lies between the curatorial and the generative, emphazising the data source while taking creative liberty with how it is mapped to sound.

FCS can be approached along all three strata, while likely aligning most closely with the *allusive* stratum. One can use it to create aesthetically pleasing visuals without regarding the mathematical nature of the functions being used. On the curatorial end, FCS can depict shapes with implicit curves, as in the top right of figure 2, as well as the interaction between various functions used to represent the shapes. These visualizations may not be particularly well suited for purely mathematical research, since they are depicting time varying patterns rather than static curves and surfaces. Additionally, the author incorporates time in varying ways in the functions for parametric surfaces and implicit curves. These functions still *alude* to the geometric objects they represent, yet are slightly customized based on the author's aesthetic choices.

FCS is quite accessible, as one can run Hydra and its extensions in the browser. The extension also integrates well into a quickly growing Hydra ecosystem. In addition to exploring interactions between various representations of curves and surfaces, and can interface FCS objects with existing Hydra objects, such as oscillators, Voronoi tessellations, and noise (figure 3).

In addition to adding a tool to the Hydra ecosystem for creating striking visuals, FCS can serve as a conceptual frameworks for artistic applications involving mappings where the inputs and outputs possess different dimensionalities. For example, one could use this approach to generate feedback between music and visuals. Consider an audio signal as a one dimensional function $A(t)$, which denotes the amplitude of the signal at time t. $A(t)$ can serve as the control parameter for a parametric curve, modulating a texture over time. One could also use an implicit surface equation to map the resulting RGB color values to scalar values. One could then choose a location on the screen, say with a mouse, and then multiply $A(t)$ by the value obtained at that position.

A Code Implementations

Using the FCS extension in Hydra, we can write the following code to produce the final image in figure 2:

```
pTorus()
    .pCircle(iCircle())
    .iSphere()
    .out()
```

In this sequence, we use a parametric circle, which take the output of an implicit circle as input, to modulate the texture produced by a parametric torus, whose colors are then mapped to greyscale with an implicit sphere. We describe the implementation of each of these functions below.

Hydra functions implement glsl shaders, and one can create a function implementing a custom shader with the **setFunction** function. Code for the entire extension can be found at https://github.com/ymaltsman/Hydra-FCS.

A.1 Implicit Curves

Implicit curves are implemented as **src** objects, which take a two dimensional position as input and output a four dimensional color. An implicit circle is implemented as follows:

```
setFunction({
    name: 'iCircle',
    type: 'src',
    inputs: [
        {
        type: 'float',
        name: 'amp',
        default: 10,
        },
        {
        type: 'float',
        name: 'freq',
        default: '.1',
        }
    ],
    glsl:
    `
        _st = _st*2.0 - 1.0;
        float x = _st.x;
        float y = _st.y;
        float u = x*x + y*y;
        u = fract(amp*sin(time*freq)*u);
        return vec4(u, u, u, 1.0);`
})
```

A.2 Parametric Curves

Parametric curves are implemented as modulation objects, which take both a color and a position as input and output a position. A parametric circle is implemented as follows:

```
setFunction({
    name: 'pCircle',
    type: 'combineCoord',
    inputs: [
      {
        type: 'float',
        name: 'a',
        default: 1.0,
    ],
    glsl: `
      float radius = a;
      float angle = length(_c0);
      float x = radius * cos(angle);
      floaft y = radius * sin(angle);
      return vec2(_st.x*x, _st.y*y);
})
```

A.3 Parametric Surfaces

Parametric surfaces are also implemented as **src** objects, mapping position to color. We implement a parametric torus as:

```
setFunction({
    name: 'pTorus',
    type: 'src',
    inputs: [
        {
        type: 'float',
        name: 'a',
        default: 1,
        },
        {
        type: 'float',
        name: 'c',
        default: 1,
        }
    ],
    glsl:
    `
        _st = _st*2.0 - 1.0;
        float x = _st.x;
        float y = _st.y;
        float r = (c + a*cos(x))*cos(y);
        r = fract(r-time);
        float g = (c + a*cos(x))*sin(y);
        g = fract(g-time);
        float b = a*sin(x);
        b = fract(b-time);
        return vec4(r, g, b, 1.0);`
})
```

A.4 Implicit Surfaces

We implement implicit surface representations as **color** functions, which both receive and output color. An implicit sphere is implemented as:

```
setFunction({
    name: 'iSphere',
    type: 'color',
    inputs: [],
    glsl: `
        float r = _c0.r;
        float g = _c0.g;
        float b = _c0.b;
        float u = r*r + g*g + b*b - 1.0;
        u = fract(u);
        return vec4(u,u,u,1.0);
})
```

References

[1] Karl Bittner. 2020. The Current State of the Art in Real-Time Cloud Rendering With Raymarching. (2020).

[2] David Cox, John Little, Donal O'shea, and Moss Sweedler. 1997. *Ideals, varieties, and algorithms*. Vol. 3. Springer.

[3] Peter Kirn. 2019. A free, shared visual playground in the browser: Olivia Jack talks Hydra - CDM Create Digital Music. https://cdm.link/2019/02/hydra-olivia-jack/. [Accessed 01-06-2024].

[4] Danbo Lai and Alexei Sourin. 2011. Visual immersive mathematics in 3D web. In *Proceedings of the 10th International Conference on Virtual Reality Continuum and Its Applications in Industry*. 519–526.

[5] Milad Khosravi Mardakheh and Scott Wilson. 2022. A strata-based approach to discussing artistic data sonification. *Leonardo* 55, 5 (2022), 516–520.

[6] Alex McLean. 2014. Making programming languages to dance to: live coding with tidal. In *Proceedings of the 2nd ACM SIGPLAN international workshop on Functional art, music, modeling & design*. 63–70.

Received 2024-06-02; accepted 2024-07-02

Trane: Musical Janet on the Web

George Ash
Independent
London, United Kingdom
05gash@gmail.com

Abstract

Trane is a domain specific language and environment for livecoding music on the web. It gives the programmer control over instruments, effects, their connectivity, and the ability to sequence well-timed events. In this paper we explore the motivation behind the language, its design, and implementation.

CCS Concepts: • **Applied computing** → **Performing arts**; **Sound and music computing**; • **Information systems** → *Web interfaces.*

Keywords: livecoding, music, lisp, janet, audio, web, browser

ACM Reference Format:
George Ash. 2024. Trane: Musical Janet on the Web. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3677996.3678285

1 Introduction

Live coding refers to a practise in which a programmer makes or modifies software live. [5] In the context of audio or music, the programmer might alter a process that generates sound and should be able to hear the modifications they are making. The number of live coding languages for audio has grown significantly since its first inception, with each offering different expressiveness to the programmer. Some languages might be a natural choice for complex drum sequencing, others might offer easy control over harmony generation, another could specialise in low-level DSP control.

2 Motivation

Trane was created from the desire for an ergonomic, mouldable, high-level language for audio sequencing, with easy control over sound-design. A subset of the language allows the programmer to define an audio graph declaratively, which was inspired by physical modular synthesizers. The other half of the language lets the programmer schedule events against this graph. Sonic Pi was a large influence in the design of the sequencing side of the language. Most notably the (`live_loop` ...) [2] and virtual time semantics [3] are borrowed.

Lowering the barrier to live-coding, audio synthesis, and lisp-style programming were also important motivations when implementing Trane, as was the ability to easily share patches and programs. The web was a natural target when considering this, with the benefit of being cross-platform.

Previous Work. Previous artistic languages or environments have, directly or indirectly, influenced the design and implementation of Trane.

Several influential languages are dependent on the SuperCollider Environment [15]. ixilang [14] contains an interpreter inside of SuperCollider, and offers high-level control over synth definitions and samples. Tidalcycles [16] is embedded in Haskell, and allows for complex pattern creation using a cyclical notion of time. Sonic Pi [3] is a ruby-based DSL, and introduces musical notions of time and concurrency. Overtone [2] is a musical programming library written in Clojure that gives the programmer control over instruments and event sequencing.

There are other languages that do not require the SuperCollider engine, and can be hosted elsewhere. Glicol [13] is a graph-oriented live-coding language implemented in Rust, which can be run in a multitude of environments. Strudel [1] is a javascript port of Tidalcycles, which uses Web Audio[17] for synthesis.

More recent work on Janet playgrounds for graphics have been the largest influence in the development of Trane.
Bauble [11], is an interactive Janet playground for creating ray-marched Signed Distance Functions [10]. It runs in the browser, and translates a Janet DSL into GLSL fragment shaders.
Toodle [12] is also a browser-based playground, in which the programmer writes Janet code to control turtle graphics.

3 Structure of a Trane Program

Trane is a musical DSL written in the Janet [6] language. Programs in Trane are conceptually programs of two halves. In one half, there is a declarative syntax that represents an audio graph. That is, modules and wires between them. In

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
FARM '24, September 2, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3678285

the other half, there are language constructs for sequencing well-timed audio events.

3.1 Defining an Audio Graph

Trane is inspired from physical modular synthesizers. Modules can generate audio signals, or they might process input signals to form a new signal.
Modules can be wired together to form a patch. A patch can be represented abstractly as a directed graph, with nodes representing modules, and edges representing the wires between them. Similarly, some audio synthesis engines use a graph based structure to model the signal flow through a series of nodes.[18][15] Audio signals are generated in nodes, travelling through edges where they are processed by connected nodes, until finally they arrive at the at the buffer of an interface and your signal is made audible.

In Trane, to define such a graph, we can instantiate modules and the wires that connect them. For example, we might want to generate a sine wave, amplify it, then output the amplified wave to a speaker.

This can be achieved with the following code:

```
(chain
  (oscillator :hello-world "sine")
  (gain :hello-gain)
  :out
)
```

3.2 Modules

Modules are declared at the top-level of a trane program. They can be created using the relevant module instance macro.

Each module requires a name, so that events can be sent to them. The arguments that follow the name are module dependent. For example, the macro
(`sample name URL pitch`) will create a new pitched sampler module, with an audio sample located at the location in `URL` and a reference pitch.

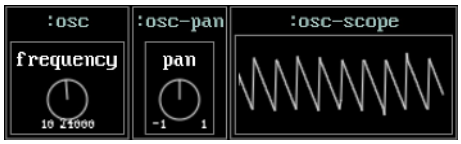


Figure 1. The Trane user interface: Each box represents a module. Here the programmer has instantiated a sawtooth oscillator, a panner, and a scope.

3.3 Wires

Modules can be wired together in two ways.

The (`wire from to param`) macro will wire two modules together. The output signal in `from` will be connected to the default input in `to`. The `param` argument is optional, and will override the default input in `to` to one of its parameters. An example of this is given in figure 3.

Modules can also be wired together using the (`chain & modules`) macro. Each module in `modules` will be wired from the previous module to the next module to form a chain.

3.4 Time, Notes

The unit of time in Trane is the beat. This means all events, durations and time-based instrument parameters are defined in beats.

MIDI notes are represented via a keyword or by their integer value. `[:Cs3 :cs3 :db3]` all refer to the same note. Chords can be represented by mutable arrays or tuples of notes. If a programmer wants to do arithmetic on notes they can convert to an integer with (`note :cs3`).

A (`play module note`) macro will schedule a note to play on a particular module. Optionally this macro will take a named `:dur duration` parameter, which defines how long the note will play for in beats.

3.5 Events

Events in Trane are timed operations on the audio graph. They have a destination (a module or parameter in the graph) and allow for precise scheduling of notes, or parameter changes. Each module in the graph may have a number of parameters, for example an oscillator module will have a frequency parameter. Other modules may accept play events, which might represent the start of a note, or the start of a particular sample.

3.6 Sequencing Events

Once we have a valid graph, we can send audio events to any of the modules in the graph in order to change parameters of the modules.
In order to send notes at a specific, accurate time, we can send events using the (`live_loop name & body`) macro.

```
(live_loop :gain-changer
  (lin :hello-gain :gain 0)
  (sleep 1)
  (lin :hello-gain :gain 1)
  (sleep 1)
)
```

Figure 2. Demonstrates scheduling parameter changes on the audio graph. This live_loop will schedule changes to the `:gain` knob on the `:hello-gain` module. Each loop body will schedule a linear interpolation of gain from a value of **0** at time t to **1** at time $t + 1$. It implements a low-frequency triangle wave oscillator with a period of 2 beats.

A (`live_loop ...`) can be viewed as a function of time. A virtual time is set as a thread-local variable when a loop body is evaluated. Each call to (`sleep t`) will advance the local time variable by `t` beats. This local variable is then used in each event producing macro. Event producing macros will add to a hidden array of events that each (`live_loop ...`) yields once its evaluation completes, along with the final virtual time.

4 Parameter Changes

Parameters are represented in the interface as knobs. They can be changed with a mouse or can be mapped to MIDI CC controllers. Knobs may have a logarithmic scale where they represent a quantity that is more uniformly perceived in the logarithmic domain [19], such as frequency and volume.

4.1 From the Code

The parameter changing macros all accept module and parameter options. These must be set. The module specified is checked to exist in the graph, and the parameter of parameter-name is checked to exist on the module.
(`lin module-name param-name p`) will linearly interpolate the specified module's parameter from its current value to the target value `p` at the scheduled time.
(`expo module-name param-name p`) behaves like (`lin ...`), but the interpolation behaviour is exponential instead of linear.
(`target module-name param-name p time-constant`) will approach the target `p` exponentially, starting at the scheduled time. The rate at which it approaches is given by `time-constant`
(`itarget module-name param-name p`) will instantaneously change the target parameter to `p` at the specified time.

4.2 From the Graph

In addition to scheduling parameter changes from live_loops, parameters can also be changed using a modulator signal from the graph.

4.3 Knobs

Module parameters can also be changed via the graphical interface. Each module parameter will have an associated knob, whose value can be changed via clicking and dragging. 4. Parameters can also be mapped to MIDI controls by shift-clicking a knob and then moving an associated MIDI CC emitter.
Each knob will advertise its parameter name, and a minimum and maximum range. Apart from being a useful performance and compositional aid, these also act as documentation to the programmer.

4.4 Which to Choose?

With a few different methods to change module parameters inside of the graph, it could be confusing to decide which

```
(lfo :lfo-tremolo "sine")

(chain
  (oscillator :drone "sawtooth")
  (gain :drone-gain)
  :out
)

(wire :lfo-tremolo :drone-gain :gain)
```

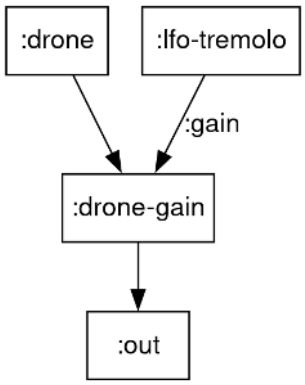


Figure 3. The code, and audio graph showing a low-frequency oscillator modulating the gain on another oscillator to produce a tremolo effect.

Figure 4. The default **gain** module. Its value can be changed with a mouse, or by mapping it to a midi CC event.

to choose when programming live. It's really up to the programmer or performer which of these to choose based on their goal. If strict timing is desired, for example in the attack phase of a note envelope, then parameter changes via code might be the right choice. However, graph based parameter changes can work in tandem with parameter changes from code or the knob interface. For example, we might wish to twist a frequency knob on a low-frequency-oscillator that is wired up to the volume of another audible-range oscillator.

5 Implementation

Trane is implemented using modern web standards such as ECMAScript 2015, Web Audio [17], and WebAssembly [9]. The next section will go over particular parts that might be non-standard for a web-application. It will follow the sequence of events that are triggered from a change to the code, and lead up to an audible sound.

5.1 Compilation

Changes to the code will trigger compilation of the buffer in the editor. During compilation, macros are expanded and certain compile time checks are made. A feature of trane, designed to avoid confusion and bugs during performance, is that connectivity constraints on the audio graph are made when the buffer is compiled. The compilation step will not complete if wires do not have valid sources or destinations.

Additionally, event destinations will be checked against the graph. For example a (`play ...`) macro will throw an error if the destination module it is sending to does not exist at compile time.

The compilation step yields a marshalled representation of the environment called an *image*. An *image* contains the environment data that is the result of any compile time processes, along with function and macro definitions. In Trane this will consist of audio graph connectivity information. Other information contained in the *image* are (`live_loop ...`) names and marshalled representations of their body, so that they can be scheduled and executed when required.

5.2 Code (re)Loading

Successful compilation yields an executable image. It's at this point that the user can swap out existing code for the new code. Once a code reload is triggered by the user we switch the images that are used by the loop manager. The next time a loop is scheduled, the loop manager will use this new code to generate events until the (`live_loop ...`) function yields a result. This method of hot code reloading was inspired by Erlang/OTP [4]

Each (`live_loop ...`) that isn't present in the new image will be cleaned up by the loop manager, but any events that were scheduled before its deletion will still be audible.

5.3 Graph Manager

The audio graph is implemented using the Web Audio standard [17]. During each compilation a set of modules and wires are generated. Once a code reload is triggered, a diff operation is made between the existing, running graph and the new one. Any modules or wires that are not present in the new image are disconnected and deleted from the running graph. Any new modules are created and wired up.

During this graph change, audible popping might be discernible if a part of the graph that is being changed contributes to the final, audible signal. While methods to change the graph with fewer such artefacts exist, for instance by smoothly mixing between old and new graphs after a code change, we welcome such artefacts: Particularly being analogous to real audio equipment, which might produce pops as instruments are being wired together.

Performers with an aversion to these pops might choose to introduce new sub-graphs behind a mixer or zeroed gain node if they choose.

```
(chain
  (oscillator :drone "sine")
- (gain :drone-level)
  :out
)

(chain
  (sample :piano "piano_c3.wav" :C3)
- (reverb :piano-verb "hall_impulse.wav")
+ (Dlay :piano-delay 1.5)
  (gain :piano-level)
  :out
)
```

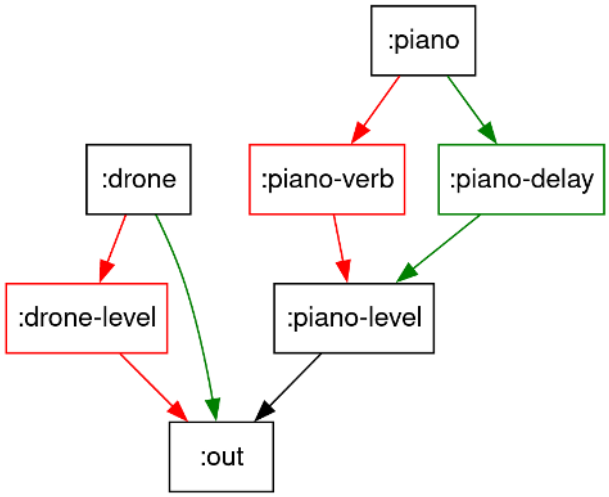


Figure 5. Operations on the graph after a code change. Old (red) modules and wires are discarded before new (green) modules are wired up.

5.4 Scheduling

A given executable (`live_loop name & body`) will produce a list of events. These all have scheduling times, in beats, which are routed to the respective graph nodes ahead of time.

Typically, in a browser, the ability to schedule events at a particular time in the future is provided to the programmer through the `setTimeout(fn, delay)` function. This API accepts a function and a delay in milliseconds. However, there are no real guarantees from the specification or implementations that the function (and any sound producing side effects) will be evaluated at exactly that time [7]. This method to schedule events in the future does not give us sufficient accuracy for most audio applications. The situation is improved by the guarantees made by the WebAudio API. Certain events can be scheduled with sample-level accuracy if they are made far enough in advance.

To take advantage of this, Trane will try to execute the body of a live_loop some fixed time ahead of its virtual time.

In this way we can generate notes or events for the future so that they can be accurately scheduled with the WebAudio API. Trane borrows the concept of virtual time semantics from Sonic Pi. live_loops are scheduled with sufficient lead time.
A sufficient lead time will be enough to evaluate the main body of the live_loop and avoid any timing jitter from the call to `setTimeout`. In Trane this is 250ms. A warning will be generated if the live_loop body exceeds its time budget so that its events can be accurately scheduled. It's the responsibility of the programmer to keep to this budget. In the future we wish to give the programmer control over this timing constant.

6 Synchronization

Each (`live_loop ...`) is run in a separate thread, and do not easily communicate. To achieve synchronization between two live_loops the loop manager will schedule all new loops to start on every bar boundary.
Unlike Sonic Pi, Trane does not provide explicit cue or sync primitives in the language. Instead, programmers are encouraged to interpret live_loops as functions of time, so that they can eventually reach synchronicity through their output, or alternatively begin simultaneously on some future measure. Some utilities in the language might make this easier;

```
(til measure-length)
```

will return the time until the start of the next measure, where *measure-length* denotes the length of the measure. The example in figure 6 demonstrates its use.

Another useful utility for writing functions of time is

```
(timesel list change-every)
```

Which will select an item from *list* based on the current time. It will select the next element from *list* after *change-every* has elapsed. This selection wraps around the whole of list.

```
(live_loop :loop_a
  (sleep (til 64))
  ... play some notes
  (sleep 64)
)

+(live_loop :loop_b
+  (sleep (til 64))
+  ... play some notes in sync with :loop_a
+  (sleep 64)
+)
```

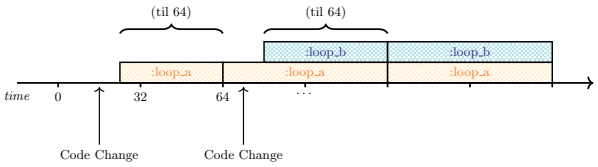


Figure 6. Demonstrates live_loop synchronisation in trane. :loop_a is run first, and sleeps until the next 64 bar boundary before scheduling any notes or events. :loop_b is then started at some point during the measure. To keep in sync with :loop_a it also sleeps until the next 64 bar boundary. After this they both produce events over the same virtual time.

7 Patterns

Trane has rudimentary support for musical patterns, similar to those seen in tidalcycles [16] or strudel [1]. A pattern in trane is encoded in an array or tuple of elements. These elements can represent notes, parameter values, or something more abstract. Patterns are evaluated over a time period, and the time between elements in the pattern is divided equally.

```
(P patt measure-length)
```

This macro will evaluate the pattern, dividing *measure-length* recursively between the elements of *patt*. It will return a list of tuples `[element, time]` that can be easily scheduled, manipulated, or combined in a (`live_loop ...`)

```
(P [[:c3 [:tie :e3]] :g3 [:c3 :e3 :d3] :c3] 4)
```

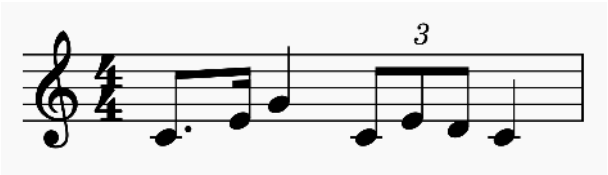


Figure 7. Demonstrates how time is divided up inside of a musical pattern.

7.1 Parsing Expression Grammars

The pattern evaluator takes as input a Janet tuple or array, which might be tedious, too verbose, or error prone to write manually. A string that matches a particular grammar might be a more enjoyable way to write patterns.

A Parsing Expression Grammar (PEG) is a concise, unambiguous set of rules that describe a top-down parser. [8] The Janet language comes bundled with a powerful Parsing Expression Grammar (PEG) engine that will match and capture a string which matches a particular grammar.
The (`peg/match peg text`) utility will match a string `text` to the grammar in `peg`. This utility is also available inside of trane, making experimentation with new musical grammars easy within the environment. The snippet of code below shows how a grammar can translate a small subset of the Tidalcycles mininotation [16][1] to a Trane pattern.

```
(def little-grammar
  ~{
    :rest (/ "~" :rest)
    :tie (/ "-" :tie)
    :note (cmt (<- (*
        (range "ag")
        (at-most 1 (+ "s" "b"))
        (range "09")))
      ,keyword)
    :subdivide (group (* "[" :sequence "]"))
    :repetition (cmt
      (*
        (+ :subdivide :note :rest :tie)
        "*"
        (number :d+))
      ,rep)
    :item (+ :repetition :note :subdivide :rest :tie)
    :sequence (+ (* :item :s :sequence) :item)
    :main (* :sequence -1)
  }
)

(peg/match
  little-grammar
  "a2*4 [g2 a2 b2 ~] b2 [a2 b2 c3 ~] c3"
)
```

8 Summary and Future Work

We've described Trane: a musical DSL embedded in the Janet language. Trane is released under the GNU AGPL license, the code is available at https://github.com/gwegash/trane. Despite being a complete environment to experiment with livecoding music and audio synthesis, Trane has need for some improvements to make it a more fun and inclusive environment to make music in. Updates to the pattern evaluator, better error reporting, more and better quality modules, and improved interaction and accessibility features are all planned for future releases.

Acknowledgments

Ian Henry deserves most thanks and gratitude for their work and documentation on Janet embedding in the browser.

References

[1] 2023. *Strudel: Live Coding Patterns on the Web.* Zenodo. https://doi.org/10.5281/zenodo.7842142
[2] Samuel Aaron and Alan F Blackwell. 2013. From sonic Pi to overtone: creative musical experiences with domain-specific and functional languages. In *Proceedings of the first ACM SIGPLAN workshop on Functional art, music, modeling & design.* 35–46. https://doi.org/10.1145/2505341.2505346
[3] Samuel Aaron, Dominic Orchard, and Alan F Blackwell. 2014. Temporal semantics for a live coding language. In *Proceedings of the 2nd ACM SIGPLAN international workshop on Functional art, music, modeling & design.* 37–47. https://doi.org/10.1145/2633638.2633648
[4] Joe Armstrong. 1996. Erlang—a Survey of the Language and its Industrial Applications. In *Proc. INAP*, Vol. 96. 16–18.
[5] Alan F. Blackwell, Emma Cocker, Geoff Cox, Alex McLean, and Thor Magnusson. 2022. *Live Coding: A User's Manual.* The MIT Press. https://doi.org/10.7551/mitpress/13770.001.0001 arXiv:https://direct.mit.edu/book-pdf/2239274/book_9780262372633.pdf
[6] Janet Contributors Calvin Rose. 2021. *Janet Language.* https://janet-lang.org/
[7] MDN Contributors. 2023. *SetTimeout Documentation.* https://developer.mozilla.org/en-US/docs/Web/API/setTimeout
[8] Bryan Ford. 2004. Parsing expression grammars: a recognition-based syntactic foundation. In *Proceedings of the 31st ACM SIGPLAN-SIGACT symposium on Principles of programming languages.* 111–122. https://doi.org/10.1145/982962.964011
[9] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Barcelona, Spain) *(PLDI 2017).* Association for Computing Machinery, New York, NY, USA, 185–200. https://doi.org/10.1145/3062341.3062363
[10] John Hart. 1995. Sphere Tracing: A Geometric Method for the Antialiased Ray Tracing of Implicit Surfaces. *The Visual Computer* 12 (06 1995). https://doi.org/10.1007/s003710050084
[11] Ian Henry. 2023. *Bauble.* https://bauble.studio
[12] Ian Henry. 2023. *Toodle.* https://toodle.studio
[13] Qichao Lan and Alexander Refsum Jensenius. 2021. Glicol: A Graph-oriented Live Coding Language Developed with Rust, WebAssembly and AudioWorklet. In *Proceedings of the International Web Audio Conference (WAC '21)*, Luis Joglar-Ongay, Xavier Serra, Frederic Font, Philip Tovstogan, Ariane Stolfi, Albin A. Correya, Antonio Ramires, Dmitry Bogdanov, Angel Faraldo, and Xavier Favory (Eds.). UPF, Barcelona, Spain.
[14] Thor Magnusson. 2011. The IXI Lang: A SuperCollider Parasite for Live Coding. (01 2011).
[15] James McCartney. 2002. Rethinking the computer music language: Super collider. *Computer Music Journal* 26, 4 (2002), 61–68. https://doi.org/10.1162/014892602320991383
[16] Alex McLean. 2014. Making programming languages to dance to: live coding with tidal. In *Proceedings of the 2nd ACM SIGPLAN international workshop on Functional art, music, modeling & design.* 63–70. https://doi.org/10.1145/2633638.2633647
[17] Hongchan Choi Paul Adenot. 2021. *Web Audio API Specification.* https://www.w3.org/TR/webaudio/
[18] Miller S Puckette et al. 1997. Pure data. In *ICMC.*
[19] S. S. Stevens and J. Volkmann. 1940. The Relation of Pitch to Frequency: A Revised Scale. *The American Journal of Psychology* 53, 3 (1940), 329–353. http://www.jstor.org/stable/1417526

Received 2024-06-02; accepted 2024-07-02

From Konnakol to Live Coding

Alex McLean
Then Try This
Sheffield, United Kingdom
alex@slab.org

Abstract

Konnakol is a South Indian, Carnatic musical practice involving the vocal recitation of algorithmic, geometric rhythmic patterns of non-lexical syllables. I reflect on the experience of learning konnakol rhythms, and of adapting the TidalCycles and Strudel live coding environments to better represent Konnakol-inspired rhythms, based on the concept of the metrical tactus. I share visualisations of examples, and the development of a hybrid practice that integrates vocal patterns with live coding. I conclude by considering the issue of cultural appropriation around this work.

CCS Concepts: • **Applied computing** → **Performing arts**; **Sound and music computing**; • **Theory of computation** → **Models of computation**; • **Software and its engineering** → **Domain specific languages**; **Functional languages**.

Keywords: carnatic music, konnakol, konnakkol, live coding, rhythm

ACM Reference Format:
Alex McLean. 2024. From Konnakol to Live Coding. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3677996.3678290

This work is licensed under a Creative Commons Attribution 4.0 International License.

FARM '24, September 2, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3678290

> In all Indian musical traditions, learning, rationalization and sharing are part of an internal process within and between artists. The magnificence of this non-scribal technique is the acceptance of diversity and the possibility of continuous musical change. The internalization of structure and form in the art, the role of memory and rediscovering without written reference keeps the music moving and varied. Every voice is not identical; the possibility of music bending and swaying across regions, traditions and time is real. At no point is the music etched in stone, trapped in time, a stone tablet of commandments.
>
> T. M. Krishna, "Music and Justice" [5]

1 Introduction

The above epigraph from T. M. Krishna celebrates the role of living memory and continuous change in 'non-scribal' musical practice. From my European perspective this is refreshing, when printed staff notation has such a hold over music-related scholarship in the West. However, this oral/scribal dichotomy is less distinct in those computer-supported musics where notations can be open to continual change, for example the 'from-scratch' style of live coding, where notations are written, manipulated and (finally) deleted live, gaining an improvisatory, time-bound, ephemeral quality. In these respects, live coding is closer to active speech than printed notation. This suggests that the comparatively new practice of live coding has a great deal to learn from the far older (and therefore far more developed) practices referred to by T. M. Krishna.

Towards this end, in the following I compare the South Indian vocal practice of *konnakol* with contemporary algorithmic music, as a means to develop new live coding practice. The conduit for this work is the software development of the TidalCycles and Strudel live coding environments, adapting their representations for musical pattern inspired by Carnatic rhythmic structures. I will conclude with a search for musical meaning in this approach, including consideration of cultural appropriation, and the possibilities of creating performances that shift between live coding of the computer and of the self.

2 Konnakol

Konnakol is a Carnatic musical practice with ancient roots, developing since around 200CE [14]. Konnakol involves the oral recitation of *solkattu* phrase groups of vocable words,

which are non-lexical, but closely associated with articulations of the mridangam drum, as well as movements within the bharatanatyam dance tradition. As a non-scribal, oral tradition, konnakol is generally transmitted and learned through recitation and listening rather than via notation. Konnakol rhythms are highly complex and heavily syncopated with frequent changes of speeds, and e.g. addition/subtraction of beats from successive repetitions. Still, there is always a steady underlying pulse, where konnakol artists are able to perfectly match their rhythmic transformations to a particular *tala* structure. While performing, konnakol artists mark the tala with their hands, for example by repeating a sequence of clap-finger counts (laghu) and clap-waves (drutam).

As a mono-lingual English speaker, I am unable to access much of the literature on the art form, but in English "The Art of Konnakkol" by Trichy Sankaran [14] is an excellent practical introduction which covers some historical and cultural context, and vocalist and activist T.M. Krishna is a key reference for cultural and political background on Carnatic music [7]. Lisa Young has made her Masters and PhD theses on konnakol available [15], Rafael Reina has a book on applying Carnatic rhythmical techniques to Western Classical music [12], and David Nelson has published a Solkattu Manual [11]. For a computer music perspective, an interview by Rotherham-based musician Mark Fell with the Glasgow-based Indian musician Nakul Krishnamurthy is insightful, exploring improvisation, notation and tradition in Carnatic music [6].

However, nothing can stand in for actually learning and practising konnakol rhythms.[1] Indeed, we should take care not to be too distracted by what Kofi Agawu calls "paper rhythms" [1]; those rhythmic transformations which are apparent on paper, but have no living reality when performed, listened to and danced to. I have been lucky to take elementary konnakol lessons with renowned percussionist B C Manjunath, changing how I think about and make both music and music software.

This learning process has allowed me to develop a sense of internal pulse within a tala, and to feel how rhythms move around within that tala. It has felt like learning rhythm from multiple perspectives – it is one thing to learn to recite a sequence, and something quite different to learn to do so while clapping the tala. The rhythms then feel different depending on whether I am attending to the syllables, the rhythmic structure, or the tala. I sometimes write the syllables down to help understand the structure of a rhythm, but it is only when I leave that notation behind that I begin to *really* learn and feel a rhythm. It is hard to explain in words, but taking in a new rhythm can at times be a slightly frustrating process of learning and forgetting as I come at it from different directions before beginning to understand it as a whole.

Konnakol rhythms are fundamentally algorithmic, and this is why it is possible to recite long, complex compositions without notation; the performer generates music based on simple numerical principles applied to a deep, embodied knowledge of rhythm. Or as B C Manjunath puts it:

"You cannot be complex just for the sake of being complex … [The Spanish dish] paella is complex, it has a lot of elements in it… but it is built out of many simple things. [If they are] imbibed into each other, then that becomes complex. … If you understand that you are trying to make something complex then that is a failure of a musician. The complexity should be for others, but for me it should be simple." [2]

3 TidalCycles and Strudel - Live Coding Languages for Pattern

TidalCycles and Strudel are software environments for live coding that I have been adapting to better represent konnakol-inspired rhythmic structures. TidalCycles (known as Tidal for short) is a domain specific language embedded in the pure functional Haskell programming language, and was developed by myself from 2009, becoming a thriving free/open source project with many contributors [10]. In recent years, Tidal has been ported to several multi-paradigm languages, most notably to JavaScript as "Strudel" [13]. Strudel has largely reached feature parity with Tidal, with some differences following from the different constraints and opportunities of JavaScript, as well as some additional e.g. user interface, visualisation, and tonality features. This paper discusses features implemented in both – for ease of explanation I will refer to the Haskell types when discussing implementation, and for accessibility to the reader with a web browser to hand, I will share examples written in Strudel. They have their own lives, but for the purposes of this paper, lets consider them two sides of the same coin.

As systems for making music[3], Tidal and Strudel enjoy vibrant communities of practice, supporting performances and workshops worldwide. These technical and cultural developments are inseparable, in that Tidal is a software environment for thinking about and creating music, but it is the end-user community of artists that populated that environment and established its musical and cultural meaning. For this reason I consider them to be software environments and not prescriptive software tools. It has been a privilege to be involved in these cultural developments, including through

[1] I use the phrase *konnakol rhythms* advisedly; konnakol is a rich art form, the majority of which I have not touched on. So far my focus has been on rhythm.

[2] Quoted from this interview with B C Manjunath: https://www.youtube.com/live/6kYwZ8S-qBQ?feature=shared&t=1761

[3] Tidal and Strudel have also been used for other pattern-based media, such as making visuals, choreographies and textiles.

online community building, festival organisation and development of the wider algorave movement [3]. The software and community had to develop together; one could not have been imagined without the other.

3.1 Musical Patterns as Functions of Time

The affordances of Tidal, Strudel and other ports follow from their common, core representation of patterns as functions of time. This is inspired by signals in Functional Reactive Programming [4], but with support for both discrete events and continuously varying values within the same `Pattern` datatype.

```
type Time = Rational
data TimeSpan = TimeSpan {begin :: Time,
                          end :: Time}
data Event a = Event {whole :: Maybe TimeSpan,
                      active :: TimeSpan,
                      value :: a}
data Pattern a =
    Pattern {query :: TimeSpan -> [Event a]}
```

A pattern then, is a function from (rational) time to events. A pattern function is queried for a particular timespan window, and returns events taking place within that window. Each such event consists of a value, and a timespan during which that event value is active within the queried time window. Discrete events have an additional timespan representing an event's 'whole' timespan. This is needed to represent an event that is a fragment of a larger one; this occurs where an event extends outside the queried window, or where a pattern transformation slices up events into parts.

Because patterns are functions, they are highly composable, in both the computer scientific and musical sense. However one problem with patterns is that there are a diverse multitude of ways to compose them together. As a result, Tidal defines a range of applicative function application and monadic bind/join functions. With a rich combinator library of pattern transformations, Tidal provides rich and flexible domain-specific library, designed to be terse and expressive enough to improvise with in live performance.

Strudel is implemented in JavaScript, and so does not have the benefits of a type system. Nonetheless it is a faithful port of Tidal as described above, including its approach to applicative and monadic-style functional composition. Indeed, all the examples in the following are in implemented in Strudel, so that the reader can play with them in a web browser without having to install Tidal. Thanks to the work of Felix Roos (a lead developer of Strudel), it is even possible to work with Strudel using Tidal's Haskell syntax, via the tree-sitter-haskell project.

3.2 Expanding Tidal and Strudel to Support Konnakol-Inspired Patterns

One limitation of representing patterns with functions is that they are opaque, so that when combining two patterns, it's difficult to fully consider the structure of those patterns. Take for example concatenation – appending one pattern to another. Functions of time are in principle infinite, so we can never reach the end of the first pattern in order to play the second one. Tidal's answer to this is cycle-based composition, where a cycle is roughly equivalent to a measure or bar in Western classical music, but is perhaps closer to the idea of a tala cycle in Indian music. Rather than concatenating whole patterns, Tidal concatenates successive cycles, resulting in a cyclic interleaving of patterns.

```
cat("red green", "blue orange purple");
```

As you can see in the above visual example using Strudel, each cycle is split in half, with the first half containing two events of one quarter of cycle each, and the second half having three events of one sixth of a cycle each.[4] Sometimes however, we might want divide time up 'stepwise', so that each event in the resulting pattern has equal duration, taking up one fifth of a cycle. Indeed, stepwise transformation and composition of patterns is a pervasive feature of Carnatic music. Strudel and Tidal now have an `s_cat` (stepwise cat) function for this:

```
s_cat("red green", "blue orange purple");
```

In the above, all five steps have the same duration. Even for this basic functionality, the `s_cat` function needs to know something about the structure of the two patterns in order to combine them. That is not possible when composing functions of time together, as the structure of the patterns is not known until the resulting function is later queried.

It took two different attempts to support this stepwise functionality. The first was to create a pattern type class, with the existing functional patterns as one instance, plus an additional type instance implementing patterns as data structures. This worked to a large extent, with a great deal of functionality implemented relative to the type class, rather than the two instances. However despite much work, I came to the conclusion that this approach was over-complicated, which showed in the resulting end-user interface. I eventually settled on a simpler approach, based on the idea of maintaining the 'tactus' of a pattern.

[4] The double-quoted text denotes the 'mini-notation', a mini-language implemented in both Strudel and Tidal for describing potentially complex polymetric rhythms. Here they are used to describe simple contiguous sequences.

3.3 Musical Tactus

In music, the *tactus* is a high-level pulse felt in the music, known as the 'clapping rate', or the pulse underlying a piece of music that an individual or crowd chooses to clap along with. There is work to automatically infer the most likely tactus from a signal in the Music Information Retrieval field, however the tactus is often ambiguous. One person might clap twice or half the speed of another, and even more problematically, in polyrhythmic music, each listener must choose which rhythm to clap to. A trained musician might even mark multiple tactuses with different limbs. Including tactus in a musical representation therefore requires this ambiguity to be confronted.

In Tidal and Strudel, the tactus is simply added as an optional pattern field, along with an additional `pureValue` field. In Tidal's Haskell implementation, the type then looks like this:

```
data Pattern a =
    Pattern {query :: State -> [Event a],
             tactus :: Maybe Time,
             pureValue :: Maybe a
            }
```

Stepwise functions can then use the tactus when combining patterns, for example to find the relative proportion to create equal steps in the result of `s_cat`. The `pureValue` field holds a value only for 'stable' patterns, i.e. those of a single value that repeat once per cycle. This is maintained to support tactus calculation; a tactus can only be calculated by pattern transforming functions where certain arguments are stable. On this basis, a range of additional stepwise pattern combinators are defined, all prefixed by `s_`.

The result of our earlier `s_cat` example is the equivalent of `"red green blue orange purple"`; this has a tactus of 5, being the sum of its component tactuses. However, the tactus of our earlier `cat` example is ambiguous. There at least four possibilities:

a) clapping the onsets of component patterns
   → tactus of 2
b) clapping onsets of elements in first component
   → tactus of 4
c) clapping onsets of elements in second component
   → tactus of 6
d) clapping the lowest common multiple
   → tactus of 12

Currently, a) is the default, and either b) or c) can be specified by marking one or the other as being the 'strong' component with a caret (`^`) symbol (e.g. `cat("^red green", "blue orange purple")` for b). d) is the result when both components are marked as 'strong' with `cat("^red green", "^blue orange purple")`, giving the tactus where every event falls on a 'clap'.

3.4 Stepwise Combinators

The combinators are in a fairly early stage of development, with the aim of making konnakol-inspired rhythms easy to express. For example, the `s_taper` function is inspired by `yati` structures, where phrase lengths are successively reduced or increased by one. The below applies `s_taper` to an integer sequence form 1 to 8, first building and then reducing the sequence by one step per repetition, as visualised in Figure 1.

```
stack(
  note("1 2 3 4 5 6 7 8").s_taper("-1 1", 8),
  s("clap clap wave").fast(6)
);
```

This increasing and decreasing is known as a *mridanga yati*, and is shown with the underlying clap-clap-wave pattern often used to mark the rupaka tala. Because the yati pattern has seventy two steps, and the rupaka tala can be viewed as having three, we can fit one cycle of the yati to six cycles of the tala, where we clap or wave every 4 steps of the yati.

```
"red pink".s_tour("black", "grey", "white");
```

The `s_tour` function above adds the "red pink" pattern progressively backwards through each repetition of the black-grey-white sequence. Note that in these examples, only the first repetition is shown, but as a function of time, the pattern will repeat forever.

```
stack(
  "1 2 3 4 5 6 7".s_expand("3 2 1 1 2 3"),
  "wave wave - clap - clap -".fast(6)
);
```

The above use of the `s_expand` function repeats the 1-7 integer sequence for each element in the 3, 2, 1, 1, 2, 3 sequence, but expanded by the given factor. The result is a 84 step pattern, which fits the 7-step "wave wave - clap - clap -" cycle of the *misra chapu* tala, as visualised in Fig. 2.

4 Fitting the Tala

According to Trichy Sankaran, the term *tala* is used in a broad sense to describe all important rhythmic principles, but also in a specific sense to refer to metrical cycles "... composed of traditionally determined rhythmic units which are indicated through conventionally followed hand gestures." The tala provides the ground for rhythm, influencing our experience of rhythmic patterns significantly, being "the foundation upon which intricate devices of cross-rhythm and syncopation are built.[14, pg. 36]

It is important for a piece to align with the cyclic time-measure of a particular tala, by having a total number of

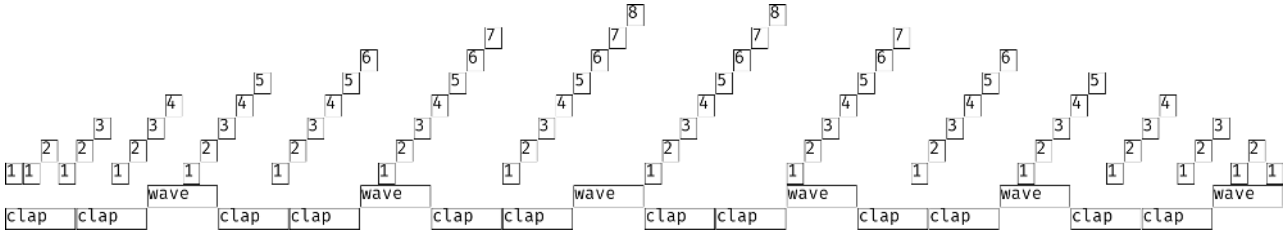


Figure 1. A mridanga yati in rupaka tala

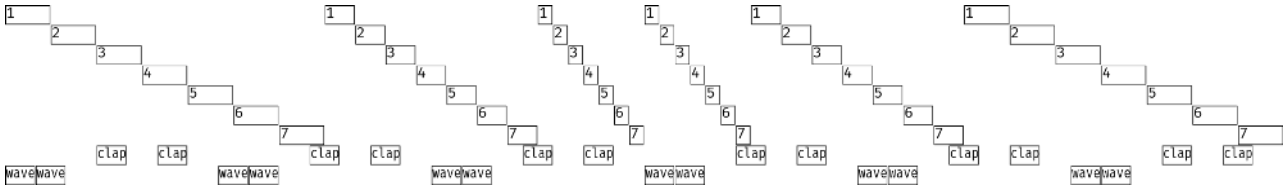


Figure 2. Grouping in misra chapu tala

beats divisible by the duration of the tala. In other words, the piece should perfectly end at the *sam* – the beginning and therefore also the end of a cycle. A subsection within a piece need not fit the tala, however. For example a traditional *Korvai* is divided into two sections, with the second part containing groups of three repetitions of a phrase, each group repeated three times divided by gaps. The piece can then fit the tala by adjusting the repeated phrase or the duration of gaps between groups. The konnakol or mridangam artist calculates how to best fit the piece to the tala through such adjustments.

From a computer scientific perspective, this looks like a problem to solve; we could attempt to formalise an algorithm for fitting a korvai to a tala. However, at this point we should pause and consider where we draw the line of automation.

This notional 'line of automation' divides aspects abstracted and automated by a computer, from those that we creatively and directly engage with as humans. Konnakol is by its nature computational, requiring calculation as part of its artistry, but this does not mean that we wish to automate it. Just as people do not generally start to learn to play the guitar by building a robot, we should be careful not to approach music as a problem to solve. Fundamentally, music is an activity, and as music-makers, we want to immerse ourselves in that activity.

As B C Manjunath points out[5], the body is a much more advanced system for thinking about sound than a computer. Accordingly, live coders use programming languages not to automate the generation of music, but to turn programming itself into music-making, as a means of creating music in and for the moment. In this I argue that they are not only lead by ideas but more importantly by perception of musical outcomes. Live coding environments support immediately turning an idea into music, but it is perception of the musical outcome with all its unexpected rhythmic complexities and juxtapositions that inspire the next edits to drive the music forward. Rather than attempting to codify the rules of konnakol then, I focus on giving space for musicians to choose, adapt and apply rules themselves, inspired by konnakol.

5 Developing Practice

To be meaningful, the technical development of computer music language should be part of a wider development of a culture of creative practice around it. To explore this, I decided to adopt a rule that I would no longer ask my computer to perform a musical algorithm that I could not perform myself. This constraint works to define a creative space, push my rhythmic practice forward, and create possibilities for shifting between and integrating code-driven and vocal performance of an algorithm.

This approach has lead me to develop a performance practice that integrates live coding with algorithmic vocal patterns, involving the audience in clapping the tala and encouraging them to look for and feel the syncopations which result, and perhaps sense the heritage connection between algorithmic patterns [9] in vocal recitation and computer music. Informal feedback from my first public performances trialling this approach at EMF camp (May 2024, Eastnor UK) and Corsica Studios (June 2024, London UK) has been encouraging. Although I have not conducted a thorough survey, informally some audience members indicated that explaining and demonstrating the mathematical aspects of carnatic rhythm using both my voice and code helped them

[5]See B C Manjunath's answer to a question on formalisation of Konnakol: https://www.youtube.com/watch?v=6kYwZ8S-qBQ&t=2697s

understand the role of algorithms in music.[6] This points to the exciting possibility that introducing konnakol-inspired vocal practice to live coding performances could allow audiences to gain deeper appreciation of algorithmic music as a whole, by offering a new way to listen to it.

6 Cultural Appropriation

Developing live coding practice inspired by konnakol as a European is of course an act of cultural appropriation. Although intermixing of cultural sources is common and broadly accepted as a function of music, it can be problematic, whether through misattribution, exploitation, copyright theft, or in the case of Deep Forest's "Sweet Lullaby", all three [5]. Kofi Agawu [2] provides an informative perspective on the pervasive spread of musics of Africa and the African diaspora in the global north. Agawu weighs appropriation against homage, and issues of copyright around traditional music where the notion of ownership does not normally apply. He does see use of traditional African 'timeline' rhythms in western minimalist compositions as potentially positive in drawing affinities with traditional music while creating something novel in a new context, but it is a complicated picture.

Returning to Carnatic music, it is certainly important to understand some of the wider context of its rhythms when making systems inspired by konnakol. This includes recognising its background in ancient sacred texts, as well as reflecting on ongoing debates on the role of caste privilege in music between traditional and progressive voices. For now, I simply acknowledge that towards my aim of enriching algorithmic music with heritage algorithms I *am* appropriating culture, and explore the resulting tensions through scholarship and collaboration, while opening myself to criticism, and opening the software itself as free/open source software.

7 Conclusion

Konnakol throws a great deal of light on 'from-scratch' live coding practice. Too often, algorithmic music is reduced to sequencing, repetition and randomness, but konnakol demonstrates how rich the generative possibilities of pattern can be when the focus is on human memory and perception, rather than fixed notation. By developing hybrid practices that involve both human- and computer-realised algorithms, we are able to embody abstraction so that we can creatively improvise with algorithms through our bodies, reclaiming computation from automation.

Acknowledgements

This work is funded by a UKRI Future Leaders Fellowship [grant number MR/V025260/1].

[6]With thanks to audience member and creative technologist Lu Wilson for this informal feedback: https://mas.to/@TodePond/112695287412247197

References

[1] Agawu, K. 2006. Structural Analysis or Cultural Analysis? Competing Perspectives on the "Standard Pattern" of West African Rhythm. *Journal of the American Musicological Society*. 59, 1 (Apr. 2006), 1–46. DOI:https://doi.org/10.1525/jams.2006.59.1.1.

[2] Agawu, K. 2016. *The African Imagination in Music*. Oxford University Press.

[3] Collins, N. and McLean, A. 2014. Algorave: A survey of the history, aesthetics and technology of live performance of algorithmic electronic dance music. *Proceedings of the International Conference on New Interfaces for Musical Expression* (2014).

[4] Elliott, C. 2009. Push-pull functional reactive programming. *Proceedings of 2nd ACM SIGPLAN symposium on Haskell 2009* (2009).

[5] Feld, S. 2000. A Sweet Lullaby for World Music. *Public Culture*. 12, 1 (Jan. 2000), 145–171. DOI:https://doi.org/10.1215/08992363-12-1-145.

[6] Fell, M. 2022. *Structure and Synthesis: The Anatomy of Practice*. Urbanomic.

[7] Krishna, T.M. 2017. *A Southern Music*. HarperCollins.

[8] Krishna, T.M. 2020. Music and justice. (New Delhi, Dec. 2020).

[9] Mclean, A. 2020. Algorithmic Pattern. *Proceedings of the International Conference on New Interfaces for Musical Expression* (Birmingham, UK, Jun. 2020), 265–270.

[10] McLean, A. 2014. Making programming languages to dance to: Live Coding with Tidal. *Proceedings of the 2nd ACM SIGPLAN International Workshop on Functional Art, Music, Modelling and Design* (Gothenburg, 2014), 63–70.

[11] Nelson, D.P. 2008. *Solkattu Manual*. Wesleyan University Press.

[12] Reina, R. 2015. *Applying Karnatic Rhythmical Techniques to Western Music*. Routledge.

[13] Roos, F. and McLean, A. 2023. Strudel: Live Coding Patterns on the Web. *Proceedings of the 7th International Conference on Live Coding* (Utrecht, Netherlands, Apr. 2023).

[14] Sankaran, T.S. 2010. *The Art of Konnakkol (Solkattu): Spoken Rhythms of South Indian Music*. Lalith Publishers.

[15] Young, L. 1998. *Konakkol: The History and Development of Solkattu : the Vocal Syllables of the Mridangam*. University of Melbourne, Victorian College of the Arts.

Received 2024-06-02; accepted 2024-07-02

Demo: Composable Compositions with Tonart

Jared Gentner
Boston, MA, USA
jagen315@gmail.com

Abstract

This demo introduces Tonart, a language and metalanguage for practical music composition. The object language of Tonart is abstract syntax modeling a traditional musical score. It is extensible- composers choose or invent syntaxes which will most effectively express the music they intend to write. Composition proceeds by embedding terms of the chosen syntaxes into a coordinate system that corresponds to the structure of a physical score. Tonart can easily be written by hand, as existing scores are a concrete syntax for Tonart. The metalanguage of Tonart provides a means of compiling Tonart scores via sequences of rewrites. Tonart's rewrites leverage context-sensitivity and locality, modeling how notations interact on traditional scores. Using metaprogramming, a composer can compile a Tonart score with unfamiliar syntax into any number of performable scores.

In this demo, we will make a small composition using Tonart. We will construct this composition by manipulating notations representing abstract music objects. These will eventually be compiled into a digital score representation, as well as a computer performance. We will add in an especially abstract object at the end, and use our creativity to compile it into something performable.

CCS Concepts: • **General and reference** → **General conference proceedings**; • **Applied computing** → **Sound and music computing**.

Keywords: Music

ACM Reference Format:
Jared Gentner. 2024. Demo: Composable Compositions with Tonart. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy*. ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3677996.3678294

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
FARM '24, September 2, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3678294

1 Demo

1.1 Introduction

This demo is intended to show how Tonart can be used for practical composition with a score containing abstract music objects. Notation softwares such as MuseScore[1] offer a score interface and many surface level transformations over notes and attributes of notes. However, they fail to provide operations over objects like chords, scale degrees, and voice leadings. Existing computer music libraries such as Euterpea [1] provide these operations or the means to build them, but do not provide them within a score-like interface. A score-based DSL is fundamental to leveraging existing knowledge and skills with score notation, a centuries old medium.

1.2 Tonart Syntax

Below is the base Tonart syntax. The undefined nonterminals are extension points which will be elaborated as we progress through the demo.

```
⟨form⟩    ::= ⟨art-id⟩
            | ⟨object⟩
            | ⟨rewriter⟩
            | ⟨context⟩
            | ( @ ( ⟨coordinate⟩* ) ⟨form⟩* )
⟨program⟩ ::= ( define-art ⟨art-id⟩ ⟨form⟩* )
            | ( realize ⟨realizer⟩ ⟨form⟩* )
```

A *context* is a coordinate structure. Tonart's primary coordinate structure is called `music`.

```
⟨context⟩ ::= ( music ⟨<form>⟩* )
```

Music has two coordinates,

```
⟨coordinate⟩ ::= ( interval
                   ( ⟨number⟩ ⟨number⟩ ) )
               | ( voice ⟨id⟩* )
```

which are orthogonal and represent the horizontal (time) and vertical (voice) dimensions of a physical score.

The `@` form is used to embed objects into a context at given coordinates.

Tonart is compiled into Racket[2] by *realizers*.

[1] https://musescore.org/en
[2] Tonart is written as an embedded DSL in Racket. It is a syntactic abstraction, running entirely at compile time, utilizing Racket's module system to implement its own extension mechanisms and libraries. [2]

Note that we will not actually write out the `music` form in this demo, as the realizers we are using treat their toplevel forms as music implicitly.

1.3 Composing In Tonart

We will begin composing with only one object and one realizer.

```
⟨object⟩   ::= ( note ⟨pitch⟩ ⟨accidental⟩ ⟨octave⟩ )
⟨realizer⟩ ::= ( staff-realizer )
```

——

```
(realize (staff-realizer)
  (@ [(interval [0 4]) (voice soprano)]
    (note c 0 5)))
```

This is a note called C5, or, C in the fifth octave. It is sung by the soprano voice, for four beats.

To play this note from the computer, we will convert it into a frequency. A frequency will be represented by the `tone` object. To turn notes into tones, we will use a straightforward rewriter called `note->tone`. Tonart rewriters are not composed into the context like objects; instead, they transform the context by adding, deleting, and modifying existing objects. Forms such as `define-art`, `realize`, and `@` evaluate their subforms from top to bottom, applying rewriters only to the objects denoted above them.

```
⟨object⟩   ::= ...
             | ( tone ⟨frequency⟩ )
⟨rewriter⟩ ::= ( note->tone )
⟨realizer⟩ ::= ...
             | ( sound-realizer )
```

——

```
(realize (sound-realizer)
  (@ [(interval [0 4]) (voice soprano)]
    (note c 0 5))
  (note->tone))
```

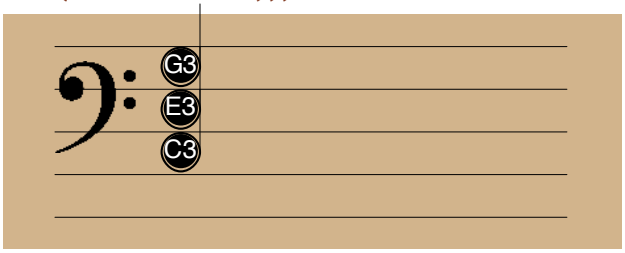
Now we will add a harmony to this note. We will express the harmony as a chord.

```
⟨object⟩   ::= ...
             | ( chord ⟨pitch⟩ ⟨accidental⟩
                 ( ⟨quality⟩ ) )
⟨rewriter⟩ ::= ...
             | ( chord->notes ⟨octave⟩ )
```

——

For this demo, `chord->notes` simply writes in the first three possible notes for each chord within its bounds, creating so-called 'snowman' triads. The number specifies which octave the chords will start in.

```
(realize (staff-realizer)
  (@ [(interval [0 4]) (voice accomp)]
    (chord c 0 [M])
    (chord->notes 3)))
```



We have not yet discussed putting objects one after another in time. We could of course use consecutive intervals. However, this gets unwieldy. We are instead going to establish a concept of a *sequence* of notes.

```
⟨context⟩    ::= ...
               | ( seq ⟨form⟩* )
⟨coordinate⟩ ::= ...
               | ( index ⟨number⟩* )
```

We define a new context. This context is called `seq` and has one coordinate, `index`, representing the position of an object in the context. `seq` contexts can be embedded in music contexts, allowing us to express an ordered sequence directly in a score, without giving specific lengths to the notes or other objects it contains.

Next, we define syntax for rhythms, which are, for our purposes, a series of consecutive durations.

```
⟨object⟩   ::= ...
             | ( rhythm ⟨number⟩* )
⟨rewriter⟩ ::= ...
             | ( apply-rhythm )
```

Now we can do something more complex with the soprano. Note: Instead of writing,

```
(seq
  (@ [(index 0)] (note a 0 3))
  (@ [(index 1)] (note b 0 3))
  ...)
```

I will write `(seq (notes [a 0 3] [b 0 3] ...))`. Below, a melody is bound.

```
(define-art melody
  (seq (notes [c 0 5] [b -1 4] [a 0 4]
              [b 0 4] [c 0 5]))
  (rhythm 3/2 1/2 1/2 3/2 2))
```

Now, we supply a harmony, which the accompaniment will outline.

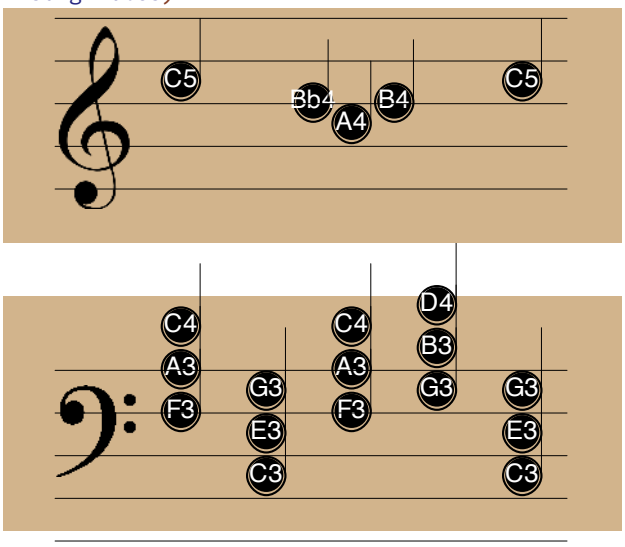
```
(define-art harmony
    (seq (chords [f 0 M] [c 0 M] [f 0 M]
                 [g 0 M] [c 0 M]))
    (rhythm 1 1 1 1 2))
```

We compose them together, and rewrite the piece into notes.

```
(define-art song-notes
  (voice@ (soprano) melody)
  (voice@ (accomp) harmony)
  (apply-rhythm) (chord->notes 3))
```

Note that the use of `apply-rhythm` above applies both the rhythm of the melody, and the harmonic rhythm of the harmony.
To see it visualized:

```
(realize (staff-realizer)
  song-notes)
```



To hear it:

```
(realize (sound-realizer)
  song-notes (note->tone))
```

1.4 Finale

To finish off, we will try adding a more obscure object to our composition.

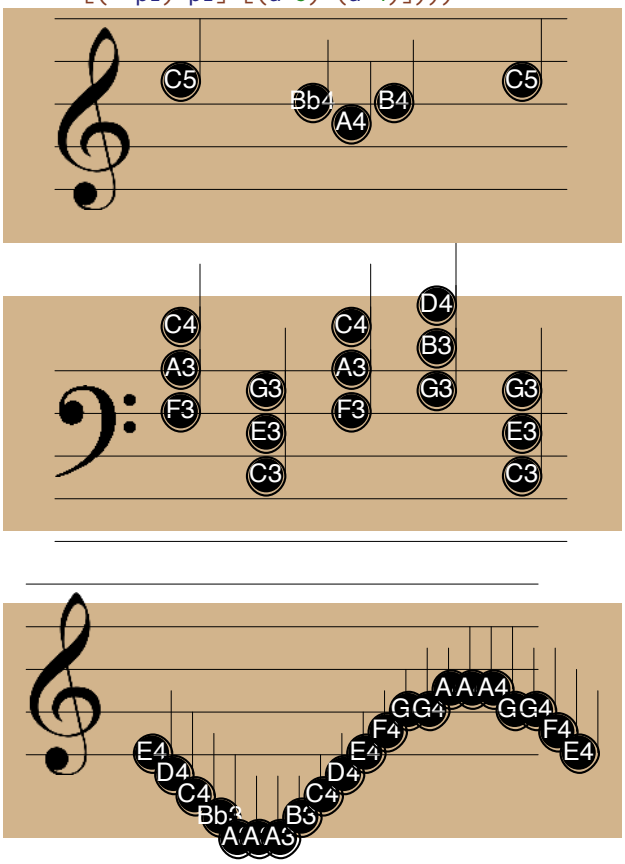
```
⟨object⟩    ::= ...
              | ( function ( ⟨id⟩ ) ⟨expr⟩ )
⟨rewriter⟩  ::= ...
              | ( function->notes
                  ( ⟨number⟩ ⟨number⟩ )
                  ( ⟨note⟩ ⟨note⟩ ) )
```

`function` is a mathematical function.

`function->notes` applies to functions, and it creates a melody that fits within the surrounding harmony and matches the contour of the function.

Here is the finished work.
I will use `(uniform-rhythm 1/4)` as a shorthand for `(rhythm 1/4 1/4 1/4 ...)`.
The function is $sin(x)$ over $(-\pi, \pi)$.

```
(realize (staff-realizer)
  song-notes
  (@ [(voice countermelody)]
    harmony (apply-rhythm)
    (@ [(interval [0 5])]
      (function (x) (sin x))
      (uniform-rhythm 1/4))
    (@ [(interval [5 6])] (note e 0 4))
    (function->notes
      [(- pi) pi] [(a 3) (a 4)])))
```



References

[1] P. Hudak. Euterpea. 2014. http://euterpea.com

[2] Flatt, Matthew. Composable and Compilable Macros: You Want it When? In *Proc. ACM Intl. Conf. Functional Programming*, pp. 72–83, 2002.

Received 2024-06-02; accepted 2024-07-02; revised 2024-06-02; accepted 2024-07-02

The Maquette Monad

Carlos Agon
Sorbonne Université, CNRS, IRCAM, STMS
Paris, France
agonc@ircam.fr

Karim Haddad
Sorbonne Université, CNRS, IRCAM, STMS
Paris, France
karim.haddad@ircam.fr

Gonzalo Romero-Garcia
Sorbonne Université, CNRS, IRCAM, STMS
Paris, France
romero@ircam.fr

Abstract

This article defines the semantics of maquettes in the visual programming language OpenMusic (OM) using monads. A maquette can be seen as a sequencer of functions. Although maquettes have been widely used, their semantic have never been formalized. Formalizing maquettes has multiple benefits; primarily, we aim to provide a better understanding for composers through the use of a mathematical language rather than discursive semantics. In this work, we use a particular case of the state monad and show with examples how this monad is visualized in OpenMusic. The use of these advanced concepts in the field of music and their availability to composers aligns with our intention to bridge the gap between the theoretical and practical aspects of the intersection between mathematics and music.

CCS Concepts: • **Software and its engineering** → **General programming languages**; • **Theory of computation** → *Program analysis.*

Keywords: Computer Music, Functional Programming, Monads

ACM Reference Format:
Carlos Agon, Karim Haddad, and Gonzalo Romero-Garcia. 2024. The Maquette Monad. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3677996.3678293

1 Introduction

A sequencer called *maquette* was introduced since the beginning of the OpenMusic (OM) programming language [1]. The main idea behind the maquette was to construct a sequencer not only for temporal objects but also to allow the inclusion of programs (functions) in time. Musically, this sequencer allows to combine the temporal arrangement of musical structures with their specification in the form of musical algorithms. For a description of how maquettes work, see [4]. For examples of musical pieces composed with maquettes, see [2].

Maquettes are highly efficient and widely used; however, unlike the rest of OM where a formal semantics is given for programs, maquettes were never formally defined and were always seen as a sequencer with a complex behavior difficult to understand. Thirty years after its computer implementation, this article provides a formal definition of what maquettes are by separating their calculative and constructive behaviors.

The semantic of the maquettes defined in this article is based on the notion of monads, and in particular, the state monad. The use of monads in music is not new; a pedagogical approach can be found in [5]. A more practical application, which considers the passage of time as a side effect taken in account by the Timed IO monad, is defined in [6].

This article is organized as follows. Section 2 provides a quick overview of functional programming from the perspective of category theory, since the monad is a concept originating from this theory. In this section, we also provide a description of visual programs in OM from the perspective of functional programming. Additionally, we informally show how maquettes work and particularly its dual function as both a program and a sequencer.

In Section 3, we provide a definition of the monad from a computer science perspective, which will serve as the basis for defining the semantics of the maquettes. In particular, we will attempt to argue for the necessity of the monad to address the question: what does it mean to evaluate a program over time?

Section 4 properly defines the semantics of the maquette using the state monad. Section 5 presents some examples illustrating how the different components of the monad are visualized within the OpenMusic environment. In particular, we will illustrate how the inclusion of a state allows for certain peculiarities inherent to the maquette, which materialize the temporal and computational aspects of a musical piece.

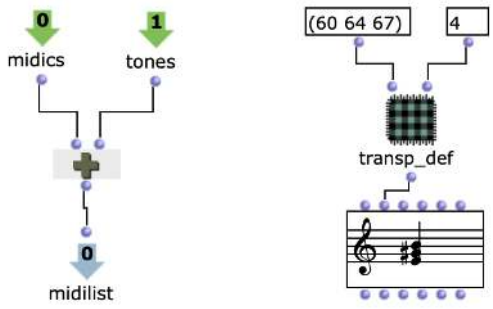
Finally, in the conclusion, we analyze how the integration of advanced functional programming paradigms, such as monads, with musical composition tools offers new perspectives and methodologies for understanding and manipulating the temporal aspects of musical pieces.

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only. Request permissions from owner/author(s).

FARM '24, September 2, 2024, Milan, Italy

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3678293



(a) (b)

Figure 1. Two programs in OM.

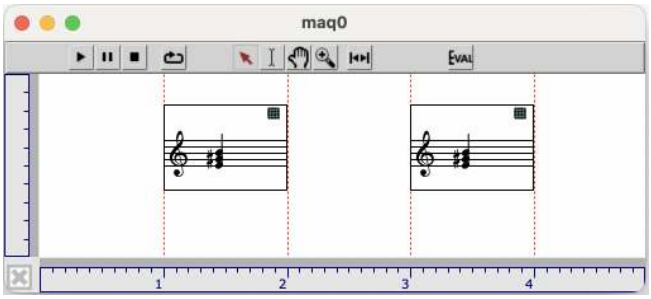


Figure 2. A maquette with two programs.

2 Preliminaries

In category theory, a category consists of a collection of objects and arrows between them, such that these arrows can be composed. Each object in a category has a unique arrow from itself to itself called the identity. For a precise definition of a category, see [7]. Programs in OM are functions and they live in the "category of types". In the context of functional programming objects in this category are types, and the arrows are functions that transform elements of one type into elements of another type.

The program `transp_def` in Figure 1 (a) has the type $List\ Int \rightarrow Int \rightarrow List\ Int$. It tells us that this program has two inputs: a list of integers and an integer, and it produces a list of integers as a result. Graphically, the inputs and the output of the program are represented as boxes, with `midics` and `tones` for the inputs, and `midilist` for the output.

Programs can be composed with other programs, yielding new programs. Composition is at the core of functional programming. We denote the composition of two functions f and g by $g \circ f$. If f has type $a \rightarrow b$ (noted $f : a \rightarrow b$) and $g : b \rightarrow c$, then $g \circ f : a \rightarrow c$. In Figure 1 (b), the program `transp_def` is called with arguments (60 64 67) and 4, producing a list of numbers that are interpreted as MIDI pitches to create a chord. Informally, the program `trans_def` takes a list of MIDI notes *midics* and a number of semitones *tones* and produces a new list *midilist* as a result of transposing *midics* by *n* semitones.

The original idea behind maquettes is to position those programs and their eventual compositions on a timeline to construct a temporal musical structure from their evaluation. Figure 2 shows a maquette in which we have placed the program from Figure 1 (b) at two onsets of 1 and 3 seconds.

The issue with the maquette arises from the fact that when we incorporate functions into a temporal sequencer, it leads us to abandon the traditional semantics of functional programming. This change implies that our programs cease to be "pure" functions. Placing functions in a temporal context introduces two main side effects: time can influence the computation; computation can change the temporal positions of the programs.

Monads come to the rescue for separating the pure functional part of maquettes from the aforementioned side effects. The concept of monad originates from category theory, [3]. Its use in functional programming is due to [9].

3 Monads

The main idea of monads is to change the output type of all functions to a new type that sets the original type in a new context where extra functional aspects can be taken into acocunt. This change of type is made by the help of a functor.

A functor m from a category C to another category $\mathcal{D}$ takes objects x from C and produces object $m\ x$ in $\mathcal{D}$. Moreover, for every arrow $f : a \rightarrow b$, the functor produces a new arrow $m\ f : m\ a \rightarrow m\ b$. In the case of functional programming, beacuse the categories C and $\mathcal{D}$ are the same (i.e., the category of types) we use endofunctors. A classic example of an endofunctor in programming is the *List* functor, [13], which for every type a produces a new type $List\ a$ and for every function $f : a \rightarrow b$ produces a new function $fmap : a \rightarrow\ b \rightarrow List\ a \rightarrow List\ b$.

A monadic function is a function whose output type is modified to enrich the function with "extra-functional" actions. A functor m will transform all functions $f : a \rightarrow b$ into new functions $f' : a \rightarrow m\ b$. These new functions are called monadic functions. The problem with this transformation is that it breaks functional composition. If before we could compose two functions f and g we can no longer do this for the two correspondant monadic functions f' and g'. We can not compose $g' \circ f'$ because $f' : a \rightarrow m\ b$ and $g' : b \rightarrow m\ c$. The problem here is that g' expects a b but f' provides it with a $m\ b$. Therefore, we need to define a new composition operator called `bind` and denotated as >>=. The type of >>= is $m\ b \rightarrow b \rightarrow m\ c \rightarrow m\ c$. The role of >>= is to take a monadic value, that is, a value of type $m\ b$, a function $f : b \rightarrow m\ c$; to extract a value x of type b from the monadic value and apply the function f to x, in order to return a monadic value of type $m\ c$.

To construct monadic values, we define an operator called *return* : $a \rightarrow m\ a$. For example, in the case of the *List* monad, if we define *return* as `return x = [x]` we can create monadic values such as `return (3) = [3]`; `return (true) = [true]`, and so on.

Definition 1. *A monad is defined by a triplet consisting of:*

- *a* `functor` *m to define new monadic types,*
- *a* `bind` *operator for the composition of monadic functions,*
- *a* `return` *operator for the construction of monadic values.*

To be a true monad, `return` and `bind` must satisfy the following three laws:

- `(return x) >>= f = f (x)`,
- `mv >>= return = mv`,
- `(mv >>= f) >>= g = mv >>=` λx`.f(x) >>= g`.

The following section defines the semantic of the maquettes by using a particular case of the `state monad`.

4 Maquette as a Monad

In order to introduce programs in a maquette, we will use a particular kind of the `state monad`. The `state monad` allows reading and writing data which can be used and modified during the evaluation of a program. The minimal informations we need to put a program into a maquette are its *onset* and its *duration*. In our monad, which we call `Maq` the state is given by a pair (Int, Int) where the first element correspond to the *onset* and the second one to the *duration*.

The idea is to transform any function of type $a \rightarrow b$ into a function of type $(a \times (Int, Int)) \rightarrow (b, (Int, Int))$, it is, a function that, in addition to performing a calculation, can read an *onset* and a *duration* as inputs and return a result along with a new *onset* and a new *duration*, potentially modified. The type thus obtained is not a monadic type, but we can easily write it in the following way: $a \rightarrow (Int, Int) \rightarrow (b, (Int, Int))$. Thus, the functor that constructs our monadic functions, called *Maq*, can be defined as follows:

```
Maq  b = (Int,Int) → (b,(Int,Int)).
```

Maq transforms any type b into a functional type that, given a pair ($onset$: Int, $duration$: Int) returns a type b along with a new pair ($onset$: Int, $duration$: Int).

Once the functor *Maq* is defined, all that remains is to define the `return` and `bind` operators to obtain our monad.

For `return`, which has type $a \rightarrow Maq\ a$, we give the following definition:

```
return x = Maq λ(o,d).(x,(o,d))
```

For example, `(return 5) = Maq` λ`(o,d).(5,(o,d))`. To evaluate the function inside this monadic value, we use the operator `runState` of type $Maq\ a \rightarrow (Int, Int) \rightarrow (a, (Int, Int))$. `runState` takes a monadic value of the *Maq* monad and an initial pair `(onset, duration)`, and returns a pair containing a result of type a and a new pair `(onset, duration)`.

Thus, for example, `runState ((return 5), (0,1))` will return `(5,(0,1))`.

For `bind (>>=)`, which has type $Maq\ a \rightarrow (a \rightarrow Maq\ b) \rightarrow Maq\ b$, we give the following definition:

```
x >>= g =
   Maq λ(o,d).
    let (x',(o',d')) = runState x (o,d) in
      runState (g x') (o',d'')
```

Take for instance `x = (return 5)` and g : $Int \rightarrow Maq\ List\ Int$ defined as:

```
g n = Maq λ(o,d).([n*2],o+2,d+2)
```

And now let's execute the expression `runState (x >>= g, (2,4))` which is equivalent to executing the following program:

```
let(x',(o',d'))=runState(return 5)(2,4) in
      runState (g x') (o',d'')
```

The result of `runState (return 5) (2,4)` returns `x'=5; o'=2 et d'=4`. With these values

```
(g 5) = Maq λ(o,d).([5*2],o+2,d+2)
```

Finally, `runState (g 5) (2,4) = ([10], (4,6))` which is of type $(List\ Int, (Int, Int))$ as we expected.

We also define 2 functions that allow manipulating the state of monadic values in `Maq`:

- *get* : $Maq\ (Int, Int)$ which return the state value (o,d) being passed around,
- *put* : $(Int, Int) \rightarrow Maq\ ()$ to replace the current state value with a new (o,d) state.

The `Maq` monad is a special case of the state monad where the state is fixed to a pair (onset, duration). For this reason, we do not prove monad laws for `Maq`; a proof that state monad satisfies the three monad laws can be found in [10].

5 Graphical Representation of the *Maquette* Monad

In OM, a temporal program is a standard program extended with a default input called `state`. Temporal programs are the graphic representation of `Maq` monadic functions. The program in Figure 1 (b) has been extended in Figure 3 (a), where we can see the "state" box with three outputs. The first one return the state of the program, which is the graphic equivalent of the `get` operator. The second and third outputs respectively retrieve the onset and the duration of this state.

A maquette proposes two main relationships between the temporal program it contains: a causality relationship and a temporal relationship. The causality between two temporal programs is defined with respect to the order of evaluation. Similarly, the temporal line of the maquette allows the comparison of temporal program based on their onset. The following examples show possible combinations of these two relationships.

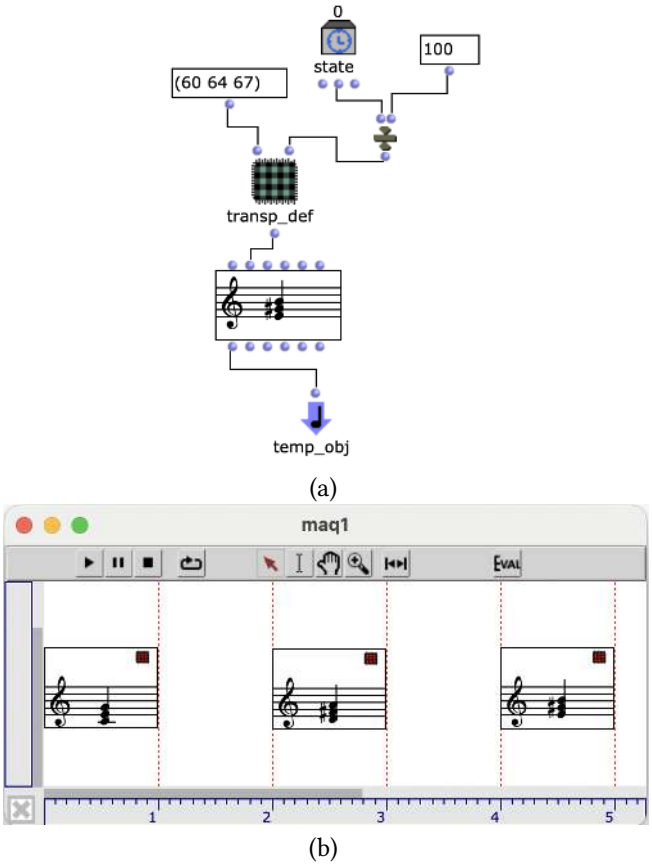


Figure 3. The onset changes the computation.

Figure 3 (b), shows the temporal program of Figure 3 (a) situated at three different onsets: 0, 2000, and 4000 milliseconds. Each time the program is evaluated (by using `runState`), a state is given as an argument containing a pair (onset,duration). Since the program has access to this state each time and because the program uses the onset to calculate the number of semitones by which the chord will be transposed, the three results will be different (i.e., transposed by 0, 2, and 4 semitones). In this example, we would say that the calculation result depends on the position of the temporal program. To quote [12], one could say that "[temporal programs] bear time and change information".

Figure 4 (c) shows a maquette where two temporal programs are composed. The temporal program located at the top is shown in Figure 4 (a) (we will call it, program 1). This produces a chord but also sends as a result the list of MIDI notes of the chord as well as the state with which the temporal program is called.

The lower temporal program in Figure 4 (b) which we will call Program 2, receives the MIDI notes and creates a chord where notes are transposed by 7 semitones. Additionally, the state of the Program 1 is modified; its onset is set immediately after the onset plus the duration given by Program 2. It

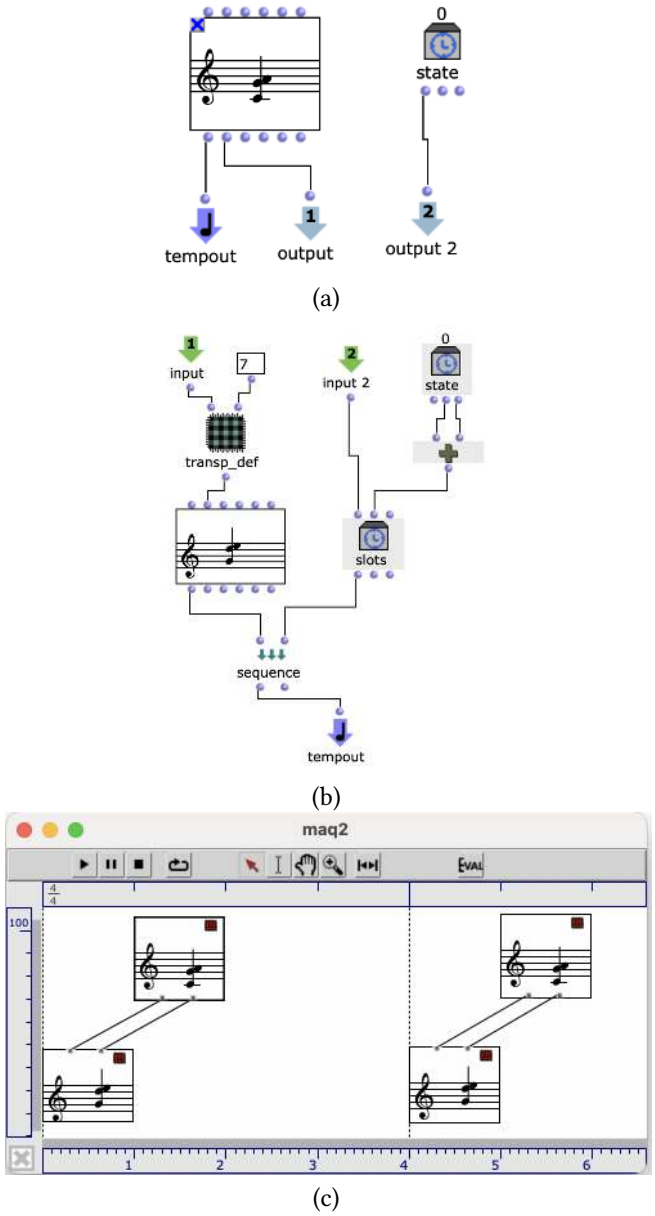


Figure 4. The computation changes the onset.

doesn't matter where the Program 2 is placed, after the evaluation, Program 1 will change its onset to be immediately after it. In this case, we will say that the calculation conditions the temporal organization. Or, quoting [12], again,"states bear information and change time".

Another feature found in this last example consists of a kind of dichotomy between the two orders defined by time and composition. From the composition perspective, Program 1 takes place before Program 2. However, from the temporal perspective, it is the opposite. This implies that the construction of one chord depends on another that is situated in the future. In other words, the second program,

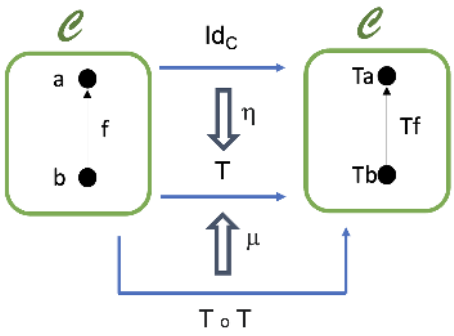


Figure 5. Monad diagram.

despite being evaluated later, actually runs earlier in time, creating a relationship where the outcome of one chord is conditioned by an event that has not yet occurred in the standard temporal flow. This inversion of orders opens up new possibilities for modeling how events can interrelate in a context where time and composition do not follow the same conventions.

6 Conclusions

Through the formalization of maquettes in OM with the help of monads, this article provides an example of the multiple possibilities of using functional programming paradigms as tools for musical composition. In particular, we aim to offer new perspectives and methodologies for understanding and manipulating the temporal aspects of musical pieces. The primary characteristic of a maquette comes from the possibility of combining computations and representations within the same object. Regarding computation, a maquette as the fonctional composition of temporal functions. However, unlike classical functional composition, in a maquette, the onset and the duration of these temporal programs, can play a determining role in the computation.

This formalization reflects our commitment to popularize access to advanced musical tools, thereby promoting a more inclusive musical landscape. Category theory is a discipline extensively studied by expert musicologists. Mazola's pioneering book,[8], has given rise to many other publications and various applications focusing on theory and analysis. Few studies are based on the use of category theory for compositional purposes, although we can cite the works of [11]. With this article, we wanted to show that the concept of monad, specific to category theory and known for its difficulty in understanding, can be explained through a generative application.

For this, we took a detour and first looked at how a monad is used in functional programming. In category theory, a monad is defined as a triplet (T, μ, η) where T is an endofunctor and μ and η are two natural transformations called `join` and `return`, see Figue 5.

Natural transformations in category theory find their equivalent in generic functions in functional programming; thus, μ and η play the roles of bind and return, respectively.

Despite this first abstraction reduction, the monad in functional programming remains an abstract concept difficult to grasp without resorting to a real application. We have attempted to explain in Section 3 the general utility of a monad in the context of functional programming, but we believe it is important to provide a concrete application, such as in the case of maquettes, which might further motivate composers to engage in this type of study.

We conclude this article by arguing that the monad, is above all, an abstraction of functional composition. That is, the monad makes the compositional operator ∘ variable. Understood in this way, the use of monads allows for the definition of spaces where functional composition can be arbitrarily defined (while respecting the laws, of course). This opens a door to the creation of novel relationships between musical components.

OM is open source; it can be downloaded at:
https://openmusic-project.github.io

7 Acknowledgments

This research is supported by European Research Council ERC-ADG-883313 REACH, and Agence Nationale de la Recherche ANR-19-CE33-0010 MERCI.

References

[1] C. Agon. "OpenMusic : Un langage visuel pour la composition musicale assistée par ordinateur". PhD Paris 6 University, 1998.
[2] C. Agon, G. Assayag, J. Bresson. *The OM Composer's Book 1.* Éditions Delatour France / Ircam-Centre Pompidou, 2006.
[3] M. Barr, J. Beck. *Acyclic models and triples.* Proc. Conf. Categorical Algebra. Springer-Verlag, Berlin (1966), 336–343.
[4] J. Bresson, C. Agon. *Visual Programming and Music Score Generation with* OpenMusic. IEEE Symposium on Visual Languages and Human-Centric Computing. Pittsburgh, 2011.
[5] P. Hudak, D. Quick. "The Haskell School of Music : From signals to Symphonies". Cambridge University Press, 2018.
[6] D. Janin. *A Timed IO monad. Practical Aspects of Declarative Languages (PADL)*, New Orleans, United States, 2020.
[7] S. MacLane. "Categories for the Working Mathematician". Springer-Verlag, New York, Graduate Texts in Mathematics, Vol. 5. 1971.
[8] G. Mazzola. "The topos of music". Birkhäuser Verlag ed. 2002.
[9] E. Moggi. *Notions of computation and monads.* Information and Computation, 1991.
[10] B. O'Sullivan, et al. "Real World Haskell". O'Reilly Media, 2008.
[11] A. Popoff , C. Agon , M. Andreatta , A. Ehresmann. *From K-Nets to PK-Nets: A Categorical Approach.* Perspectives of New Music. Volume 54, Number 2, Summer 2016.
[12] V. R. Pratt. *The Duality of time and information.* Stanford University, 1996.
[13] P. Wadler. *Computational lambda calculus and monads.* In IEEE Symposium on Logic in Computer Science 1989.

Received 2024-06-02; accepted 2024-07-02

Demo: A Geometric Approach to Generate Musical Rhythmic Patterns in Haskell

Xavier Góngora
Universidad Nacional Autónoma de México
Mexico City, Mexico
xavier.gongora@comunidad.unam.mx

Abstract

We present work-in-progress on RTG, a domain specific language embedded in Haskell designed to explore the affordances of geometry as a means to generate and manipulate rhythmic patterns in live coded music. Examples of how simple geometry is capable of producing interesting rhythms are shown to support our use of binary lists as a pattern representation. We introduce Erlangen's Program notion of geometry as encoded in *groups*, using such structure as the focus of a combinator interface based on an archetypal `RhythmicPattern` type implemented using a type class. Examples of the interface usage are provided. Future work targets the definition of `Group` instances for the *rhythmic pattern types* such that the *group* laws are fulfilled and its operations lift to the interface in a musically coherent and engaging way.

CCS Concepts: • **Applied computing** → **Sound and music computing**; • **Software and its engineering** → *Domain specific languages.*

Keywords: geometry, rhythm, music, pattern, group theory, functional programming, Erlangen program, live coding

ACM Reference Format:
Xavier Góngora. 2024. Demo: A Geometric Approach to Generate Musical Rhythmic Patterns in Haskell. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3677996.3678295

1 Introduction

In previous work [5] we showcased design criteria for RTG:[1] a live coding library for the generation and manipulation of rhythm, influenced by the Tidal Cycles language.[2] Its central idea is the definition of *group structures* for rhythmic pattern types (*i.e.* rhythm families defined by computational and geometric considerations, represented using algebraic data types), so that rhythms can *transform* one another.

Our canonical rhythmic pattern type is that of Euclidean rhythms, shown briefly in section 2. Rhythmic patterns are considered language primitives as values of such types. The goal is to develop a concise and terse domain specific language such that the combination and transformation of rhythms is done within musically significant geometric constraints that afford effective exploration strategies during a live coding performance. To the best of our knowledge this is a novel approach to rhythm manipulation which is yet to prove itself before an audience.

Studies of rhythm's geometric properties are usually intended for music theoretic analysis or music information retrieval. In contrast, our approach is led by the intuition that rhythm itself is a geometric aspect of time and the search for the creative potential of such an aspect. This made us speculate about an inner *geometry* of rhythm, hidden in its *group of transformations*. We discuss *group* structure and the Erlangen Program as the origin of our concept of geometry in section 5.

The current prototype is build using lists and abstracts musical rhythm to its bare bones. We motivate this simplification in section 2 by showing that interesting structure persists in this setting. Next, in sections 3 and 4, we use the Haskell's type system to implement a shared interface for distinct *group operators* that includes some basic combinators.[3] Afterwards, in section 6, we provide examples of the interface use.

2 What about Geometry in Rhythm?

Rhythm is a multidimensional phenomenon, too complex to account for in any single framework. In all its generality, it could be described as a natural law underlying all dimensions of being. In the case of music, aspects like volume, timbre and cultural context underlie our perception (or induction) of rhythm [2]. What is certain, and a useful approach towards

[1] The library's source code repository: https://github.com/ninioArtillero/RTG

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

FARM '24, September 2, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3678295

[2] Tidal Cycles oficial website: https://tidalcycles.org/

[3] We use the terms *operator* and *operation* in an interchangeable fashion, as in many contexts they are equivalent. The former is more common in programming, while the later in arithmetic.

its computational abstraction, is that rhythm is fundamentally bound to the arrangement of sound events in time. This arrangement alone produces salient perceptual structures, most notably meter and rhythmic grouping [2, 13], which seem to correlate with geometric properties.

Toussaint's [2020] working hypothesis is that geometric properties of rhythms translate to musical qualities that consistently make them being perceived as *good*, evidenced by their repeated appearance among diverse cultural contexts and musical styles. His paradigmatic example is the *Son Clave* found in music genres related to the african diaspora (its name comes from cuban music). It is a 5 onset pattern in a 16 isochronous (*i.e.* of the same time duration) pulse measure. We can represent this pattern as a list, where "1" stands for a sound *onset* and "0" for a *rest*.[4]

```
son = [1,0,0,1,0,0,1,0,0,0,1,0,1,0,0,0]
```

An important family of pattern examples are Euclidean rhythms. These are constructed using Björklund's algorithm, which is equivalent to Euclid's algorithm for computing the maximum common divisor of a pair of integers. Geometrically, the `(k,n)` Euclidean rhythm can be described as the configuration that maximizes the evenness of `k` ones among `n-k` zeros in a binary pattern [12]. For example, the Euclidean rhythm denoted by `(7,12)` is a common West African bell pattern [12].

```
e (7,12) = [1,0,1,1,0,1,0,1,1,0,1,0]
```

Another interesting example discussed in Toussaint [2020] is that of the rhythmic pattern used by Steve Reich in his piece "Clapping Music". Out of the 495 possible 8 onset in 12 pulses measure patterns, this pattern can be selected by means of regularity and diversity constraints, like allowing only one-pulse rests and excluding cyclic permutations. A key constraint, which is relevant to the structure of "Clapping Music", is that the pattern doesn't align with itself while rotating it step-wise until it reaches the starting position.[5] This pattern happens to be a *rotation* of another one obtained as the result of superposing the three clapping patterns performed in the Beer Dance of the Lala people [13].

```
clap1     = [1,0,0,0,1,0,0,0,1,0,0,0]
clap2     = [1,0,1,0,1,0,0,1,0,1,0,0]
clap3     = [1,0,1,0,1,1,0,1,0,1,0,1]

beerDance = [1,0,1,0,1,1,0,1,1,1,0,1]

reichP    = [1,1,1,0,1,1,0,1,0,1,1,0]
```

[4]The terms *onset* and *offset* are standard in the theory of music, and refer to the start and end (respectively) of a sound event associated with a specific source. As part of the simplicity of our approach, we do not consider offsets.
[5]Steve Reich's "Clapping Music" starts with two performers clapping this pattern in unison. One of them will carry on this way the whole piece. After every twelfth repetition, the other performer shifts its pattern by one step (rotation). The piece ends after both performers play in unison again.

Incidentally, "Clapping Music" is a good fit for naming the domain of rhythm the present work focuses on. Clapping is commonly used when communicating a rhythmic pattern. This might be related to its short and fast decaying envelope resembling a time point in perception. These examples show that engaging clapping music can be generated out of a simple geometric procedure such as the Euclidean algorithm or by the operation of superposition that underlies polyrhythms.[6]

3 Representing Rhythm

Our model of rhythm is a sequence of isochronous sound onsets and rests called a `RhythmicPattern`. We implement this in Haskell by first defining a `Binary` data type to represent onsets and rests.

```
data Binary = Onset | Rest
  deriving (Eq, Ord)

instance Show Binary where
  show Rest  = show 0
  show Onset = show 1

instance Semigroup Binary where
  Rest  <> Onset = Onset
  Onset <> Rest  = Onset
  _     <> _     = Rest

instance Monoid Binary where
  mempty = Rest

instance Group Binary where
  invert = id
```

Listing 1. A binary data type representing sound onsets and rests

We will discuss groups with some detail in section 5, but for the time being we note that the `Group` instance provided for the `Binary` type is isomorphic to the integers modulo 2 (which is the only group structure possible for a two value type).[7] We'll use this type in the definition of our archetypal rhythmic pattern type to have a standard way of combining onsets and rests. Now we define `Rhythm` as a `newtype` wrapper around `Pattern`, which is a type synonym for lists.

```
type Pattern a = [a]

newtype Rhythm a =
```

[6]It might be argued that these properties are arithmetic rather than geometric. Nevertheless, both fields of mathematics are intricately related to each other [11], so for us this is a matter of appreciation and not an assertion on mathematical classification.
[7]The `Group` type class is exported by the `Data.Group` module from the `groups` package. https://hackage.haskell.org/package/groups

```
  Rhythm { getRhythm :: Pattern a }

instance Functor Rhythm where
  fmap f (Rhythm xs) = Rhythm (fmap f xs)

instance Applicative Rhythm where
  pure xs = Rhythm $ pure xs
  Rhythm fs <*> Rhythm xs =
    Rhythm (zipWith ($) fs xs)

instance Semigroup a =>
  Semigroup (Rhythm a) where
  Rhythm pttrn1 <> Rhythm pttrn2 =
    Rhythm $ pttrn1 `EuclideanZip` pttrn2

instance (Semigroup a, Monoid a)
  => Monoid (Rhythm a) where
  mempty = Rhythm $ repeat mempty

instance (Semigroup a, Monoid a, Group a)
  => Group (Rhythm a) where
  invert = fmap invert
```

Listing 2. Rhythm defined as a newtype wrapper around the pattern type

The previous declarations are pretty straight forward, but the `Semigroup` instance has some crucial consequences for us, so we look into it. The constraint there is caused by the `EuclideanZip` function. It works by zipping the `k` elements of the smaller list as evenly as possible among the `n` elements of the bigger list, combining then with the `Semigroup` operator, according to the Euclidean pattern `(k,n)` (see section 2). In the case when both lists have the same length, it simply does **zipWith** (<>) pttrn1 pttrn2.

This is not the only operation possible. Indeed, from Haskell's type system point of view, we could implement it with any operation imaginable as long as it fulfills the corresponding function types (signatures). Nevertheless, one design strategy is looking for simplicity and symmetry [5]. That such an operation is a good fit is to be decided by its fulfillment of group laws and its (musical) performance.

4 The Interface

Now we are in a position to display the full interface to manipulate rhythmic patterns.

```
type RhythmicPattern = Rhythm Binary

class Semigroup a => Rhythmic a where

  -- Minimal complete definition.
  toRhythm :: a -> RhythmicPattern

  -- Group operator lifting.
  (&) :: Rhythmic b =>
    a -> b -> RhythmicPattern
  x & y = toRhythm x <> toRhythm y

  (!&) :: a -> a -> RhythmicPattern
  x !& y = toRhythm (x <> y)

  -- Basic combinators.
  inv  :: a -> RhythmicPattern
  co   :: a -> RhythmicPattern
  rev  :: a -> RhythmicPattern
  (|>)  :: Rhythmic b =>
    a -> b -> RhythmicPattern
  (<+>) :: Rhythmic b =>
    a -> b -> RhythmicPattern
```

Listing 3. The main rhythmic pattern interface

The default implementation of the basic combinators is not shown. They are group inversion, complement, reverse, sequencing and superposition, respectively. We have also provided specifications for this functions as formal properties verifiable by `QuickCheck`, but are omitted here. Formal verification, by proof or property based testing, is important to assert a group structure.

The group lifting functions combine patterns using *group* operations. As shown by their default implementations, values of any two rhythmic types can be combined with the `RhythmicPattern` group operation after lifting the values using the `&` operator. The `!&` operator combines the elements of the same type using their group operation before lifting the result. They don´t necessarily match, as the corresponding group operations can be different. When group operators are curried, each element of a group can be seen as a *transformation* for elements of the same group. Also, transformations of objects and spaces form groups, and this simple fact take us near the realm of geometry.

5 Groups and Geometry

Our focus on the search, definition and use of group operations for rhythmic pattern is founded on speculative possibilities informed by the Erlangen Program. In this section we introduce it along with the laws defining the (algebraic) group structure.

The Erlangen Program is a research program proposed by Felix Klein at the end of the nineteen century, which ultimately asserts the study of geometric structure and properties is equivalent to that of the invariants under a *group of transformations* [14].

As an example, a polygon's enclosing area, perimeter and number or vertices are invariants of the group of isometric

movements of the plane (any combination of translations, reflections and rotation). On the other hand, a regular polygon on a specific configuration, for instance a triangle centered in the coordinate origin on the plane, is invariant only under a *subgroup* of the group of isometries of the whole plane.

In this context, a *group* is defined as a set with an associative binary operation (here represented polymorphically by `<>`), having a unique identity element and a unique inverse for each element. In terms of Haskell's standard type classes, a group is a type with a `Subgroup` and `Monoid` instances, and also a `Group` instance implementing the invert function, fulfilling the following laws (*i.e.* all expressions evaluate to **True** for any elements x, y and z of the group):

```
(x <> y) <> z == x <> (y <> z)
mempty <> x == x
x <> mempty == x
invert x <> x == mempty
x <> invert x == mempty
```

Listing 4. Groups laws as Haskell boolean expressions

As simple as they are, the group laws encode the mathematical notion of symmetry [14]. They tell us that property preserving transformations compose and they can always be undone by a corresponding (backwards) transformation. At the time of Erlangen's Program publication [7], geometry was blooming into a diverse subject with the emergence of non-Euclidean geometries, differential geometry and projective geometry. Perhaps this contrasted sharply with the previous monolithic classical geometry (that of Euclid's) and a unifying framework was called forth. The Erlangen Program today is perhaps an artifact in the history of mathematics, but it underlies many ground-breaking developments in the theoretical physics of the twentieth centry, such as Kaluza-Klein theory, Yang-Mills theory, string theory and non-commutative geometry [6].

6 Usage and Examples

Euclidean rhythms (see section 2), onset binary patterns (the archetypal RhythmicPattern type, as defined in listing 3) and *time patterns* are our starting cases of rhythmic pattern types. Time patterns are a collection of prescribed patterns, based on equal-tempered scales and popular rhythmic *timelines* [13].[8]

```
diatonic :: Pattern Time
diatonic = [0/12, 2/12, 4/12, 5/12,
            7/12, 9/12, 11/12]

rumba :: Pattern Time
rumba = [0/16, 3/16, 7/16, 10/16, 12/16]
```

[8]Timelines are short *ostinatos* that contextualize the other rhythmic and melodic elements in a piece within a given style of music.

The elements of these patterns are rationals numbers normalized to range from 0 to 1 non-inclusive, as 1 matches the beginning of the next *cycle*. They are eventually converted to the binary representation for playback (as all rhythmic pattern types).

Rhythmic patterns are played in a cyclic fashion, repeated indefinitely during a performance, and their playback *tempo* is derived from a variable that defines how many times it is played per second.

The group operations `<>` of the three types are implemented as follows: As time patterns are just type synonyms, we use the standard list append as operation. In the binary representation this operation lifts to the superposition of the patterns. Euclidean rhythms use modular arithmetic on their tuple components. With its current operation the Euclidean type has the following Cartesian product as denotation: $\mathbb{Z}_n \times \{n\} \times \mathbb{Z}_n$ for all $n \in \mathbb{N}$.

Elements of both types can be lifted to a RhythmicPattern using the toRhythm function, and are combined using the euclideanZip function described at the end of section 3. Space constraints prevent us from showing the implementations or detailed examples of the individual operations.[9]

What we have seen so far allows us to express the combination of scales and rhythms.

```
playR $ diatonic & rumba & diminished
-- [1,0,0,1,1,0,1,1,1,0,0,1,1,0,0,0]
```

The playR function implements rhythm playback. It is currently a quick and dirty solution using the SuperDirt sound engine and a clap sample.[10] Euclidean rhythms can be thrown into the mix.

```
playR $ e(3,8) & e(13,33) & son
-- [1,0,0,1,0,1,1,0,1,0,1,0,0,1,0,
--   1,0,0,1,0,0,0,0,1,0,1,0,0,1]

playR $ e(3,8) !& e(13,33) & son
-- too long to display
```

When two patterns are combined with the & operator, the RhythmicPattern operation is used, while !& uses the specific rhythmic pattern type operation (so both patterns must be of the same type). In other words, we have means to choose the operation to use: each type's own group operation or the RhythmicPattern one. Is also of note that these operators substitute the direct use of `<>`. This is an elegant way of experimenting with different combinators, thinking of rhythmic patterns both as language primitives and group elements leveraging ad-hoc polymorphism.

[9]The current implementation of the operations described can be found at this point in the source code history: https://github.com/ninioArtillero/RTG/tree/a3f90bb70105628c6eca17cf35c09b9ad7f28951

[10]SuperDirt source code repository: https://github.com/musikinformatik/SuperDirt

7 Discussion

Future work is aimed at testing and comparing alternative representations of rhythmic patterns as the group operations proposed so far have failed to fulfill group laws. We have focused on a variety of zip-like operations differing in the way list elements are matched when list lengths are not equal. Zip-like operations are natural candidates considering rhythm is "temporal media". As pointed out by Apfelmus [2019], the other natural applicative structure for lists (associated with the standard monad instance) is analogous to a product which might not be suitable for temporal media. This could be signaling that either a relaxation of the group axioms is called forth or that a different representation is needed. Both cases would reveal an actual fact about the structure of rhythmic patterns.

Also, rhythmic structure has the peculiarity of being a second order pattern in the sense that the insertion of a single onset changes the whole structure [10].

From this considerations, the implementation of the rhythmic pattern representation as a binary list might need to be enriched to carry more structural information. Maybe a state monad to keep track of context information such as rhythmic grouping (or clustering, to avoid confusion) and meter, to bind it through exotic pattern combinators. Another approach that has proven musically robust is Tidal Cycles representation of patterns [8, 9] based on the Functional Reactive Programming idea of a "behavior" as a function of continuous time [4]. This approach has had a long life cycle and has been subject to many refinements. Leveraging some of this work might be of great value to our task.

8 Conclusions

We exposed the rhythmic pattern representation that sits at the core of RTG's prototype and showed some examples of its intended use. The current aim is to provide a new way of combining rhythmic patterns for live performance using group structures.

During development the abstraction power of standard type classes, like `Functor` and `Applicative`, play the role of a gravitational field guiding design decisions and implementation strategies. Along our way, we keep finding clues and intuitions supporting our search for the geometric structures of rhythmic patterns. Existing limitations and inconsistencies might be clues of where the inner geometry of time and rhythm lies. Perhaps we need "some deeper structure, going far beyond the parts, points, local neighborhoods... and fragmented classical geometrical views in general" [3]. For certain, geometry is a field of mathematics that stills holds promise for the domain of music creation.

Acknowledgments

We thank Roberto Velasco, Juan S. Lach, and the anonymous reviewers for their valuable feedback. This ongoing research is supported by the *Consejo Nacional de Humanidades, Ciencias y Tecnologías* (Conahcyt) as part of my PhD studies under the supervision of Hugo Solís at the *Universidad Nacional Autónoma de México* (UNAM).

References

[1] Heinrich Apfelmus. 2019. Demo: Functors and Music. In *Proceedings of the 7th ACM SIGPLAN International Workshop on Functional Art, Music, Modeling, and Design*. ACM, Berlin Germany, 52–55. https://doi.org/10.1145/3331543.3342582

[2] Eric F. Clarke. 1999. Rhythm and Timing in Music. In *The Psychology of Music (Second Edition)*, Diana Deutsch (Ed.). Academic Press, San Diego, 473–500. https://doi.org/10.1016/B978-012213564-4/50014-7

[3] Micho Durdevich. 2017. Music of Quantum Circles. In *The Musical-Mathematical Mind: Patterns and Transformations*, Gabriel Pareyon, Silvia Pina-Romero, Octavio A. Agustín-Aquino, and Emilio Lluis-Puebla (Eds.). Springer International Publishing, Cham, 99–110. https://doi.org/10.1007/978-3-319-47337-6

[4] Conal Elliott. 2009. Push-Pull Functional Reactive Programming. In *Proceedings of the 2nd ACM SIGPLAN Symposium on Haskell (Haskell '09)*. Association for Computing Machinery, New York, NY, USA, 25–36. https://doi.org/10.1145/1596638.1596643

[5] Xavier Góngora. 2023. Rhythm, Time and Geometry. In *Algorithmic Pattern Salon*. Then Try This. https://doi.org/10.21428/108765d1.e65cd604

[6] Lizhen Ji and Athanase Papadopoulos (Eds.). 2015. *Sophus Lie and Felix Klein: The Erlangen Program and Its Impact in Mathematics and Physics*. Number 23 in IRMA Lectures in Mathematics and Theoretical Physics. European Mathematical Society, Zürich.

[7] Felix Klein. 1983. A Comparative Review of Recent Researches in Geometry. *Bulletin of the New York Mathematical Society* 2, 10 (July 1983), 215–249.

[8] Alex McLean. 2014. Making Programming Languages to Dance to: Live Coding with Tidal. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Functional Art, Music, Modeling & Design (FARM '14)*. Association for Computing Machinery, New York, NY, USA, 63–70. https://doi.org/10.1145/2633638.2633647

[9] Alex McLean. 2020. Algorithmic Pattern. In *Proceedings of the International Conference on New Interfaces for Musical Expression*. Birmingham City University, Birmingham, UK, 265–270. https://doi.org/10.5281/zenodo.4813352

[10] Orestis Melkonian, Iris Yuping Ren, Wouter Swierstra, and Anja Volk. 2019. What Constitutes a Musical Pattern?. In *Proceedings of the 7th ACM SIGPLAN International Workshop on Functional Art, Music, Modeling, and Design*. ACM, Berlin Germany, 95–105. https://doi.org/10.1145/3331543.3342587

[11] John Stillwell. 1998. *Numbers and Geometry*. Springer, New York, NY. https://doi.org/10.1007/978-1-4612-0687-3

[12] Godfried Toussaint. 2005. The Euclidean Algorithm Generates Traditional Musical Rhythms. In *Renaissance Banff: Mathematics, Music, Art, Culture : Conference Proceedings 2005*, Reza Sarhangi and Robert V. Moody (Eds.). Bridges Conference, Southwestern College, Winfield, Kansas, 47–56.

[13] Godfried T Toussaint. 2020. *The Geometry of Musical Rhythm: What Makes a "Good" Rhythm Good?* (second edition ed.). CRC Press, Boca Raton London New York.

[14] I. M. Yaglom. 1988. *Felix Klein and Sophus Lie: Evolution of the Idea of Symmetry in the Nineteenth Century*. Birkhäuser, Boston.

Received 02-JUN-2024; accepted 2024-07-02

Diffusion-Based Sound Synthesis in Music Production

Pierre-Louis Wolfgang Léon Suckrow
Berlin University of the Arts
Berlin, Germany
Technical University of Berlin
Berlin, Germany
p.suckrow@udk-berlin.de

Christoph Johannes Weber
University of Television and Film Munich
Munich, Germany
LMU Munich
Munich, Germany
c.weber@hff-muc.de

Sylvia Rothe
University of Television and Film Munich
Munich, Germany
s.rothe@hff-muc.de

Abstract

In this paper, we explore the usability of generative artificial intelligence in music production through the development of a digital instrument that incorporates diffusion-based sound synthesis in its sound generation. Current text-to-audio models offer a novel method of defining sounds, which we aim to render utilizable in a music-production environment. Selected pretrained latent diffusion models, enable the synthesis of playable sounds through textual descriptions, which we incorporated into a digital instrument that integrates with standard music production tools. The resultant user interface not only allows generating but also modifying the sounds by editing model and instrument-specific parameters. We evaluated the applicability of current diffusion models with their parameters as well as the fitness of possible prompts for music production scenarios. Adapting published diffusion model pipelines for integration into the instrument, we facilitate experimentation and exploration of this innovative sound synthesis method. Our findings show that despite facing some limitations in the models' responsiveness to specific music production contexts and the instrument's functionality, the tool allows the development of novel and intriguing soundscapes. The instrument and code is published under https://github.com/suckrowPierre/WaveGenSynth.

CCS Concepts: • **Applied computing** → **Sound and music computing**; • **Computing methodologies** → *Artificial intelligence*; • **Human-centered computing** → *Text input*; *User studies.*

Keywords: sound generation, text-to-sound, user interface, user study

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
FARM '24, September 2, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3678289

ACM Reference Format:
Pierre-Louis Wolfgang Léon Suckrow, Christoph Johannes Weber, and Sylvia Rothe. 2024. Diffusion-Based Sound Synthesis in Music Production. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3677996.3678289

1 Introduction

Recent advancements in artificial intelligence and generative models have significantly enhanced and simplified the process of creating new sounds. Traditionally, artists and sound engineers relied on large and comprehensive sound libraries to meet their creative and technical needs. While these libraries provide quick access to a broad spectrum of sounds, various problems can be identified.

Recorded sounds, bound by distinct licenses and copyrights, often limit sound usability and adaptation to safeguard creators, yet these restrictions can impede the creative process. AI-generated content emerges as a solution to these legal hurdles, though the status of AI authorship and copyright is currently under debate. Another point is that the complexity and utility of digital sound libraries increase with their size and chosen compression, facilitating the discovery of specific sounds but complicating their search as the collection expands. This challenge becomes more pronounced beyond a certain threshold, where the addition of new sounds entails the inclusion of variations on existing themes, necessitating that artists compare multiple similar sounds to find the one that precisely meets their requirements. AI's capability to generate any sound from text descriptions enables artists to efficiently create unique sounds, circumventing traditional library constraints.

Musicians and composers have always explored new methods for creating and performing music, from traditional instruments, analog synthesizers to digital computation [27]. The evolution of computing technology, driven partly by the quest to innovate and preserve sounds, underscores a historical symbiosis between binary computing and musical advancement [27]. This relationship, rooted in the ancient connection between mathematics and music theory [16],

highlights the continuous impact of technological progress on musical creativity. Generative models represent the latest advancement in a series of innovations that have transformed music production and sound design. Thus we formulate the following research question:

RQ: Are diffusion models suitable for use as a sound source in a music production process?

This paper investigates the implementation of a synthesizer leveraging novel *latent diffusion* [26] technology. We present a digital instrument called *WaveGenSynth*[1], which allows us to incorporate pretrained music generation models and generate sounds with natural language. We also analyzed the quality of the generated audio samples and their correspondence with the input prompt.

Both the resulting digital instrument and the potential advantages and limitations of this implementation are examined. In addition, the potential of the *diffusion* models used in the context of musical applications is assessed. Our realization aims to empower users to generate, modify, and play sounds according to their individual preferences. The primary objective of this work is to conduct a comprehensive analysis of various implementation strategies as well as the results of the digital instrument and the *diffusion* models used. This involves identifying their advantages and limitations, and evaluating their effectiveness as a novel tool for musical expression. Additionally, our paper seeks to design the instrument in a way that facilitates its smooth integration into modern music production workflows, thereby offering musicians novel avenues for sound exploration and synthesis techniques.

2 Related Work

Synthesizing data through generative AI is rendered possible under the premise that examined data emanates from a distinct distribution, which Machine-Learning models strive to approximate, thereby enabling the extraction of novel data points through distribution sampling [6, 13, 28]. The models employed in this work synthesize sound through a *diffusion* process. As stated by Haohe Liu[2] utilizing *diffusion* [4, 7, 18, 30] as a means to synthesize sound offers numerous benefits, including the potential to democratize the sector by reducing dependency on expensive recorded sounds, giving users enhanced control over timbres and sound properties, and from a creative standpoint, enable the shaping of unique sounds that may inspire artists.

In the *diffusion* process, drawing on statistical thermodynamics [11] and sequential Monte Carlo methods [17], a neural network aims to iteratively denoise data in the backward process by learning from distortions typically introduced via Gaussian noise [29] in the forward process. This generative model demonstrates enhanced efficiency over similar models [18, 30]. The denoising of a data point makes it appear as though new data is generated from random noise. Building upon this concept, *latent diffusion* [26] allows the generation of images from textual descriptions by shifting the *diffusion* process from operating on raw pixel values to acting within a latent space created by a *variational autoencoder* (*VAE*) [12]. This innovation not only minimizes computational requirements for broader hardware compatibility but also supports diverse media conditioning. It optimizes semantic information retention, enhancing generated image quality [26].

Inspired by *Contrastive Language-Image Pretraining* (*CLIP*) [22], Large-Scale *Contrastive Language-Audio Pretraining* (*CLAP*) [34] develops a unified latent representation for audio and text through the training on audio-text pairs, effectively mapping their shared information into corresponding embeddings. This latent space enables tasks like text extraction from audio and audio generation from text descriptions. [34]

Music, as an art form and cultural expression, involves the structured organization of sound over time, distinguished from noise by its structured harmonies pleasing to the human ear [19, 31]. Sound, at its physical level, is a wave generated by vibration and includes a fundamental tone accompanied by resonating overtones, giving rise to various harmonics that define its timbre [19, 31]. The analysis of sound focuses on the relationships among tones, classifying the lowest dominant frequency as the fundamental tone and higher frequencies as overtones, which are considered harmonic when their frequencies are integer multiples of the fundamental tone, and inharmonic otherwise, leading to the unique sonic qualities of different instruments [19, 27, 33]. The sound's texture, timbre and characteristics, are described through the relationship of its underlying tones and their amplitude and frequency and can be visualized and distinguished over time in a spectrogram [24]. This foundational understanding supports the use of spectrograms in representing audio as images, facilitating the extraction of information or generation of audio signals through image-based computational intelligence methods.

Applying this methodology of using image based architecture on audio is the generative model *AudioLDM* [14] allowing for the synthesis and manipulation of audio through a natural language input. Unlike previous efforts such as *Diffsound* [35], *AudioLDM* does not directly use text-audio pairs in training, which could limit the scope of available data, and instead relies only on audio data by incorporating *CLAP* into its architecture. Following [26], a *VAE* [12] is employed to convert mel-spectrogram audio into a latent space for

[1] https://github.com/suckrowPierre/WaveGenSynth
[2] https://www.youtube.com/watch?v=6qtL9_T8m3c

diffusion. The embeddings from *CLAP* are used to condition the model both during training and at inference, enabling it to extract information from the input prompts. The process culminates in the generation of a spectrogram that is subsequently transformed into an audio signal with a vocoder. [14]

Expanding on *AudioLDM*, the *AudioLDM2* [15] model introduces a novel architecture that promises enhanced outcomes in audio generation. It aims to overcome the constraints of prior audio synthesis models, which were often biased to specific tasks or domains, by integrating a novel *language of audio* (*LOA*) representation for conditioning. This representation aims to generalize across various domains and modalities such as text, audio, images and videos. The *diffusion* process occurs in the same latent space as proposed in [14]. The used neural network, has been further refined, amongst other things with attention mechanisms [32]. The model employs the self-supervised *AudioMAE* [10] for converting audio into *LOA* and *GPT-2* [23] for other modalities. The former playing a role during training and fine-tuning, and the latter being employed during inference and the majority of fine-tuning. [15]

Possible methodologies and approaches for creating instruments that facilitate sound synthesis via artificial intelligence have been explored by the "Neural Audio Plugin"[3] competition, organized by *The Audio Programmer Ltd*[4] in the spring of 2023. Among the participants, the digital instrument *VroomAI*[5] stood out, offering the capability to generate playable sounds from textual descriptions using generative AI models. *VroomAI* offers two usable models: *AudioLDM* [14] and *AudioLM* [1], the former of which is also employed in our tool. The subsequent model, *AudioLDM2* [15], was not available at the time of *VroomAI*'s latest maintaining date, thus not being available as an option, unlike in our instrument. The architecture used in *VroomAI* mirrors that of our approach, with both initiatives utilizing the *Juce*[6] *C++* framework for digital instrument development. Unlike this project, *VroomAI* depends on the *AudioLDM* published *Gradio*[7] inference server[8] for sound generation. A process complicated by hardware limitations, particularly on devices lacking *NVIDIA CUDA*[9] support, leading us to the development of a bespoke inference server for our application using *FastAPI*[10] and the published *AudioLDM* endpoints on the Huggingface repository. In contrast to *VroomAI*, we enable the capability to adjust audio parameters such as envelope, gain, and tuning, as well as model parameters including the negative prompt, number of inference steps, guidance scale, audio length, and the seed directly within the instrument. Unlike our current version, it is possible to set looping point in *VroomAI*, a feature absent in our project but that we want to explore in later revisions of this first prototype. Lastly, the distribution process for *VroomAI* and it's *Gradio* server necessitates source code compilation by the user, a technical barrier that has been successfully eliminated in the deployment of our final product.

3 Methods

3.1 Application Design

To ensure the developed instrument exhibits minimal latency during use, the predominant choice for programming language in real-time audio applications C++ [3, 5] is being used in conjunction with the *JUCE* framework. Enriching the framework with the Module *PluginGuiMagic*[11] helped in the development of the GUI (see figure 1).

JUCE is an open-source C++ codebase, enabling the creation of standalone software across multiple platforms and different plugin formats. The framework provides an abstraction layer for audio processing and *MIDI* from native audio devices on each platform or a Host-DAW. The digital signal processing (DSP) library offered by *JUCE* facilitates rapid prototyping and implementation of various audio effects, filters, instruments, and generators. Additionally, *JUCE* offers a multitude of classes addressing common challenges in audio project development, including the management of graphics, sound, user interaction, and network communication. [25]

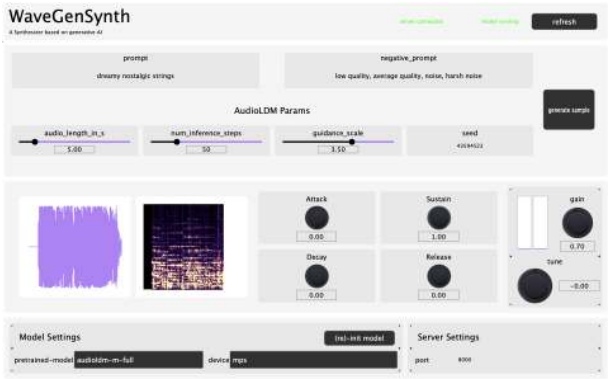


Figure 1. Graphical interface to connect to inference server, choose pretrained model, define prompt, model parameters, playback parameters and visualize generated waveform

[3] https://www.theaudioprogrammer.com/neural-audio
[4] https://www.theaudioprogrammer.com/
[5] https://vroomai.com/
[6] https://juce.com/
[7] https://www.gradio.app/
[8] https://github.com/haoheliu/AudioLDM/
[9] https://developer.nvidia.com/cuda-zone
[10] https://fastapi.tiangolo.com/
[11] https://foleysfinest.com/developer/pluginguimagic/

To integrate the *diffusion* models *AudioLDM* and *AudioLDM2* into the *JUCE* digital instrument framework, it is necessary to facilitate their inference directly from the instrument's C++ code. Oliver Larkin's suggestion[12] of generating binary representations in ONNX [13] or Torchscript [14] formats was not feasible due to the models' complex structures and dependencies on various sub-models. Instead, we employed a *FastAPI*[15] server to host instances of *AudioLDM* and *AudioLDM2* models, such as *audioldm-s-full-v2*, *audioldm-m-full*, *audioldm-l-full*, *audioldm2*, *audioldm2-large*, and *audioldm2-music*, published via HuggingFace[16,17]. These models are accessible via an API endpoint, allowing initialization with a specified *torch device*. For silicone architectures, "mps" is the recommended torch device, whereas "cuda" is suitable for NVIDIA GPUs, when supported, and "cpu" for remaining cases. Choosing model, device and initializing is facilitated in the Instruments GUI (see figure 2). Model inference is performed by defining parameters such as prompt, negative-prompt, audio length, inference steps, guidance scale and an optional 8-Byte large seed, also settable in the GUI (see figure 3), with the dedicated endpoint returning the generated audio as a *Base64* encoded signal for processing in the instrument's C++ code.

Once generated, the audio signal is visualized through

Figure 2. Options to set model, torch-device and initialize the model in the GUI

Figure 3. Options to set prompt, model parameters and seed in the GUI

its waveform and spectrogram and loaded into a sampler, which was implemented by augmenting the provided *Juce* class for continuous pitch shifting over an octave and also allows usage of the keyboard's pitch bender with a standard two-semitone range. During playback, the GUI (see figure 4) allows users to modify the sound's Envelope by adjusting

Figure 4. Options to change the envelope, gain, and tuning in the GUI

the Attack, Decay, Sustain, and Release parameters. It also provides options to adjust the gain and tune the instrument, ensuring the pitch of the generated audio aligns with the MIDI mapping.

For macOS, we generated a *.app* application, a *VST3*, and an *AU* file, which were collectively packaged into a *pkg* installer. The inference server was encapsulated into a standalone application using PyInstaller[18], simplifying its launch by bundling all necessary dependencies into a single package. Despite the original Silicon server script being only 5 KB, this packaging process increases the file size to 450 MB due to the inclusion of all required modules. The increase in storage requirement is substantial but indispensable for distributing and publishing the server and thus a usable instrument. For systems experiencing unstable performance or issues with the server application, running the server script within a specified Conda[19] environment is an alternative solution.

The digital instrument was tested both on macOS with Silicon, Intel chipset. When using the server application on Intel Macs with a mandatory torch-device "cpu", poor and unstable performance was observed. In this case, it is advisable to run the script for the server in the Conda environment with the specified Python modules.

The code for the instrument is published at *GitHub*[20]. The repository includes the source code for both the digital instrument and the *FastApi* server. Additionally, a plugin installer for Mac, executable server files for Silicon and Intel architectures on Mac, and a zip file are published[21]. The zip file contains the Python script for the server and the required Python packages for Conda, enabling the server to run without an executable file. The README file provides instructions for installing and running the code.

[12] https://www.youtube.com/watch?v=t662qg12f_Y
[13] https://onnx.ai/
[14] https://pytorch.org/docs/stable/jit.html
[15] https://fastapi.tiangolo.com/
[16] https://huggingface.co/cvssp/audioldm
[17] https://huggingface.co/cvssp/audioldm2
[18] https://pyinstaller.org/en/stable/
[19] https://www.anaconda.com/
[20] https://github.com/suckrowPierre/WaveGenSynth
[21] https://github.com/suckrowPierre/WaveGenSynth/releases/

3.2 Study Design

To assess the suitability of *AudioLDM* and *AudioLDM2* models for generating short audio samples in our application, we conducted a survey involving n=6 participants, some of whom are experienced in music production, making them apt for this evaluation. Participants rated their music production expertise on a 1-10 scale, with the average score being 5, indicating a moderate level of experience. The survey's design used Euler's square to ensure question distribution diversity, mitigating the risk of repetitive answer patterns due to convenience or habit. Our results were measured by posing questions related to both the quality of sound and how well the audio reflects the given input prompt (audio-text alignment) on a Likert scale of 1-5, where 1 represents the most unpleasant and 5 represents the most pleasant result.

All audio was generated with a fixed seed and following negative prompts: "low quality", "average quality", "noise", "high pitch", "artefacts". Our main goal was to investigate the following points:

Influence of device: Due to numerical precision and therefore small precision variations, parallelization and order of operations, and other reasons, results may sometimes show small differences when executed on different torch-devices. Therefore, as a first step, we decided to check whether the choice of device had a influence on the result of the generated sounds. The measurement was done for four prompts "Warm organ chord", "Ambient ocean waves", "Bright bell sound" and "Smooth synth lead" (see figure 5). The length of the audio has been adjusted to 5 seconds, with 50 inference steps and a guidance scale of 3.

Influence of guidance scale: *Diffusion* models use a guidance scale to tell the model how closely it should stick to the given prompt [8, 15]. Lower numbers allow for more diversity and creativity in the results, while higher numbers cause the model to adhere more strictly and thus more fittingly to the text prompt. We let the participants evaluate generated audio for the prompts "Gentle guitar strum", "Tribal African drum circle", "vocal harmonies". (see table 1).

Influence of inference steps: In *stable diffusion*, the term *inference steps*, also referred to as *reverse diffusion steps* refers to the number of passes the model makes over an image before it produces a final result [14, 15]. As a rule, more steps mean better results. However, beyond a certain threshold, the marginal benefit of increasing the number of steps diminishes [14]. We let participants rate the generated audio for different inference steps to determine if there was a sweet spot for generating audio with audio length adjusted to 5 seconds and the guidance scale set to 3. For this purpose, we generated audio for 5, 10, 20, 50, 100, 200, and 400 steps for the same prompts (see figure 6).

Influence of duration: To be usable for digital instruments, the sound quality and audio-text alignment of the sounds must be as good as possible for sounds of different lengths. If there is a loss of quality, it is important to know this to limit the generation of sounds to the range in which the quality is still sufficiently good. We have therefore analyzed the sound quality and the audio-text alignment of the sounds as a function of the audio length for the prompts "Chirping birds at dawn", "A choir pad", and "An ambient electronic pad". The guidance scale has been adjusted to 3 and the number of inference steps has been set to 50.

Influence of the prompt: The prompt has a decisive influence on the sound. Therefore, the participants were asked to rate 32 generated sounds based on their fidelity to the describing prompt. The length of the audio has been adjusted to 5 seconds, with 100 inference steps and a guidance scale of 3. (see appendix A.1 table 2 - 6)

Other observations: While this study primarily focused on specific variables and parameters, it also yielded complementary observations that, although not directly incorporated into the study design, provide valuable insights into the broader context of the research topic. Despite their anecdotal or tangential nature, these observations merit recognition and consideration of their potential implications. These observations were brought to the attention of the study participants and discussed verbally with us. The results of this will be discussed in the following chapter.

4 Results

The results of the present study are based on a small sample size (n=6), which leads to a limitation in terms of statistical significance. With so few participants, the statistical power of the ANOVA test is low [21], which affects the reliability of the results and the generalisability of the conclusions. Consequently, the results of this study should be treated with particular caution, as ANOVA tests with such a small sample are not powerful enough to draw reliable conclusions. It is recommended that these results be considered as preliminary and that further research be conducted with a more appropriate number of participants to confirm the findings and increase their validity.

Analysis of participant feedback reveals that model selection impacts outcomes more than the choice of computing device (see figure 5). Interestingly enough using consistent seeds and parameters, audio generation varied across different torch devices, with CPU-produced audio being less favorable. Among the tested devices, the base *AudioLDM2*

model was preferred for its superior general audio quality and accurate audio-text alignment. Surprisingly, despite its simplicity, the base *AudioLDM* model outperformed the more complex *audioldm2-large* model, indicating that a higher number of parameters does not necessarily equate to better performance in this context.

Model characteristics other than the larger parameter size of

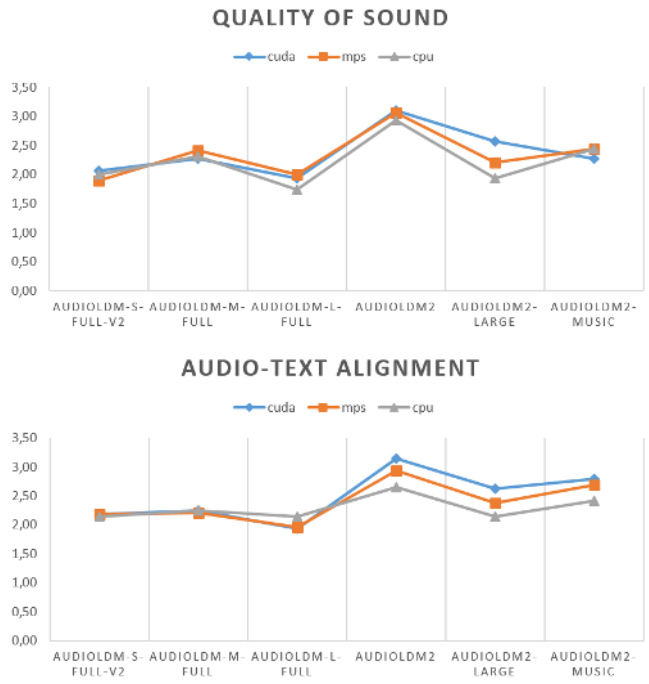


Figure 5. Impact of the choice of device type and model on the average perceived sound quality and the audio-text alignment on a Likert scale from 1-5.

the *AudioLDM2* models may also influence the results. The smaller training set of the *audioldm2-music* model compared to the other *Audioldm2* models, the specific music conditioning of *audioldm2-music*, or the fact that *audioldm-m-full* also performs the audio conditioning have different effects on the results depending on the task.

Lower guidance-scale values cause the model to follow the prompt less strictly. Therefore it is natural that the perceived audio-text alignment increases with higher values of the guidance scale. However, the perceived quality of the sound also seems to increase slightly. The results showed that optimum results can be expected for values from a guidance scale of 4-5 (see table 1)

When looking at the results of the inference steps, it is noticeable that perceived sound quality as well as audio-text alignment initially rises sharply, but then falls. Naturally,

Table 1. Guidance scale evaluation on perceived sound quality and audio-text alignment for all models on average, measured on a Likert scale from 1 to 5 while 1 means very poor and 5 is very high quality.

	1	2	3	4	5
Sound Quality	2,03	2,27	2,33	2,42	2,41
Audio-Text Alignment	2,10	2,34	2,33	2,46	2,47

one anticipates enhanced outcomes through increased iterations, as repeated refinement generally improves quality. Yet, encountering a deterioration or stagnation of results appears paradoxical initially. This phenomenon, common in *diffusion* generators, occurs because adding more information to a fully generated image yields no additional value and merely supersedes existing details.

The participants' ratings were approximately equal for the

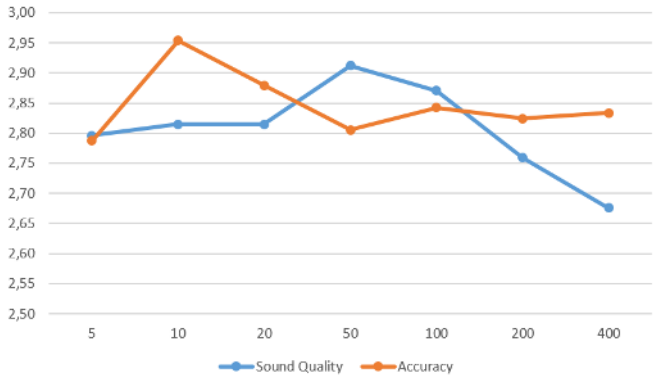


Figure 6. Effect of number of inference steps from 5-400 on the perceived audio quality and audio-text alignment on the average of all variants of *AudioLDM*, measured on a Likert scale from 1-5.

sound quality and audio-text alignment of the audio for all audio lengths between 5 and 30 seconds. In some of the longer generated audio a sort of pitch drifting was observed. This could be an undesired or appreciated side effect based on the use case and personal preference.

The expectation that prompts like "A kickdrum", "A snare", "Loud clap sound", and "A gong hit" would elicit a singular, non-repetitive audio event was not fully met (see appendix A.1 figure 2). The models often produced signals with repeated audio events, indicating a tendency towards creating rhythmic musical patterns rather than isolating a single sound event. A more precise prompt, such as "A single", seems to be more effective with *audioldm-m-full*. The *audioldm2-music* model tends to generate repetitive structures, presumably because its primary function is to produce

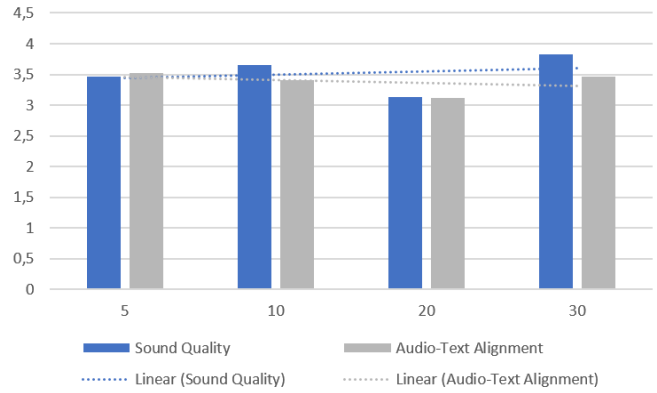


Figure 7. Impact of the duration of the generated audio (5 to 30 seconds) on the perceived audio quality and audio-text alignment, measured on a Likert scale from 1-5.

musical compositions rather than singular sound events.

Attempts to synthesize characteristic instrument sounds using prompts like "FM synthesis bells", "mellotron chords", "A bagpipe melody", "A guitar string", and "A piano chord" did not achieve the anticipated outcomes with *audioldm-m-full*, while *audioldm2* and *audioldm2-music* proved more adequate for this task. (see appendix A.1 figure 3). The sounds generated by *audioldm-m-full* were abstract and failed to accurately replicate the intended instruments. Additionally, it was noted that the less satisfactory outcomes from *AudioLDM2* exhibited a tendency to sound alike.

Adding emotional and descriptive elements to prompts, including "Dark pad sound", "An ethereal shimmering synth pad", "An angelic choir", "dreamy nostalgic strings", and "a sad violin solo", generally did not lead to favorable outcomes, (see appendix A.1) figure 4). It has been noted again that the outcomes from *AudioLDM2*, despite variating of prompts, show that less satisfactory results share similar sound structures.

The models appear capable of adequately responding to specific descriptions of musical and acoustic effects (see appendix A.1 figure 5). Interestingly, the *AudioLDM* models seem to perform better in this aspect than the *AudioLDM2* models, although one might expect the latter to achieve better results given their *LOA* representation.

Efforts to craft prompts incorporating specific sounds and terminology common in music production and pop culture resulted in unsatisfactory outcomes with *audioldm2* and achieved mediocre results with *audioldm2-music* (see appendix A.1 figure 6). Attempts to synthesize a 808-Kickdrum, prevalent in hip-hop, or a 909-Kickdrum, a staple in electronic music, resulted in timbres that failed to match the expected specifications. Similarly, generating the widely used Amen Break did not lead to the anticipated success. Only *audioldm-m-full* managed a resemblance to a 303 Baseline, with other models falling short. The prompts "A Juno-106 pad" and "Oberheimer OB-Xa string pads" yielded functional pads, implying sustained tones or chords, yet whether they authentically replicate the unique sounds of these instruments remains a matter of debate.

5 Discussion

The analysis indicates that existing models in music production may benefit from further refinement to improve their sensitivity to input prompts. Addressing these issues may involve training on a broader range of audio data from sources such as synthesizer or sampler manufacturers, audio libraries (e.g., *Splice*[22]), music studios, or producers, who often maintain personal sound libraries for their work. Fine-tuning models with more fitting music production data may lead to a more tailored and effective generative AI tool, as demonstrated by the customization of *AudioLDM* discussed in [20]. This approach aligns with the trend towards personalized *LORA*s [9] in other creative fields. Aggregating individual sound libraries into a unified database could provide a democratic solution to the lack of datasets in this area, offering a basis for more precise fine-tuning and training of models.

Employed models should aim to produce outputs at the sampling rate of 48kHz recommended by the *Audio Engineering Society* [2]. The models *AudioLDM* and *AudioLDM2* output at a rate of 16kHz, thus falling short of generating sounds in High Fidelity.

The failure to create an *ONNX* or *TorchScript* representation of the model for inference in C++ introduces complications in bundling and distributing the model. The creation of such a binary file for the utilized model would eliminate the need for a server and an API. Likewise, the requirement to use multiple programming languages for an implementation would no longer be necessary.

The current Sampler implementation lacks the ability to set start and end points for signal playback, limiting the selection of specific sound events from sequences. Additionally, it misses the capability to loop segments for continuous play, a standard feature in samplers since the 1990s. This brings forth the question of the necessity to develop a proprietary

[22] https://splice.com/

Sampler. Inspired by interfaces like *Text generation web UI*[23], *ComfyUI*[24], and *Stable Diffusion web UI*[25], a specialized interface could be created. This interface could integrating with current music production software while allowing detailed parameter adjustments, model fine-tuning, inpainting, style transfer and many other options. Assuming the model's effectiveness, such a platform could potentially make services like *Splice* redundant.

Incorporating machine learning into development of a custom Sampler could enhance realism by adjusting overtones for different pitches, a necessity beyond simple frequency modulation. This approach reflects the complexity of piano timbres, where each note's spectrum varies significantly [19]. A machine learning model could be trained to modify these subtle differences accurately, taking an audio signal as input and producing outputs for all possible pitches. This process necessitates a comprehensive dataset of audio signals from multiple instruments, encompassing all pitches.

Some musicians show a preference for hardware over a digital workflow due to its tactile feedback. The sound synthesis technique described in this study could be adaptable to dedicated hardware, utilizing deep learning solutions on microcontrollers, as seen with Google's *Coral*[26] and Nvidia's *Jetson*[27]. This would be a more accessible approach for musicians not wanting to invest time in setting up the software. Nonetheless, compatibility issues may arise, such as the requirement for models to be compatible with *TensorFlow Lite*[28] on *Coral* devices. We were able to run *AudioLDM2* successfully on Nvidia's *Jetson AGX Xavier Development Kit*, suggesting that the proposed methodology could extend to portable *Jetson* devices, transforming them into standalone, text-driven musical instruments with the addition of a soundcard and a MIDI keyboard for sound output and control.

The instrument exhibits several deficiencies that require optimization, such as the manual tuning process, which could benefit from automation. Additionally, it lacks the capability to save and recall generated sounds and settings, a functionality prevalent in numerous digital instruments. During the development of the initial prototype, the specification and verification of the source code were not conducted. Future implementations should include these processes to reduce the likelihood of bugs and errors in the software. Despite the identified limitations, the digital instrument, through *diffusion* models, has shown capability in producing playable and editable sounds based on user inputs. It also integrates seamlessly into contemporary music production and applications, pioneering AI-driven sound synthesis. Unlike other media, such as images, even suboptimal outputs from this instrument can be creatively repurposed, enhancing soundscapes through extensive editing and effects. This is illustrated by musician "Heinbach," who adopts Bob Ross's "Happy Little Accidents"[29] approach to refine unexpected sounds into novel timbres and soundscapes. Consequently, while not primarily designed for precise sound modeling, the instrument encourages exploration with innovative and unforeseen sounds.

6 Conclusions

Our research shows the capabilities and limitations of audio generation models within the framework of a digital instrument. Despite *AudioLDM* and *AudioLDM2* models not achieving perfection in the music-production context, and their complexity preventing binary conversion to simplify implementation and distribution, the trajectory of audio generation development is foreseeable. The evaluation of these models has uncovered limitations that mainly restrict their use in the discovery of novel, sometimes unforeseen sounds. Nonetheless, despite these limitations, there lies the potential to use these unforeseen and perhaps alien sounds as a basis for further manipulation and transformation. The modular design of our digital instrument notably facilitates the seamless replacement of one *diffusion* model with another, thereby ensuring adaptability and readiness for future enhancements. Future models with improved sound quality and fewer artifacts could challenge traditional instruments. Additionally, the potential for individual fine-tuning through one's sound libraries has been recognized, and an improvement in sampling rate to 48kHz has been advised.

The absence of features that allow for the adjustment of start and end points of the played sounds as well as the definition of loop points for sustaining a sound continuously limits the editing capabilities of the generated sound. Moreover, the instrument would benefit from additional optimizations such as an automatic tuning function and the ability to save and retrieve sounds.

Despite these deficiencies, it is undeniable that the work highlights the potential of musical applications of *diffusion* models, offering improved and more versatile integration possibilities into modern music production ecosystems compared to similar applications. Users can parameterize and generate sounds according to their preferences and modify the sound behavior during play. Input devices can be connected via MIDI.

[23] https://github.com/oobabooga/text-generation-webui
[24] https://github.com/comfyanonymous/ComfyUI
[25] https://github.com/AUTOMATIC1111/stable-diffusion-webui
[26] https://coral.ai/
[27] https://developer.nvidia.com/embedded/jetson-modules
[28] https://www.tensorflow.org/lite
[29] https://www.youtube.com/watch?v=Ibajy9A91ls

We hope that future research in this field will benefit from the discoveries made in this work and will continue to build upon them. Perspectives for further diverse potential integrations of generative Artificial Intelligence into the music production process have been highlighted.

Acknowledgments

We would like to express our gratitude to Daniel Walz of *Plugin GUI Magic*[30] for his invaluable assistance and guidance throughout the implementation of the graphical user interface in this research project. We also extend our thanks to the *AudioLDM* and *AudioLDM2* team for their significant contributions to the development and publication of their models. Additionally, we appreciate the support of *The Audio Programmer*[31] Community.

A Study Data

A.1 Prompt Evaluation

Table 2. Perceived audio-text alignment in describing a **single event** on Likert scale from 1 Orange to 5 Blue where 1 represents the most unpleasant and 5 represents the most pleasant result.

	Model		
Prompt	audioldm2	audioldm-m-full	audioldm2-music
A kickdrum	2.17	3.33	2.67
A single kickdrum	3.17	2.67	1.00
A snare	2.67	2.67	1.67
A single snare	2.25	3.67	1.50
A single Light triangle ting	3.17	3.67	4.00
Loud clap sound	1.00	1.17	1.33
A gong hit	2.17	3.50	2.50

Table 3. Perceived audio-text alignment in **describing an instrument** on Likert scale from 1 Orange to 5 Blue where 1 represents the most unpleasant and 5 represents the most pleasant result.

	Model		
Prompt	audioldm2	audioldm-m-full	audioldm2-music
FM synthesis bells	3.67	2.00	3.83
Mellotron chords	2.67	1.17	2.67
A bagpipe melody	4.67	2.00	2.17
A guitar string	1.67	1.00	3.17

Table 4. Perceived audio-text alignment for audios that should encode **emotions and adjectives in prompt** on Likert scale from 1 Orange to 5 Blue where 1 represents the most unpleasant and 5 represents the most pleasant result.

	Model		
Prompt	audioldm2	audioldm-m-full	audioldm2-music
Dark pad sound	3.50	1.50	3.67
An ethereal shimmering synth pad	2.50	2.67	2.00
An angelic choir	1.80	3.33	1.50
Dreamy nostalgic strings	1.67	1.50	1.83
A sad violin solo	1.67	3.67	2.00

Table 5. Perceived audio-text alignment in translating **audio effects** on Likert scale from 1 Orange to 5 Blue where 1 represents the most unpleasant and 5 represents the most pleasant result.

	Model		
Prompt	audioldm2	audioldm-m-full	audioldm2-music
Long sustain snare hit	1.67	2.33	2.00
A fluttering harp with crystal echoes	3.33	1.00	3.50
A synth with delay effect	3.50	3.50	2.67
Echoing synth stabs	2.33	3.80	3.50
A distorted synth	2.83	4.83	1.33
A detuned synth	2.17	3.00	2.50
Reverse cymbal	1.83	2.83	2.00
A kickdrum with a lot of reverb	2.80	2.00	2.80

Table 6. Perceived audio-text alignment in describing **music production specific prompts** on Likert scale from 1 Orange to 5 Blue where 1 represents the most unpleasant and 5 represents the most pleasant result.

	Model		
Prompt	audioldm2	audioldm-m-full	audioldm2-music
an 808 kickdrum	1.83	3.50	1.67
The amen break	1.83	2.17	2.17
a 909 snare	1.00	2.00	2.33
a 303 baseline	1.83	3.00	2.33
A jungle drum break	1.67	3.00	1.83
A Juno-106 pad	1.83	3.00	2.67
Oberheimer OB-Xa string pads	2.33	2.83	2.67

[30] https://foleysfinest.com/developer/pluginguimagic/
[31] https://www.theaudioprogrammer.com/

References

[1] Andrea Agostinelli, Timo I. Denk, Zalán Borsos, Jesse Engel, Mauro Verzetti, Antoine Caillon, Qingqing Huang, Aren Jansen, Adam Roberts, Marco Tagliasacchi, Matt Sharifi, Neil Zeghidour, and Christian Frank. 2023. MusicLM: Generating Music From Text. https://doi.org/10.48550/arXiv.2301.11325 arXiv:2301.11325 [cs.SD]

[2] Audio Engineering Society, Inc. 2018. *AES5-2018 (Rev. AES5-2008) - AES recommended practice for professional digital audio — Preferred sampling frequencies for applications employing pulse-code modulation.* Technical Report AES5-2018. Audio Engineering Society, Inc. https://www.aes.org/tmpFiles/aessc/20231014/aes05-2018-r2023-i.pdf

[3] Richard Charles Boulanger and Victor Lazzarini (Eds.). 2011. *The audio programming book* (Cambridge, Mass, 2011). MIT Press. OCLC: ocn503654559.

[4] Prafulla Dhariwal and Alex Nichol. 2021. Diffusion Models Beat GANs on Image Synthesis. https://doi.org/10.48550/arXiv.2105.05233 arXiv:2105.05233 [cs.LG]

[5] Timur Doumler. 2015. C++ in the Audio Industry. (2015). https://www.youtube.com/watch?v=boPEO2auJj4 CppCon 2015.

[6] Harshvardhan GM, Mahendra Kumar Gourisaria, Manjusha Pandey, and Siddharth Swarup Rautaray. 2020. A comprehensive survey and analysis of generative models in machine learning. *Computer Science Review* 38 (2020), 100285. https://doi.org/10.1016/j.cosrev.2020.100285

[7] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising Diffusion Probabilistic Models. https://doi.org/10.48550/arXiv.2006.11239 arXiv:2006.11239 [cs.LG]

[8] Jonathan Ho and Tim Salimans. 2022. Classifier-Free Diffusion Guidance. https://doi.org/10.48550/arXiv.2207.12598 arXiv:2207.12598 [cs.LG]

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. https://doi.org/10.48550/arXiv.2106.09685 arXiv:2106.09685 [cs.CL]

[10] Po-Yao Huang, Hu Xu, Juncheng Li, Alexei Baevski, Michael Auli, Wojciech Galuba, Florian Metze, and Christoph Feichtenhofer. 2023. Masked Autoencoders that Listen. https://doi.org/10.48550/arXiv.2207.06405 arXiv:2207.06405 [cs.SD]

[11] C. Jarzynski. 1997. Equilibrium free-energy differences from nonequilibrium measurements: A master-equation approach. *Physical Review E* 56, 5 (Nov. 1997), 5018–5035. https://doi.org/10.1103/physreve.56.5018

[12] Diederik P Kingma and Max Welling. 2022. Auto-Encoding Variational Bayes. https://doi.org/10.48550/arXiv.1312.6114 arXiv:1312.6114 [stat.ML]

[13] Alex Lamb. 2021. A Brief Introduction to Generative Models. https://doi.org/10.48550/arXiv.2103.00265 arXiv:2103.00265 [cs.LG]

[14] Haohe Liu, Zehua Chen, Yi Yuan, Xinhao Mei, Xubo Liu, Danilo Mandic, Wenwu Wang, and Mark D. Plumbley. 2023. AudioLDM: Text-to-Audio Generation with Latent Diffusion Models. arXiv:2301.12503 [cs.SD] https://arxiv.org/abs/2301.12503

[15] Haohe Liu, Qiao Tian, Yi Yuan, Xubo Liu, Xinhao Mei, Qiuqiang Kong, Yuping Wang, Wenwu Wang, Yuxuan Wang, and Mark D. Plumbley. 2023. AudioLDM 2: Learning Holistic Audio Generation with Self-supervised Pretraining. https://doi.org/10.48550/arXiv.2308.05734 arXiv:2308.05734 [cs.SD]

[16] Eli Maor. 2018. *Music by the Numbers: From Pythagoras to Schoenberg*. Princeton University Press.

[17] Radford M. Neal. 1998. Annealed Importance Sampling. https://doi.org/10.48550/arXiv.physics/9803008 arXiv:physics/9803008 [physics.comp-ph]

[18] Alex Nichol and Prafulla Dhariwal. 2021. Improved Denoising Diffusion Probabilistic Models. https://doi.org/10.48550/arXiv.2102.09672 arXiv:2102.09672 [cs.LG]

[19] Barry R. Parker. 2009. *Good vibrations: the physics of music*. Johns Hopkins University Press. OCLC: ocn320194527.

[20] Manos Plitsis, Theodoros Kouzelis, Georgios Paraskevopoulos, Vassilis Katsouros, and Yannis Panagakis. 2023. Investigating Personalization Methods in Text to Music Generation. https://doi.org/10.48550/arXiv.2309.11140 arXiv:2309.11140 [cs.SD]

[21] Patrick J Potvin and Robert W Schutz. 2000. Statistical power for the two-factor repeated measures ANOVA. *Behavior Research Methods, Instruments, & Computers* 32, 2 (2000), 347–356. https://doi.org/10.3758/BF03207805

[22] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. https://doi.org/10.48550/arXiv.2103.00020 arXiv:2103.00020 [cs.CV]

[23] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. https://api.semanticscholar.org/CorpusID:160025533

[24] Hannes Raffaseder. 2010. *Audiodesign* (2., aktualisierte und erweiterte auflage ed.). Hanser.

[25] Martin Robinson. 2013. *Getting started with JUCE: leverage the power of the JUCE framework to start developing applications*. Packt Publishing. OCLC: 862386437.

[26] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-Resolution Image Synthesis with Latent Diffusion Models. https://doi.org/10.48550/arXiv.2112.10752 arXiv:2112.10752 [cs.CV]

[27] André Ruschkowski. 2019. *Elektronische Klänge und musikalische Entdeckungen* (3., ergänzte auflage 2019 ed.). Number 19613 in Reclams Universal-Bibliothek. Reclam.

[28] Lars Ruthotto and Eldad Haber. 2021. An introduction to deep generative modeling. *GAMM-Mitteilungen* 44, 2 (2021), e202100008. https://doi.org/10.1002/gamm.202100008 arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/gamm.202100008

[29] C.E. Shannon. 1949. Communication in the Presence of Noise. *Proceedings of the IRE* 37, 1 (1949), 10–21. https://doi.org/10.1109/JRPROC.1949.232969

[30] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. 2015. Deep Unsupervised Learning using Nonequilibrium Thermodynamics. https://doi.org/10.48550/arXiv.1503.03585 arXiv:1503.03585 [cs.LG]

[31] Kinko Tsuji and Stefan C. Müller. 2021. *Physics and music: essential connections and illuminating excursions*. Springer Nature.

[32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. https://doi.org/10.48550/arXiv.1706.03762 arXiv:1706.03762 [cs.CL]

[33] Harvey Elliott White and Donald H. White. 2014. *Physics and music: the science of musical sound*. Dover Publications, Inc. OCLC: 878661301.

[34] Yusong Wu, Ke Chen, Tianyu Zhang, Yuchen Hui, Taylor Berg-Kirkpatrick, and Shlomo Dubnov. 2023. Large-scale Contrastive Language-Audio Pretraining with Feature Fusion and Keyword-to-Caption Augmentation. https://doi.org/10.48550/arXiv.2211.06687 arXiv:2211.06687 [cs.SD]

[35] Dongchao Yang, Jianwei Yu, Helin Wang, Wen Wang, Chao Weng, Yuexian Zou, and Dong Yu. 2023. Diffsound: Discrete Diffusion Model for Text-to-sound Generation. https://doi.org/10.48550/arXiv.2207.09983 arXiv:2207.09983 [cs.SD]

Received 2024-06-02; accepted 2024-07-02

A Progressive-Adaptive Music Generator (PAMG)

An Approach to Interactive Procedural Music for Videogames

Alvaro Eduardo Lopez Duarte
University of California, Riverside, USA
Riverside, USA
alvaro.lopez@ucr.edu

Abstract

Procedural music generation in video game is rarely used in development despite its potential support for interactive narratives, and falls behind broadly-used procedural methods like graphic management. Currently, developers rely on pre-produced audio sequences, focusing on sound quality while balancing storage and variety. Techniques like layering and re-sequencing are standard through sound design platforms like FMOD and Wwise, allowing adaptation and interactivity in gameplay. However, this can lead to repetitive audio, which becomes particularly noticeable in extended gaming sessions, diminishing the impact of musical storytelling.

This study introduces the Progressive Adaptive Music Generator (PAMG), an algorithmic system that generates continuous music streams based on gameplay variables, seamlessly transitioning between moods, styles, and tension levels. It includes motivation, theoretical framework in the field of generative music, overview of the system structure, and a preliminary implementation test involving a Trial Game that compares conventional implementation with PAMG. In the discussion section, it addresses test results and issues, and future work. The study reveals a preference for PAMG's music over traditional methods in certain aspects, although results are not conclusive.

CCS Concepts: • **Computing methodologies** → Modeling methodologies; • **Applied computing** → Sound and music computing.

Keywords: AI Music, Game Music, Procedural Music

ACM Reference Format:
Alvaro Eduardo Lopez Duarte. 2024. A Progressive-Adaptive Music Generator (PAMG): An Approach to Interactive Procedural Music for Videogames. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy*. ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3677996.3678291

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
FARM '24, September 2, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3678291

1 Introduction

In game development, elements like characters, environments, and events are algorithmically linked through gameplay variables. Real-time interactions between light, textures, and colors determine rendering. Such realism enhances storytelling, narrative immersion, and potentially extends gameplay. From a player's perspective, gameplay combines all elements to create an immersive experience. Engagement hinges on a mix of surprise, satisfaction, and challenge [1], requiring varied emotional support from music. Current technologies offer interactive solutions based on audio clips that use variables to layer, re-sequence, and remix material which has supported transitions and immersive player experiences. Real-time music generation, compared to longer audio segments, is believed to offer greater responsiveness, much like smaller buffer sizes reduce latency in digital audio. Note-sized musical elements potentially provide deeper adaptability and variety, tightly responding to game states, actions, and events. Unlike machine learning approaches requiring extensive data sets, PAMG utilizes a multidimensional algorithmic music-feature setup that responds in real time to gameplay variables, making it less resource intensive and providing a wider and more direct control over known musical characteristics. This project explores algorithmic composition through the design and demonstration of PAMG for multi-linear, real-time music generation. It involves the translation of compositional concepts into logical structures and the design of parameter input/output for interactive, reactive music creation. Building upon theoretical roots and existing methods in algorithmic composition, it assesses their applicability and modifications. The project features a gameplay experiment to test PAMG's real-time control against traditional clip-based implementation (CBI) of music, followed by data analysis, identification of limitations, and future research directions. Testing includes the design and development of the Trial Game in which participants assess PAMG's performance against CBI.

2 The Field of Algorithmic Generative Music

This overview contextualizes the PAMG design within the broader field of algorithmic composition, tracing its inspiration from general music generation concepts and seminal works to specialized generative adaptive music for video games. For comprehensive historical model survey refer to [40] and [65].

2.1 Algorithmic Composition in Connection with PAMG

Algorithmic composition, a methodical instruction sequence for problem-solving, has been instrumental in music creation pre-dating computers [38]. Cope's Experiments in Musical Intelligence [9] and Hiller and Isaacson's Illiac Suite [21], employing computational models with Markov chains [41], are seminal works. While these models use numerical symbols for note representation, akin to PAMG, their approach differs: EMI transforms existing music, and Hiller and Isaacson apply Markov models for feature analysis [2]. In contrast, PAMG generates music using measurable parameters, aligning with knowledge-based systems [42] that follow explicit rules for aesthetic consistency. PAMG's structures are akin to grammars, extensively used in harmonic progression construction [8, 9, 54], a distinct approach within algorithmic composition.

2.1.1 Multi-Agent Systems (MAS). MAS, known for distributed autonomy and perceptual-action capabilities [56], mirror musical ensembles in their coordination of independent entities. PAMG exemplifies this by utilizing agents responsible for creating distinct musical features like melody, harmony, rhythm, and orchestration, functioning within an ensemble-like network with a structured I/O distribution for algorithmic communication.

2.1.2 Melody Creation and Accents. Initiating with a clock-triggered pseudo-random number generator (PRNG) [36], PAMG generates melodic content via seeding, a method widely used in stochastic processes and Markov chains [3]. Unlike Markov chains that require statistical inputs, PAMG opts for seeded random walk algorithms for melody and sequence generation, offering potential for stylistic replication enhancements. In PAMG, melodic accents, characterized by perceived importance due to factors like sound level and timing variations [58], are achieved by offsetting velocity parameters. These accents are aligned with beats, including delayed or anticipated ones (syncopation), constructing the rhythmic structure of the main melody. Additionally, the pattern is applied to pitch-contour nodes in the "dcontour" mode (a mode that uses random walk to create melody contour), accommodating listener expectations of periodicity related to meter.

2.1.3 Tension, Complexity and Progression. PAMG's design prioritizes progressive transitions from simple to complex music, guided by quantized tension and complexity within Western tonal harmony [5], aligning with most commercial game music's incidental style. Key literary influences are outlined next:

Tension: Efforts to quantify musical tension reveal significant challenges, lacking a universal definition [50]. Tension in music psychology is often linked to expectations of form's conclusion or continuation [16, 22]. Tonal harmony, central to these discussions, associates tension with dualities like stability/instability and consonance/dissonance [33]. Tonal progressions, characterized by rising stress towards climaxes followed by resolution, travel within tonal space [29, 39]. Lerdahl's "tonal pitch space" [31] calculates psychological distances in music, aligning with GTTM's principles. PAMG incorporates these concepts, progressively by adding pitches across 12 levels per chord, producing increased tension. This pitch sequence mirrors historical music periods use of dissonance [10, 25] and is crafted using spectral distance-based perceptual affinities [19, 43], varied by chord function within progressions [28].

Progression Complexity: Complexity in Music Information Retrieval (MIR) often relates to cultural music evolution, preferences, and arousal [20, 63]. In tonal harmony, factors like harmonic rhythm, dissonance, and evolution define complexity [57]. PAMG's network, inspired by David Cope's use of an Augmented Transition Network (ATN), a generative-grammars approach for musical concatenation [9], manages transitions between chord levels. Unlike Cope's method, which relies on learned segments for transitions, PAMG uses Western tonal system-based chord transition probabilities [10] without pre-composed patterns, a similar approach to Tymoczko's firs-order properties on progressions and functional harmony [62]. These generative grammars handle harmonies for chords and melodic tonal grids.

Temporal Complexity: Temporal pattern perception in music is linked to an internal hierarchical clock, which adapts based on the sequence and accented events [47, 58]. [53] suggest a complexity score for this clock, favoring syncopated patterns. [32] present a hierarchical complexity framework based on cognitive segmentation of musical material. Patterns with more sub-symmetries are deemed simpler [61], while measures like the pairwise variability index [60] and Keith's complexity measure [27] link rhythmic complexity to metric grouping. In PAMG's rhythm generation, the simplest pattern is one sub-symmetry per beat. Complexity increases by progressively displacing onsets, with the direction of displacement and onset choice being crucial. Alternating directions and dividing pulses into even and odd for anticipation/delay avoids simplification. As structures of 2 are less complex than those of 3 [18], the PAMG model prefers half-pulse (8th note) displacements over quarter-pulse (16th note).

The resulting displaced pattern forms the base accent structure, akin to terminal branches in Lerdahl and Jackendoff's tree structures.

Algorithmic Orchestration: Computer-aided orchestration focuses on symbolic and sample-based methods [4]. Sample-based approaches gather timbral data from audio samples to construct instrument combinations meeting perceptual criteria [13, 48]. While playability is key in algorithmic composition [7], it's treated as a constraint in machine performance contexts [30]. In PAMG's orchestration agent, tasks include setting instrument range constraints, recombining instruments, and harmonizing with the melody agent's output. Basic orchestral functions, like spacing in lower registers and texture management, are handled by the source agent [14]. The design, currently limited to full-range instrument families, aims for modular expansion to accommodate virtual instruments for style diversification.

2.2 Adaptive-Generative Music for Videogames

Music significantly enhances immersion in media, particularly in video games, by focusing attention and reinforcing goal-oriented actions [12, 24, 26, 68]. It supports emotions, plot changes, environmental mood, pacing, aesthetic consistency, and thematic unity [64], with adaptability to multi-linear narratives boosting immersion and engagement [17, 55, 66]. Adaptive, interactive, or dynamic music, crucial in multi-linear narratives, involves music alteration based on control inputs [11, 23, 34, 46, 52]. Collins [6] defines procedural composition as controlled, rule-based, real-time evolution, echoing adaptability. Procedural content generation in games, creating content with minimal user input, also adapts music [59]. Generative music, described as ever-changing by Brian Eno, divides into Linguistic/Structural, Interactive/Behavioral, Creative/Procedural, and Biological/Emergent categories [32, 49, 67, 69]. PAMG incorporates these, generating music without pre-existing material and using algorithms configured by users. It employs non-seeded pseudo-random number generators for variance, suggesting a biological/emergent aspect, though deterministic in nature. Plut et al. [46] define generative music in games as systemically autonomous from game logic, a framework applied to PAMG. The field of generative adaptive systems in video games is expanding [23, 44], with a comprehensive list of examples provided in [45].

3 System Design

The series of figures shown in this document (Figures 1 to 7) explain the system using custom flow diagrams. Algorithms and processes are boxes with input on top and output below. Their input parameters are shown in an enclosed bold box. Processed data accessible at several points in the system by the agents is shown in ovals. Dashed boxes show process

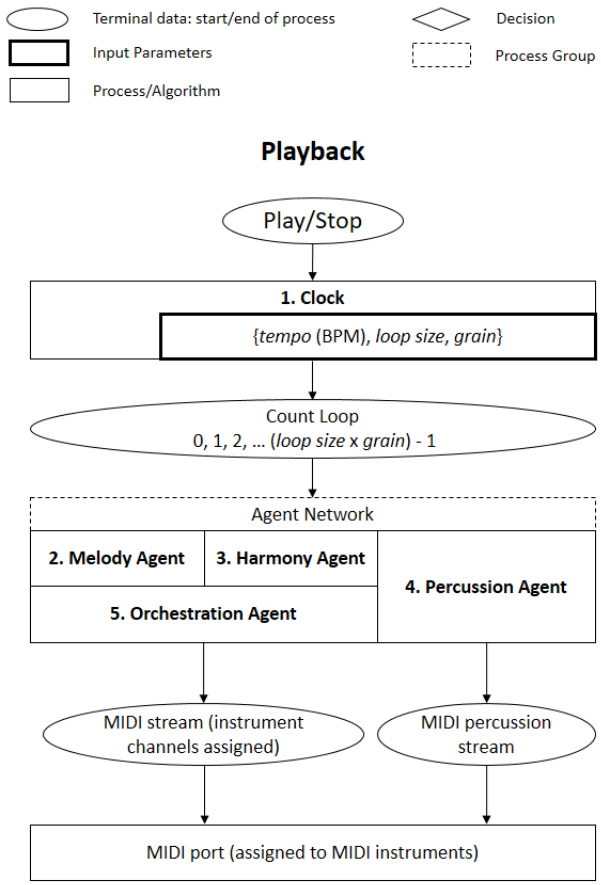


Figure 1. System overview. The flow starts with "Play/Stop" that activates the **1. Clock** which generates a time grid for events, constituting the playback algorithm's foundation. It creates a loop of indexed pulses, setting the minimum onset. Parameters include Tempo, Size, and Grain. Clock events are sent to the agent network

4 Discussion

To test PAMG, a first person shooter The Trial Game was designed. Then, setting up a Clip-Based Implementation (CBI) that employs conventional methods and software (Audiokinetic's Wwise) for game music, and that provides similar interactive properties as PAMG proved to be more time-consuming and offered less interpolation granularity. To establish a fair comparison, CBI design also employed recordings from PAMG to use the same stylistic and instrumental properties. This secures comparison between implementation results such as interactive capabilities and not music quality or audio. PAMG simplifies transitions through variable assignments and preset interpolations, whereas CBI demands extensive design and testing of numerous transition rules. The preliminary test intended to compare CBI and the real-time music generation of PAMG from the player's experience. The primary question is: "How does PAMG's

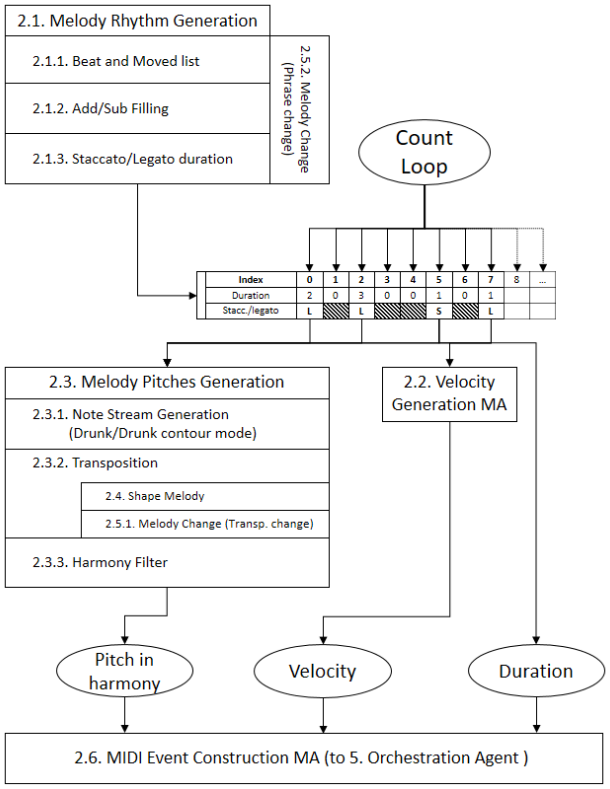


Figure 2. Melody Agent (MA): *Rhythm Generation* constructs a list of onsets based on beats and syncopates them using parameters like move even/odd and motion value. Additionally, motion patterns are shared with the other agents to emphasise the melody accents completely or partialy. *Velocity Generation* assigns velocities to onsets using a random-walk sequence, with accents on syncopated onsets. *Pitch Generation* utilizes 'drunk' and 'drunk-contour' modes for real-time pitch assignment, with parameters for step, range, and non-repetition. *Transposition and Harmony Filter* adjust pitch values according to melody shape and chord progressions sent by the Harmony Agent. *Shape Melody* modifies a long term shape through transposition over time using using the parameters depth and length. *Change Algorithm* analyzes previous pitches to adjust transposition and phrase structures. *MIDI Event Construction* combines rhythm, velocity, and pitch data to create MIDI event streams to be sent to the Orchestration Agent. It appears as a MIDI output in the three main agents.

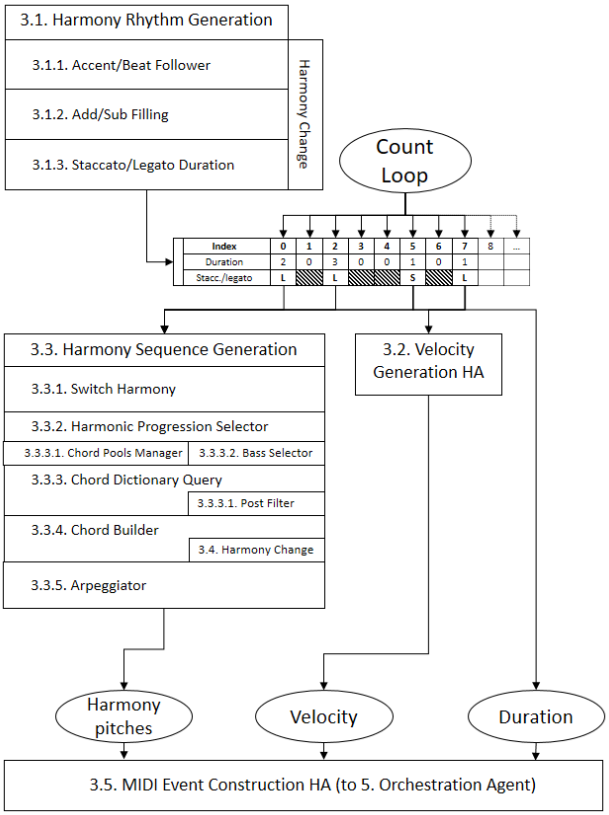


Figure 3. Harmony Agent (HA). Similar *Rhythm Generation* to MA, with accent/beat following and additive/subtractive filling. *Velocity Generation* emphasizes chord onsets depending on the main accent list provided by MA. *Harmony Sequence Generation* includes the *Switch* mechanism, the *Chord* structures available in a dictionary, and *Progression Selector* that manages harmonic content and complexity. A *Post Filter* enables filtering to switch rapidly to a new pitch sub-set. *Chord Builder* and *Arpeggiator* construct harmonies for playback. *Harmony Change* adjusts parameters based on chord progressions such as modulations and changes in rhythmic patterns.

music influence the gameplay experience compared to the pre-recorded-music in CBI?". The dependent variables were player assessments of engagement, emotional response, immersion, and curiosity for CBI and PAMG in a questionnaire, rated as clearly more/better, slightly more/better, or no significant difference. 18 participants were split equally into two groups: CBI-first and PAMG-first. After playing for 10-20 minutes the game with each implementation, they rated gameplay experience. In general, results suggested that PAMG had a slightly higher impact on participants than CBI with a punctuation in favor of PAMG of 184 (128 slightly + 56 clearly more/better) and 134 in favor of CBI (99 slightly + 35 clearly more/better). Participants who played either CBI or PAMG second tended to rate that model slightly higher. For instance, CBI slightly better increased from 16% when played first to 32% when played second. However, in both groups, PAMG clearly better consistently outscored CBI clearly better (15% vs. 8%, and 12% vs. 9%, respectively). An ANOVA with an alpha of .05 was used to test two hypotheses: H1, a significant model preference (CBI or PAMG), and H2, the

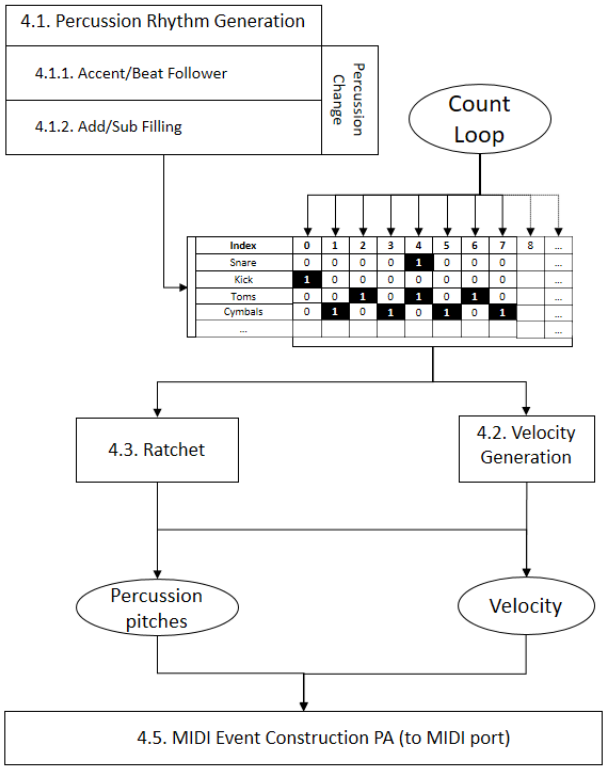


Figure 4. Percussion Agent (PA): *Rhythm Generation* acts similar to the equivalent in MA and HA but by each percussion instrument. *Velocity Generation* accentuates specific percussion onsets. *Ratchet* decides when and how fill-in rhythmic repetitions appear. *Change* aligns percussion with chord changes.

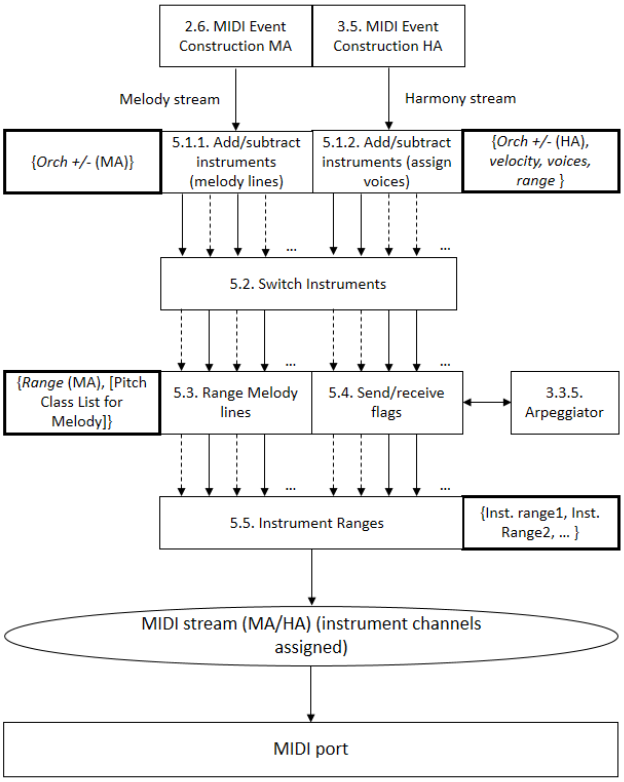


Figure 5. Orchestration Agent (OA): Manages instrument assignments for melody and harmony. Adjusts instruments, switches orchestration, and sets pitch ranges with information from the current loop and HA. Coordinates with HA's arpeggiator and manages instrument-family ranges. Submits final MIDI events.

influence of play order (CBI-first or PAMG-first) on preference. The analysis showed no significant overall preference for either model, $F(1, 17) = 2.46$, $p = .13$, slightly favoring PAMG.Additionally, no significant differences emerged in model preference between the two groups, indicated by between-subjects effects, $F(1, 16) = 1.82$, $p = .19$, and within-subjects effects, $F(1, 16) = 2.35$, $p = .14$. Votes for PAMG vs. CBI per group revealed a marginally significant effect of group interaction, $F(1,16) = 3.79$, $p = .06$, suggesting a skewed preference towards the second model. This tilt may be explained by the priming effect. Initially focused on objectives and mechanics, players gradually expand their attention to elements like background music, enhancing music assessment reliability over time.

The test issues may be solved with a larger sample, a priming gameplay without music, and a longer test. The short gameplay duration in our study limits the assessment of liking versus repeated exposure effects [37, 51]. Players encountered a musical theme 4 to 10 times, possibly experiencing only the initial phase of increased liking due to familiarity, which tends to be more prolonged in incidental than focused music settings. This could account for the 'repeating' music preference. A significant decrease in liking might require at least 32 repetitions, or roughly 80 minutes of continuous gameplay, to manifest. More information about the test can be found on [35].

PAMG has shown versatility in game music creation, yet it primarily produces western tonal orchestral scores. Adapting it for other styles as for example EDM music, a common genre employed in videogames, requires significant changes in algorithms and instrument management. For instance, EDM involves modifying bass ranges, simplifying rhythmic complexity, and generating continuous and idiomatic aural changes. In terms of real-time interaction, while PAMG allows for immediate parameter adjustments, this can disrupt musical phrasing, which is often tied to rhythmic structures like meter and tempo. Currently, some parameters are quantized to sync with phrases or measures, ensuring musical coherence, but the balance between real-time responsiveness

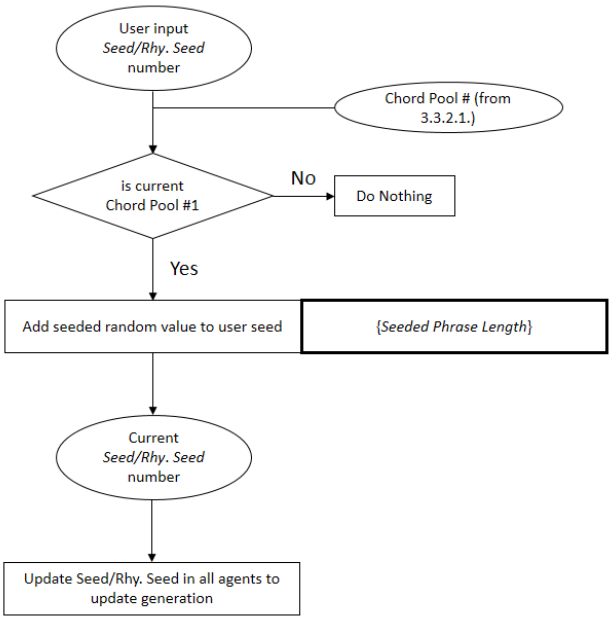


Figure 7. Seeded Phrase Generator. It generates seeds for rhythm and pitch, changing music content per chord or phrase and provides improvisational passages with 0-seeded values.

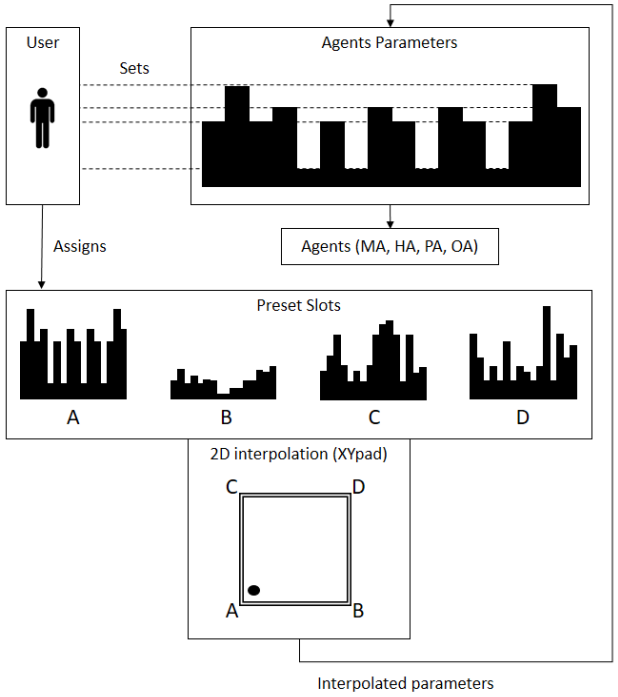


Figure 6. Multi-Parameter Preset Interpolator. It manages agent parameters and presets, and also interpolates among presets, which facilitates seamless musical transitions and allows saving and assigning multi-parameter presets to themes (XY pads' corners).

and phrase integrity is an ongoing challenge. The system's control of resolution for musical changes also poses issues. Given music's discrete nature, using full MIDI ranges for certain parameters can result in no perceptible change, or conversely, abrupt musical shifts. This is partly mitigated by phrasing quantization and gradual transitions, but managing multiple simultaneous parameter changes remains complex. Integrating PAMG into game development raises several questions, including how it aligns with existing authoring systems, handles various game and music genres, and leverages automation to create value. Two primary concerns are its integration with a composer's aesthetic and its method to handle samples and timbre. The former could involve analyzing and translating existing game music to inform PAMG's outputs, possibly using machine learning or Markov chains. The latter, related to licensing and idiomatic textures, might require new approaches in sample management or re-synthesis. PAMG's future could also see composers using it more as an instrument, crafting adaptive generative music with clear authorship in the framework of co-creativity [15].

PAMG's approach aims to augment the granularity, diversity, and dynamics of adaptive game music setups, offering new creative and experiential possibilities in game music composition.

References

[1] Amir Zaib Abbasi, Ding Hooi Ting, and Helmut Hlavacs. 2017. Engagement in Games: Developing an Instrument to Measure Consumer Videogame Engagement and Its Validation. https://doi.org/10.1155/2017/7363925 ISSN: 1687-7047 Pages: e7363925 Publisher: Hindawi Volume: 2017.

[2] Charles Ames. 1987. Automated Composition in Retrospect: 1956-1986. *Leonardo* 20, 2 (1987), 169–185. https://doi.org/10.2307/1578334 Publisher: The MIT Press.

[3] Charles Ames. 1989. The Markov Process as a Compositional Model: A Survey and Tutorial. *len Leonardo* 22, 2 (1989), 175–187. OCLC: 30366918912817.

[4] Grégoire Carpentier, Eric Daubresse, Marc Garcia Vitoria, Kenji Sakai, and Fernando Villanueva Carratero. 2012. Automatic Orchestration in Practice. *Computer Music Journal* 36, 3 (2012), 24–42. https://www.jstor.org/stable/23254380 Publisher: The MIT Press.

[5] Thomas Street Christensen. 2002. *The Cambridge history of Western music theory.* Cambridge, U.K. ; New York : Cambridge University Press. http://archive.org/details/cambridgehistory00chri

[6] Karen Collins. 2009. An Introduction to Procedural Music in Video Games. *Contemporary Music Review* 28, 1 (2009), 5–15. https://doi.org/10.1080/07494460802663983

[7] N Collins. 2000. Caring for the Instrumentalist in Automatic Orchestration. *Proceedings for Acoustics and Music: Theory and Applications* (2000), 32–38.

[8] David Cope. 1991. *Computers and musical style.* A-R Editions, Madison, Wis. OCLC: 23356215.

[9] David Cope and David Cope. 1996. *Experiments in musical intelligence.* Number v. 12 in The Computer music and digital audio series. A-R Editions, Madison, Wis.

[10] Carl Dahlhaus. 2014. Studies on the Origin of Harmonic Tonality. In *Studies on the Origin of Harmonic Tonality*. Princeton University Press. https://doi.org/10.1515/9781400861316

[11] Elmsley, A, Velardo, V, and Grooves, R. 2017. Deep Adaptation: How Generative Music Affects Engagement and Immersion in Interactive Experiences. https://qmro.qmul.ac.uk/xmlui/bitstream/handle/123456789/30583/ID%3A%2097442919-02C3-424B-9C18-B08D362CCC22.pdf?sequence=1

[12] Laura Ermi and Frans Mäyrä. 2005. Fundamental Components of the Gameplay Experience: Analysing Immersion.

[13] Philippe Esling, Grégoire Carpentier, and Carlos Agon. 2010. Dynamic Musical Orchestration Using Genetic Algorithms and a Spectro-Temporal Description of Musical Instruments. In *Applications of Evolutionary Computation (Lecture Notes in Computer Science)*, Cecilia Di Chio, Anthony Brabazon, Gianni A. Di Caro, Marc Ebner, Muddassar Farooq, Andreas Fink, Jörn Grahl, Gary Greenfield, Penousal Machado, Michael O'Neill, Ernesto Tarantino, and Neil Urquhart (Eds.). Springer, Berlin, Heidelberg, 371–380. https://doi.org/10.1007/978-3-642-12242-2_38

[14] Paul Gilreath and Jim Aikin. 2004. *The Guide To MIDI Orchestration* (3rd edition ed.). Musicworks, Marietta, GA.

[15] Artemi-Maria Gioti. 2021. Artificial Intelligence for Music Composition. In *Handbook of Artificial Intelligence for Music: Foundations, Advanced Approaches, and Developments for Creativity*, Eduardo Reck Miranda (Ed.). Springer International Publishing, Cham, 53–73. https://doi.org/10.1007/978-3-030-72116-9_3

[16] Roni Y. Granot and Zohar Eitan. 2011. Musical Tension and the Interaction of Dynamic Auditory Parameters. *Music Perception* 28, 3 (Feb. 2011), 219–246. https://doi.org/10.1525/mp.2011.28.3.219 Publisher: University of California Press.

[17] Mark Grimshaw and Gareth Schott. 2008. A Conceptual Framework for the Analysis of First-Person Shooter Audio and its Potential Use for Game Engines. *International Journal of Computer Games Technology* 2008 (Jan. 2008), e720280. https://doi.org/10.1155/2008/720280 Publisher: Hindawi.

[18] Christopher Hasty. 2020. *Meter as Rhythm: 20th Anniversary Edition* (new to this edition:, new to this edition: ed.). Oxford University Press, Oxford, New York.

[19] Hermann von Helmholtz. 1885. *On the Sensations of Tone as a Physiological Basis for the Theory of Music*. Longmans, Green. Google-Books-ID: GwE6AAAAIAAJ.

[20] Ronald G. Heyduk. 1975. Rated preference for musical compositions as it relates to complexity and exposure frequency. *Perception & psychophysics* 17, 1 (1975), 84–90. https://doi.org/10.3758/BF03204003

[21] Lejaren Hiller and Leonard M. (Leonard Maxwell) Isaacson. 1959. *Experimental music; composition with an electronic computer*. New York, McGraw-Hill. http://archive.org/details/experimentalmusi00hill

[22] David Huron. 2006. *Sweet Anticipation: Music and the Psychology of Expectation*. The MIT Press. https://doi.org/10.7551/mitpress/6575.001.0001

[23] Patrick Edward Hutchings and Jon McCormack. 2020. Adaptive Music Composition for Games. *IEEE Transactions on Games* 12, 3 (Sept. 2020), 270–280. https://doi.org/10.1109/TG.2019.2921979

[24] Charlene Jennett, Anna L. Cox, Paul Cairns, Samira Dhoparee, Andrew Epps, Tim Tijs, and Alison Walton. 2008. Measuring and defining the experience of immersion in games. *International Journal of Human-Computer Studies* 66, 9 (Sept. 2008), 641–661. https://doi.org/10.1016/j.ijhcs.2008.04.004

[25] Kristoffer Jensen and David G. Hebert. 2016. Evaluation and Prediction of Harmonic Complexity Across 76 Years of Billboard 100 Hits. In *Music, Mind, and Embodiment (Lecture Notes in Computer Science)*, Richard Kronland-Martinet, Mitsuko Aramaki, and Sølvi Ystad (Eds.). Springer International Publishing, Cham, 283–296. https://doi.org/10.1007/978-3-319-46282-0_18

[26] Jiulin Zhang and Xiaoqing Fu. 2015. The Influence of Background Music of Video Games on Immersion. *Journal of Psychology & Psychotherapy* 5, 4 (2015), 191. https://doi.org/10.4172/2161-0487.1000191

[27] Michael Keith. 1991. *From Polychords to Pólya: Adventures in Musical Combinatorics*. Vinculum Press. Google-Books-ID: s5BJAAAACAAJ.

[28] Stefan M. Kostka. 2013. *Tonal harmony: with an introduction to twentieth-century music* (7th ed. ed.). McGraw-Hill, New York.

[29] Carol L. Krumhansl. 2001. *Cognitive Foundations of Musical Pitch*. Oxford University Press. https://doi.org/10.1093/acprof:oso/9780195148367.001.0001

[30] Mikael Laurson and Mika Kuuskankare. 2007. A Constraint Based Approach to Musical Textures and Instrumental Writing. https://www.semanticscholar.org/paper/A-Constraint-Based-Approach-to-Musical-Textures-and-Laurson-Kuuskankare/3ad6ee366d7e4d526dbf2b0bb76f1f778c8974b5

[31] Fred Lerdahl. 1988. Tonal Pitch Space. *Music Perception: An Interdisciplinary Journal* 5, 3 (1988), 315–349. https://doi.org/10.2307/40285402 Publisher: University of California Press.

[32] Fred Lerdahl and Ray Jackendoff. 1983. *A generative theory of tonal music*. MIT Press, Cambridge, Mass. OCLC: 8729599.

[33] Fred Lerdahl and Carol L. Krumhansl. 2007. Modeling Tonal Tension. *Music Perception: An Interdisciplinary Journal* 24, 4 (2007), 329–366. https://doi.org/10.1525/mp.2007.24.4.329 Publisher: University of California Press.

[34] Alvaro E. Lopez Duarte. 2020. Algorithmic interactive music generation in videogames. *SoundEffects - An Interdisciplinary Journal of Sound and Sound Experience* 9, 1 (Jan. 2020), 38–59. https://doi.org/10.7146/se.v9i1.118245

[35] Alvaro E. Lopez Duarte. 2023. *A Progressive-Adaptive Music Generator for Videogames (PAMG): an Approach to Real-Time Algorithmic Composition*. Ph. D. Dissertation. UC Riverside. https://escholarship.org/uc/item/1zh86812

[36] Michael George Luby. 1996. *Pseudorandomness and cryptographic applications*. Princeton University Press, Princeton, NJ.

[37] Guy Madison and Gunilla Schiölde. 2017. Repeated Listening Increases the Liking for Music Regardless of Its Complexity: Implications for the Appreciation and Aesthetics of Music. *Frontiers in Neuroscience* 11 (2017). https://www.frontiersin.org/articles/10.3389/fnins.2017.00147

[38] Edited by Alex McLean and Roger T. Dean (Eds.). 2018. *The Oxford Handbook of Algorithmic Music*. Oxford University Press, Oxford, New York.

[39] María Navarro-Cáceres, Marcelo Caetano, Gilberto Bernardes, Mercedes Sánchez-Barba, and Javier Merchán Sánchez-Jara. 2020. A Computational Model of Tonal Tension Profile of Chord Progressions in the Tonal Interval Space. *Entropy* 22, 11 (Nov. 2020), 1291. https://doi.org/10.3390/e22111291

[40] Gerhard Nierhaus. 2009. *Algorithmic composition: paradigms of automated music generation*. Springer, Wien ; New York. http://dx.doi.org/10.1007/978-3-211-75540-2

[41] J Norris. 1998. *Markov chains* (1st pbk. ed. ed.). Cambridge University Press, Cambridge UK ;;New York.

[42] George Papadopoulos and Geraint Wiggins. 1999. AI Methods for Algorithmic Composition: A Survey, a Critical View and Future Prospects. *Proceedings from the AISB'99 Symposium on Musical Creativity* (1999).

[43] Richard Parncutt. 1989. *Harmony: a psychoacoustical approach*. Number 19 in Springer series in information sciences. Springer-Verlag, Berlin ;.

[44] David Plans and Davide Morelli. 2012. Experience-Driven Procedural Music Generation for Games. *IEEE Transactions on Computational Intelligence and AI in Games* 4, 3 (Sept. 2012), 192–198. https://doi.org/10.1109/TCIAIG.2012.2212899

[45] Cale Plut and Philippe Pasquier. 2020. Generative music in video games: State of the art, challenges, and prospects. *Entertainment Computing* 33 (March 2020), 100337. https://doi.org/10.1016/j.entcom.2019.100337

[46] Cale Plut, Philippe Pasquier, Jeff Ens, and Renaud Bougueng Tchemeube. 2022. *PreGLAM-MMM: Application and evaluation of affective adaptive generative music in video games.* https://doi.org/10.1145/3555858.3555947 Pages: 11.

[47] Dirk-Jan Povel and Peter Essens. 1985. Perception of Temporal Patterns. *Music Perception: An Interdisciplinary Journal* 2, 4 (1985), 411–440. https://doi.org/10.2307/40285311 Publisher: University of California Press.

[48] David Psenicka. 2003. SPORCH: An Algorithm for Orchestration Based on Spectral Analyses of Recorded Sounds. (Jan. 2003).

[49] Robert Rowe. 1993. *Interactive music systems: machine listening and composing.* MIT Press, Cambridge, Mass.

[50] Germán Ruiz Marcos. 2022. Tension-driven Automatic Music Generation. (2022). https://doi.org/10.21954/OU.RO.000142CF Publisher: The Open University.

[51] E. Glenn Schellenberg, Isabelle Peretz, and Sandrine Vieillard. 2008. Liking for happy- and sad-sounding music: Effects of exposure. *Cognition and Emotion* 22, 2 (Feb. 2008), 218–237. https://doi.org/10.1080/02699930701350753 Publisher: Routledge _eprint: https://doi.org/10.1080/02699930701350753.

[52] Marco Scirea, Julian Togelius, Peter Eklund, and Sebastian Risi. 2017. Affective evolutionary music composition with MetaCompose. *Genetic Programming and Evolvable Machines* 18, 4 (Dec. 2017), 433–465. https://doi.org/10.1007/s10710-017-9307-y

[53] I. Shmulevich and D.-J. Povel. 1998. Rhythm complexity measures for music pattern recognition. In *1998 IEEE Second Workshop on Multimedia Signal Processing (Cat. No.98EX175).* 167–172. https://doi.org/10.1109/MMSP.1998.738930

[54] Mark J. Steedman. 1984. A Generative Grammar for Jazz Chord Sequences. *Music Perception* 2, 1 (Oct. 1984), 52–77. https://doi.org/10.2307/40285282

[55] Michael Sweet. 2015. *Writing interactive music for video games a composer's guide.* Addison-Wesley, Upper Saddle River, NJ; Boston; Indianapolis; San Francisco; New York; Toronto; Montreal; London; Munich; Paris; Madrid; Capetown; Sydney; Tokyo; Singapore; Mexico City. OCLC: 946779709.

[56] Kıvanç Tatar and Philippe Pasquier. 2019. Musical agents: A typology and state of the art towards Musical Metacreation. *Journal of New Music Research* 48, 1 (Jan. 2019), 56–105. https://doi.org/10.1080/09298215.2018.1511736 Publisher: Routledge _eprint: https://doi.org/10.1080/09298215.2018.1511736.

[57] David Temperley. 2004. *The Cognition of Basic Musical Structures.* MIT Press. Google-Books-ID: IDoLEvTQuewC.

[58] Joseph M. Thomassen. 1982. Melodic accent: Experiments and a tentative model. *The Journal of the Acoustical Society of America* 71, 6 (June 1982), 1596–1605. https://doi.org/10.1121/1.387814 Publisher: Acoustical Society of America.

[59] Julian Togelius, Georgios N. Yannakakis, Kenneth O. Stanley, and Cameron Browne. 2011. Search-Based Procedural Content Generation: A Taxonomy and Survey. *IEEE Transactions on Computational Intelligence and AI in Games* 3, 3 (Sept. 2011), 172–186. https://doi.org/10.1109/TCIAIG.2011.2148116 Conference Name: IEEE Transactions on Computational Intelligence and AI in Games.

[60] G. Toussaint. 2012. The Pairwise Variability Index as a Tool in Musical Rhythm Analysis. https://www.semanticscholar.org/paper/The-Pairwise-Variability-Index-as-a-Tool-in-Musical-Toussaint/abecf85144a253305a01c77d85f19bbddd22accd

[61] Godfried T. Toussaint and Konstantinos Trochidis. 2018. On Measuring the Complexity of Musical Rhythm. In *2018 9th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON).* 753–757. https://doi.org/10.1109/UEMCON.2018.8796634

[62] Dmitri Tymoczko. 2023. *Tonality: An Owner's Manual.* Oxford University Press, Incorporated, Oxford, UNITED STATES. http://ebookcentral.proquest.com/lib/ucr/detail.action?docID=7294520

[63] Paul C. Vitz. 1964. Preferences for rates of information presented by sequences of tones. *Journal of experimental psychology* 68, 2 (1964), 176–183. https://doi.org/10.1037/h0043402 Place: United States Publisher: American Psychological Association.

[64] Peter Vorderer and Jennings Bryant (Eds.). 2006. *Playing Video Games: Motives, Responses, and Consequences* (1st edition ed.). Routledge, Mahwah, N.J.

[65] Lei Wang, Ziyi Zhao, Hanwei Liu, Junwei Pang, Yi Qin, and Qidi Wu. 2023. A Review of Intelligent Music Generation Systems. https://doi.org/10.48550/arXiv.2211.09124 arXiv:2211.09124 [cs, eess].

[66] Alexander Wharton and Karen Collins. 2011. Subjective Measures of the Influence of Music Customization on the Video Game Play Experience: A Pilot Study. *Game Studies* 11, 2 (May 2011). https://gamestudies.org/1102/articles/wharton_collins

[67] T. Winkler. 2001. *Composing Interactive Music: Techniques and Ideas Using Max.* The MIT Press. https://www.biblio.com/book/composing-interactive-music-techniques-ideas-using/d/1567101032

[68] Richard T.A. Wood, Mark D. Griffiths, and Adrian Parke. 2007. Experiences of Time Loss among Videogame Players: An Empirical Study. *CyberPsychology & Behavior* 10, 1 (Feb. 2007), 38–44. https://doi.org/10.1089/cpb.2006.9994

[69] René Wooller and Andrew R. Brown. 2005. Investigating morphing algorithms for generative music. In *Third Iteration: Third International Conference on Generative Systems in the Electronic Arts, Melbourne, Australia.*

Received 2024-06-02; accepted 2024-07-02

Demo: Progressive-Adaptive Music Generator (PAMG) and the Trial Game

Alvaro Eduardo Lopez Duarte
University of California, Riverside, USA
Riverside, California, USA
alvaro.lopez@ucr.edu

Figure 1. The Trial Game

Abstract

The present Demo is submitted as a companion of the paper "A Progressive-Adaptive Music Generator: an Approach to Interactive Procedural music for Videogames". PAMG is a software able to generate music in real time adapting and progressing to game variables. The experience demonstrates PAMG through a gameplay session of the Trial Game, a first-person shooter (FPS) specifically designed to illustrate PAMG capabilities. It starts by a brief presentation of the model, its motivation, architecture, and interface. Then, a description of the Trial Game and the preliminary evaluation test with the main discussion points that stem from its results. After the introduction (5-7 minutes), a person from the audience will be invited to play in a table in front of a big screen using a conventional computer keyboard and a mouse. The audience will be able to see the gameplay, listen to the progressive-adaptive music, and compare the result with the clip-based implementation (CBI) that was used for the test. In an additional monitor screen, the PAMG's development interface will display the parameters and activity in real time.

CCS Concepts: • **Computing methodologies** → Modeling methodologies; • **Applied computing** → Sound and music computing.

Keywords: AI Music, Game Music, Procedural Music

ACM Reference Format:
Alvaro Eduardo Lopez Duarte . 2024. Demo: Progressive-Adaptive Music Generator (PAMG) and the Trial Game. In *Proceedings of the 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM '24), September 2, 2024, Milan, Italy*. ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3677996.3678296

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
FARM '24, September 2, 2024, Milan, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1099-5/24/09
https://doi.org/10.1145/3677996.3678296

1 Introduction

In the present Demo, PAMG is showcased in comparison with a conventional implementation of music sequences in

which layering and re-sequencing are implemented through sound design platforms like Audiokinetic's Wwise.

The experience demonstrates the *progressive-adaptive* property, where values correlate to complexity in music constructions within traditional tonal systems and music cognition studies. For example, PAMG offers access to range, register, harmonic tension, and harmonic/rhythmic complexity, among others, through a relative increase/decrease control. The explicit use of humanly-understandable parameters is an adopted approach in response to the concealed nature of machine learning models. The demonstration illustrates the model's capability of producing a cohesive music flow from a semi-unpredictable series of events using a custom-design FPS action videogame that controls PAMG parameters. For full project documentation see [1, 2].

2 Program

The experience will be divided in three sections:

2.1 Description and Motivation

This introduction includes a brief description of the project, its motivation and research field, and also and overview of PAMG and the Trial Game. It will also enunciate the main findings of the experiment that compared PAMG and CBI, and future work. This section will last between 5 and 7 minutes.

- Motivation:
  In game development, characters, environments, and events are linked through gameplay variables, with real-time interactions. This is a called procedural generation and supports multi-linear storytelling. Immersive gameplay combines elements driven by surprise, satisfaction, and challenge, requiring varied and responsive musical support. Game developers frequently utilize pre-produced audio sequences, prioritizing sound quality and balancing storage constraints and auditory variety. Current technologies employ these audio clips to layer, re-sequence, and remix music material to be able to diversify otherwise linear music segments. Nevertheless, satiation caused by playback repetition has been reported especially in long gaming sessions. Real-time music generation, like smaller buffer sizes in digital audio, theoretically would offer higher responsiveness and adaptability. PAMG responds to these needs by provides efficient, real-time musical generation tied to gameplay variables. To illustrate, a first person shooter game (the Trial Game) is designed to control PAMG, and also a conventional music implementation is designed for a comparative test.
- Algorithmic Composition in Connection with PAMG:
  The current model employs algorithmic methods that are not based on statistical analysis of musical corpora. Instead, it employs stochastic methods and rules rooted on the Western tonal music tradition, constituting a knowledge-based system.
- Multi-agent systems (MAS):
  MAS, known for distributed autonomy and perceptual-action capabilities, resemble musical ensembles in their coordination of independent entities. PAMG exemplifies this by employing agents to create distinct musical features such as melody, harmony, rhythm, and orchestration. These agents function within an ensemble-like network, utilizing a structured input/output distribution for algorithmic communication.
- Melody creation and accents:
  Starting with a clock-triggered pseudo-random number generator (PRNG), PAMG generates melodic content through seeding, a common method in stochastic processes and Markov chains. PAMG uses seeded random walk algorithms for melody and sequence generation and motive cohesion.
  In PAMG uses melodic accents which are used to create the rhythmic structure of other musical events. This pattern applies to pitch-contour nodes in the "dcontour" mode, accommodating listener expectations of periodicity related to meter.
- Tension, complexity and progression:
  PAMG's design features progressive transitions from simple to complex music, guided by quantized tension and complexity within Western tonal harmony, aligning with most commercial game music's incidental style.
  Tension: Musical tension is challenging to quantify, often linked to expectations of form's conclusion or continuation. In tonal harmony, tension is associated with dualities like stability/instability and consonance/dissonance. Lerdahl's "tonal pitch space" calculates psychological distances in music, aligning with GTTM's principles. PAMG reflects increasing tension by progressively adding pitches across 12 levels per chord, mirroring historical music periods' dissonance use.
  Progression Complexity: In tonal harmony, factors like harmonic rhythm, dissonance, and evolution tend to suggest levels of complexity. Inspired by David Cope's Augmented Transition Network (ATN), PAMG manages transitions between chord levels using chord transition probabilities based on the Western tonal system.
  Temporal Complexity: In music, temporal pattern cognition is linked to an internal hierarchical clock. Subsymmetries in rhythm tend to define complexity, which also arises from metric grouping. In PAMG's rhythm generation, complexity increases by progressively displacing and adding onsets. This breaks sub-symmetries allowing progressive complexity increment.

- The test and future work:
  The gameplay experiment compared PAMG's real-time control with traditional clip-based implementations (CBI). Results suggested PAMG was perceived with greater adaptability and responsiveness to gameplay dynamics than CBI but the results are not conclusive. Data analysis highlighted PAMG's strengths in creating seamless transitions and maintaining musical coherence. Test limitations were noted in the length (ability to test satiation) and lack of a priming section to reduce differences between groups. Future research will focus on refining PAMG's algorithmic composition capabilities and stylistic diversity. This includes expanding the system's dynamic range, and integrating idiomatic music behavior analysis. Also the model supports a clean human-centered co-creative tool for music performance and interactive music authoring.

2.2 Gameplay

In this section, a person from the audience will be invited to play the game in the large screen. They will be seated in a table facing the projection seen by the audience and play for as long as the player character is alive or 10 minutes maximum. During this experience, especially if the player has experience in FPS, the presenter will switch the music from PAMG to CBI to allow the audience and player to perceive the difference. When the player dies, another person from the audience will be invited to play. At the same time, the presenter will show PAMG activity through the development interface displayed in an alternate monitor screen. This section will last between 10 and 15 minutes.

2.3 Discussion

In this section, the audience will be invited to participate in Q & A. This will round up the 30-minute program and will last from 5 to 7 minutes. Topics of discussion will include:

- Reception of PAMG's support capabilities to gameplay situations and events and how it managed transitions.
- Comments about PAMG's music generation and the system architecture.
- Debate synthetic music generation, authorship, and the role of PAMG in AI music.

3 Technical Requirements

For the current demonstration the following elements are required:

- Demo Room. A standard space able to accommodate an audience and the presentation equipment disclosed below.
- A projector and large projection screen. It should include a HDMI or any other video input cable with adapter to USB-C.

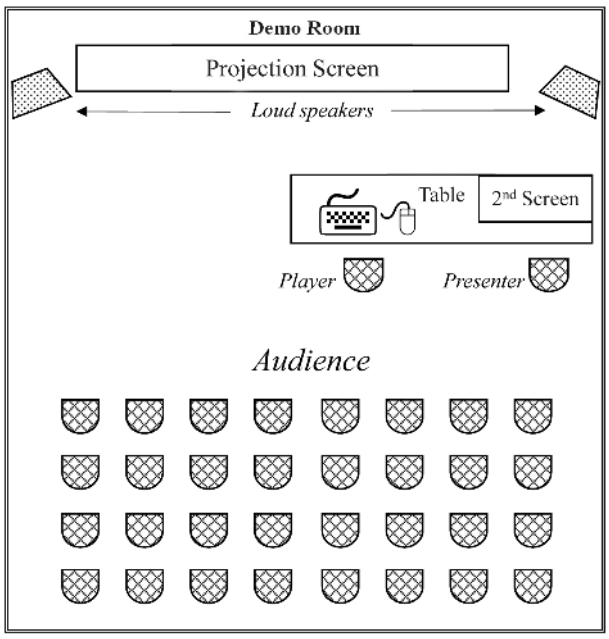


Figure 2. Floor Plan.

- A PA audio system with full range, stereo Loud Speakers with required cables and power source.
- A large table (shown in Figure 2) or two single tables for player and presenter.
- Two Chairs: 1 for the presenter AND 1 for the player. Also chairs for the audience as the space is able to fit.
- A 2nd screen monitor. For example an LCD Display screen monitor with input cable that fits the presenter table.
- Power Strip (should accommodate Type C/Euro Plug) for a computer in addition to the rest of the equipment.
- Miscellaneous: Wifi Internet (four devices need internet access). A standard computer keyboard and mouse for the player.

References

[1] Alvaro E. Lopez Duarte. 2020. Algorithmic interactive music generation in videogames. *SoundEffects - An Interdisciplinary Journal of Sound and Sound Experience* 9, 1 (Jan. 2020), 38–59. https://doi.org/10.7146/se.v9i1.118245

[2] Alvaro E. Lopez Duarte. 2023. *A Progressive-Adaptive Music Generator for Videogames (PAMG): an Approach to Real-Time Algorithmic Composition.* Ph. D. Dissertation. UC Riverside. https://escholarship.org/uc/item/1zh86812

Received 2024-06-02; accepted 2024-07-02

Author Index